

FOUNDAMENTALS IN AI and ML (CSA2001)

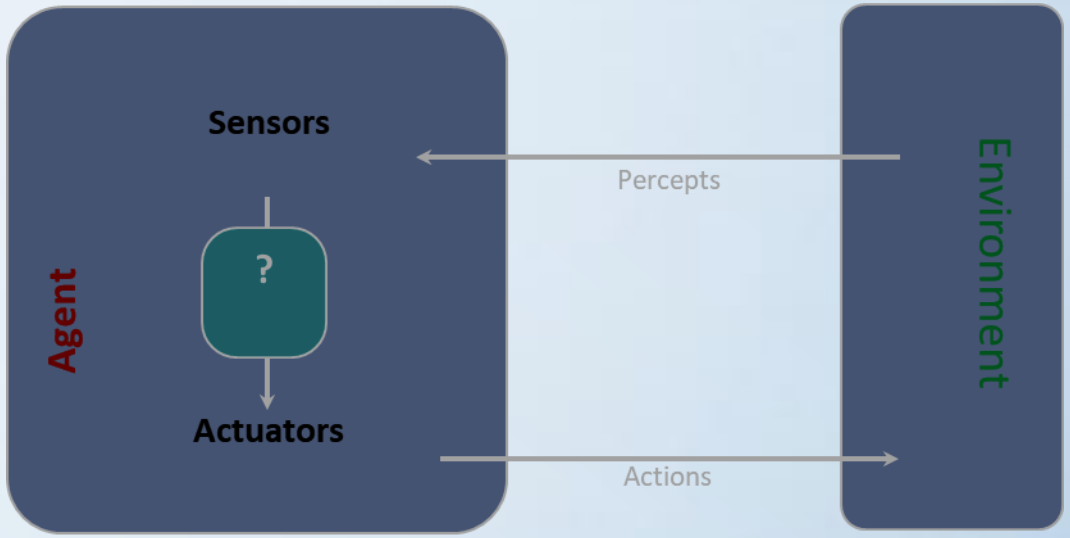
Module-2-part-1

Problem Solving Agent, Problem Formulation, Search Strategies: Uninformed Search Strategies

Dr. Rudra Kalyan Nayak
Senior Assistant Professor
School of Computing Science & Engineering

VIT BHOPAL

Problem Solving Agent



- ✓ When the correct action to take is not immediately obvious, an agent may need to plan ahead: to consider a sequence of actions that form a path to a goal state. Such an agent is called a problem-solving agent, and the computational process it undertakes is called search.

- ➢ **Problem-solving agent -** Goal-based agent
- ➢ **Goal formulation -** is the first step in problem solving.
- ➢ **Problem formulation -** is the process of deciding what actions and states to consider

- ❑ Atomic Agents
  - ○ States are indivisible
  - ○ Searching through the states to reach the goal.

What is a state ?

- ✓ All information about the environment.
- ✓ All information necessary to make a decision for the task at hand.

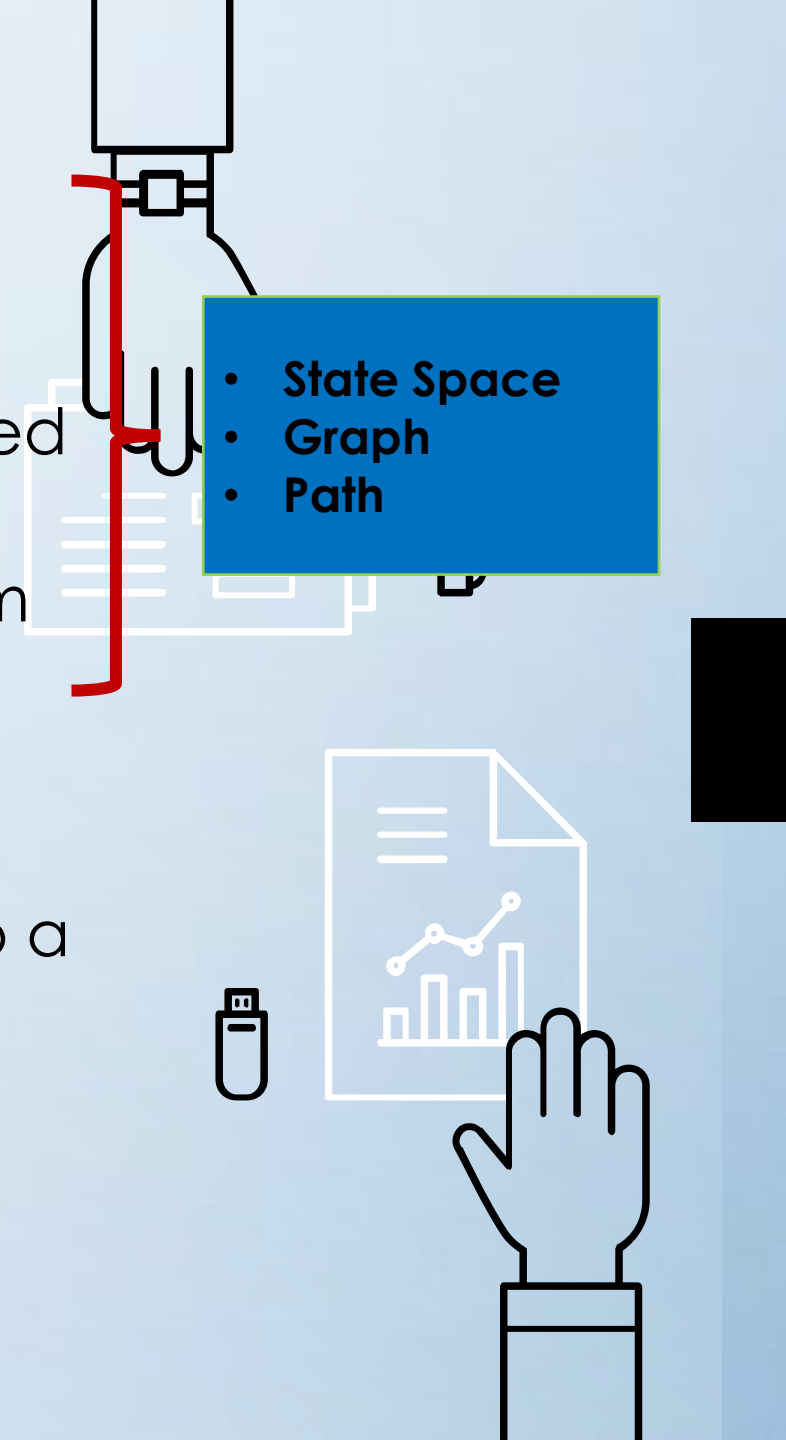
State space search algorithm to solve almost all AI problems …!!



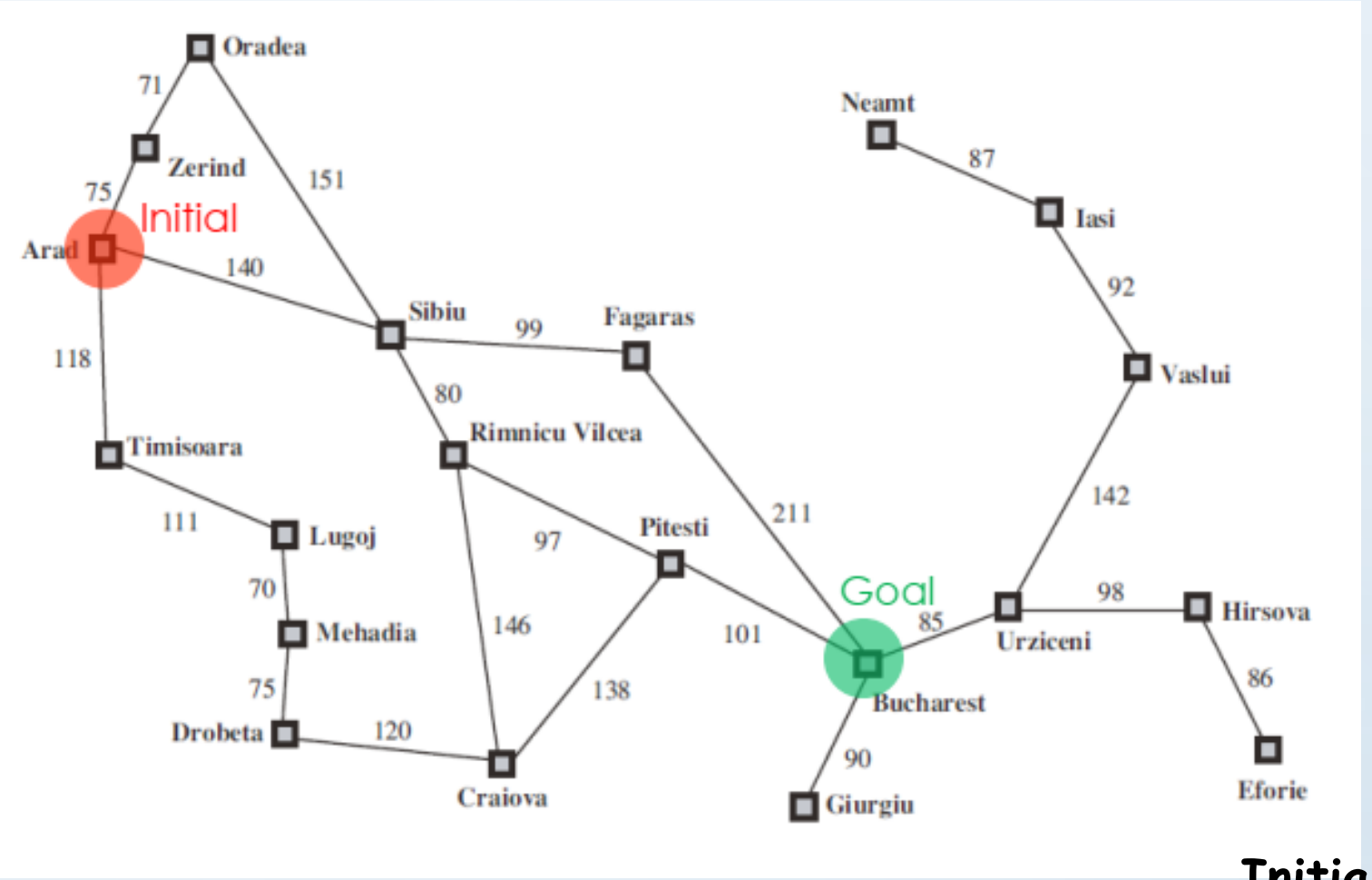
Problem Formulation

Problem can be defined by 5 components:

1. **Initial State**: the state the agent starts
2. **Actions**: the set of operators that can be executed at a state.
3. **Transition model**: returns the state that results from doing an action in a state
4. **Goal test**: determines whether a given state is a goal state
5. **Path Cost**: function that assigns a numeric cost to a path
   - ✓ Step cost: cost of taking a single action

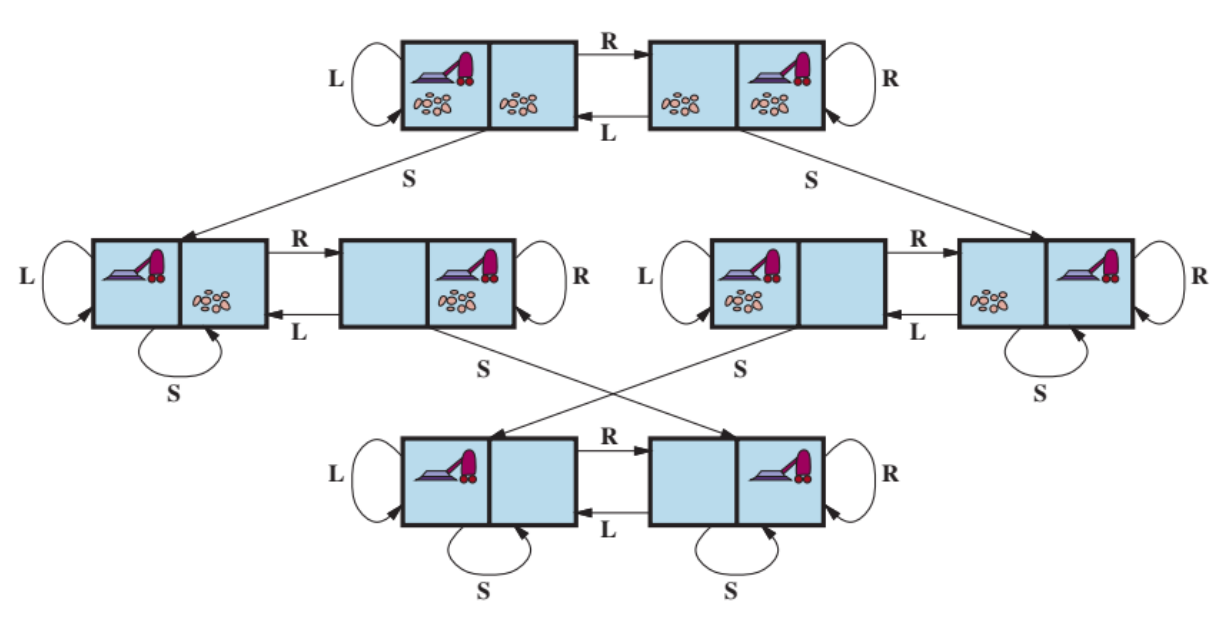


Example: Travelling Sales Man Problem



Initial State: Arad
Actions: Drive(Sibiu),Drive(Timisora)
Goal Test: In(Bucharest)
Path Cost: ?

Example: Toy Vacuum Problem

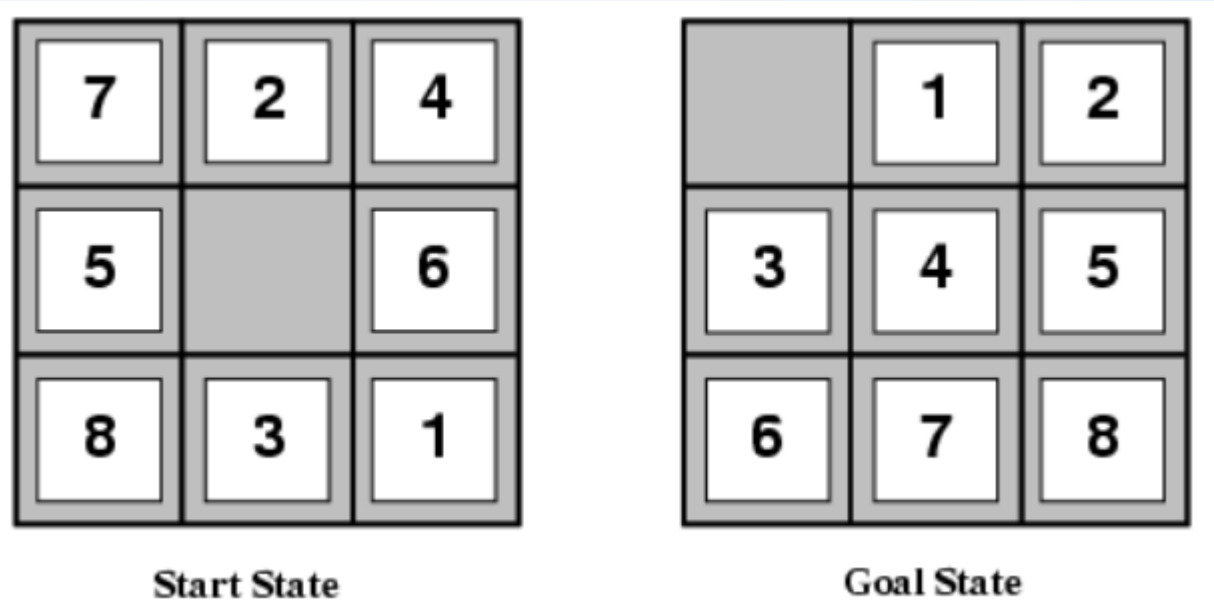


Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮

Partial tabulation of a simple agent function for the vacuum-cleaner world

State: Robot and Dirt Locations
Initial State: Any State
Actions: Left, Right Suck
Goal Test: No Dirt
Path Cost: cost 1 per action?

Example: Eight Puzzle Problem

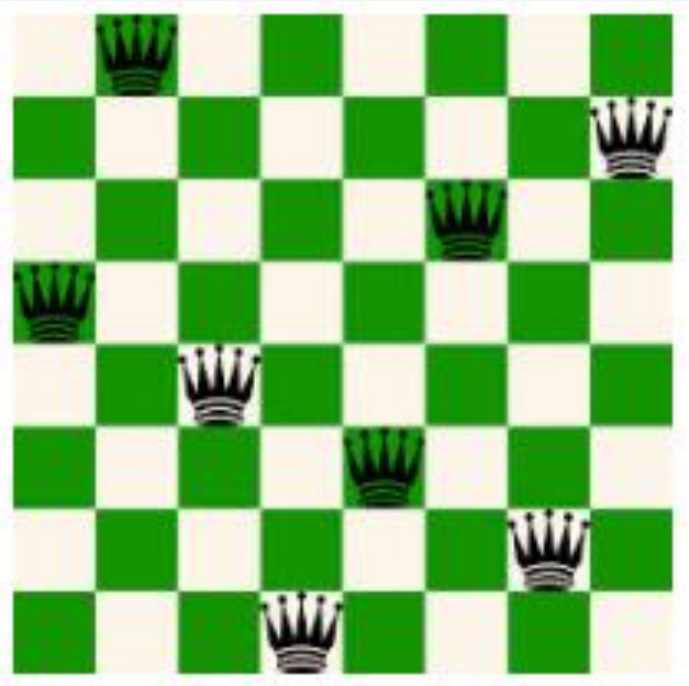


State: Tile Locations
Initial State: A specific tile configuration
Actions: move the blank tile left, right, up or down
Goal Test: tiles are in the required configuration
Path Cost: cost 1 per move?

Note: Optimal solution for an n-puzzle family is NP hard.

A problem is NP-hard if an algorithm for solving it can be translated into one for solving any NP-problem (nondeterministic polynomial time) problem.

Example: Eight Queens Problem



State: Configuration of the Queens

Initial State: Empty board

Actions: Add a queen to the board

Goal Test: configuration with 8 queens on the board with none attacking another

Path Cost: time taken to solve?

Example: Rubik's Cube



State: List of colors on each face

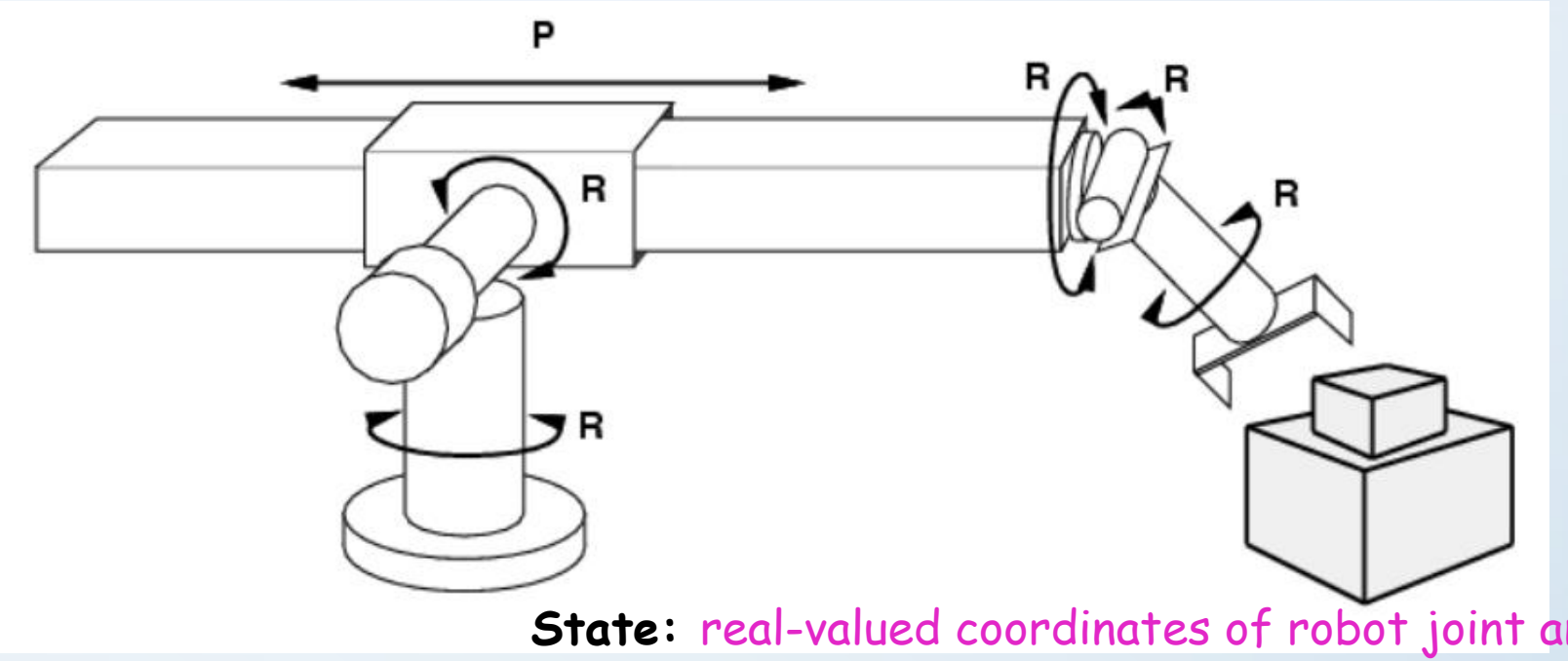
Initial State: A specific color pattern

Actions: rotate a row or column or a face

Goal Test: configuration has the same color on all tiles on every face

Path Cost: cost 1 per move?

Example: Robotic Assembly



State: real-valued coordinates of robot joint angles parts of the object to be assembled

Initial State: A specific position of robot and to be assembled object

Actions: continuous motions of robot joints

Goal Test: complete assembly

Path Cost: time to execute

Example: Water Jug Problem



No Marks on Jug.



- ✓ We can fill the jugs using a pump.
- ✓ We can pour water out of the jug to ground.
- ✓ We can pour water from one jug to another.
- ✓ No measuring device available.

State: (x, y): Liters of water in Jug from 0 to 4.

Initial State: (0,0): Empty jugs

Actions: Empty jug, fill jug, Pour water between jugs

Goal Test: (2, n): 2 Liters in 4-Liter jug.

Path Cost: cost 1 per move?

Rules for Water Jug Problem & State Space Representation for Solution

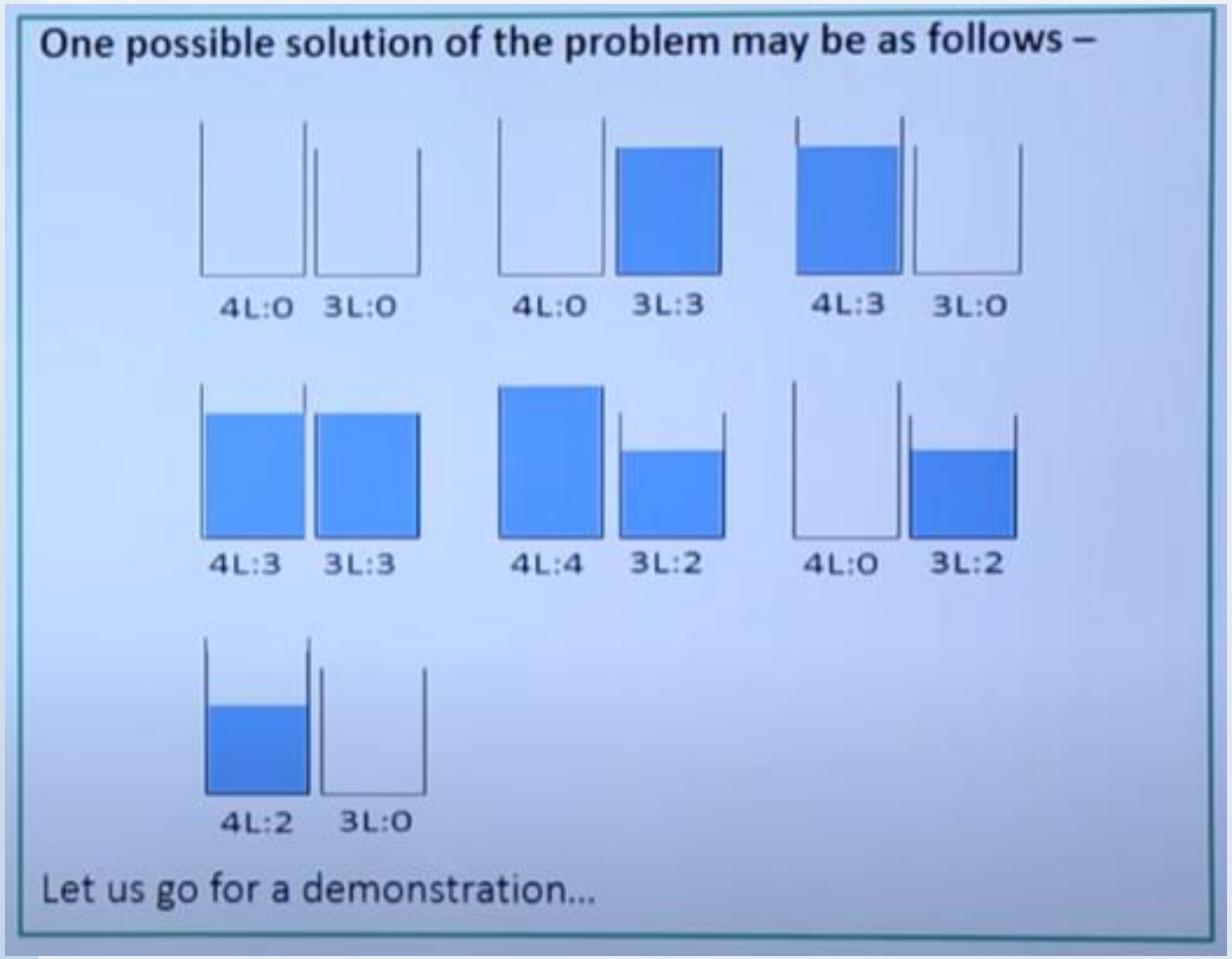
Sl. No.	Current State	Next State	Description
01	(x, y) If x<4	(4,y)	fill the 4 gallon jug
02	(x, y) If y<3	(x,3)	fill the 3 gallon jug
03	(x, y) If x>0	(x-d, y)	pour some water out of the 4-gallon jug.
04	(x, y) If y>0	(x, y-d)	pour some water out of the 3-gallon jug.
05	(x, y) If x>0	(0,y)	empty the 4-gallon jug on the ground
06	(x, y) If y>0	(x,0)	empty the 3-gallon jug on the ground
07	(x, y) If $x + y >= 4$ and y>0	(4,y-(4-x))	pour water from the 3-gallon jug into the 4-gallon,until the 4-gallon jug is full
08	(x, y) If $x + y >= 3$ and x>0	(x-(3-y),3))	pour water from the 4-gallon jug into the 3-gallon, until the 3-gallon jug is full.
09	(x, y) If $x + y <= 4$ and y>0	(x+y,0)	pour all the water from the 3-gallon jug into, the 4-gallon jug.
10	(x, y) If $x + y <= 3$ and x>0	(0,x+y)	pour all the water from the 4-gallon jug into the 3-gallon jug.
11	(0,2)	(2,0)	pour the 2-gallon from the 3-gallon jug into the 4-gallon jug.
12	(2,y)	(0,x)	empty the 2 gallon in the 4 gallon on the ground.

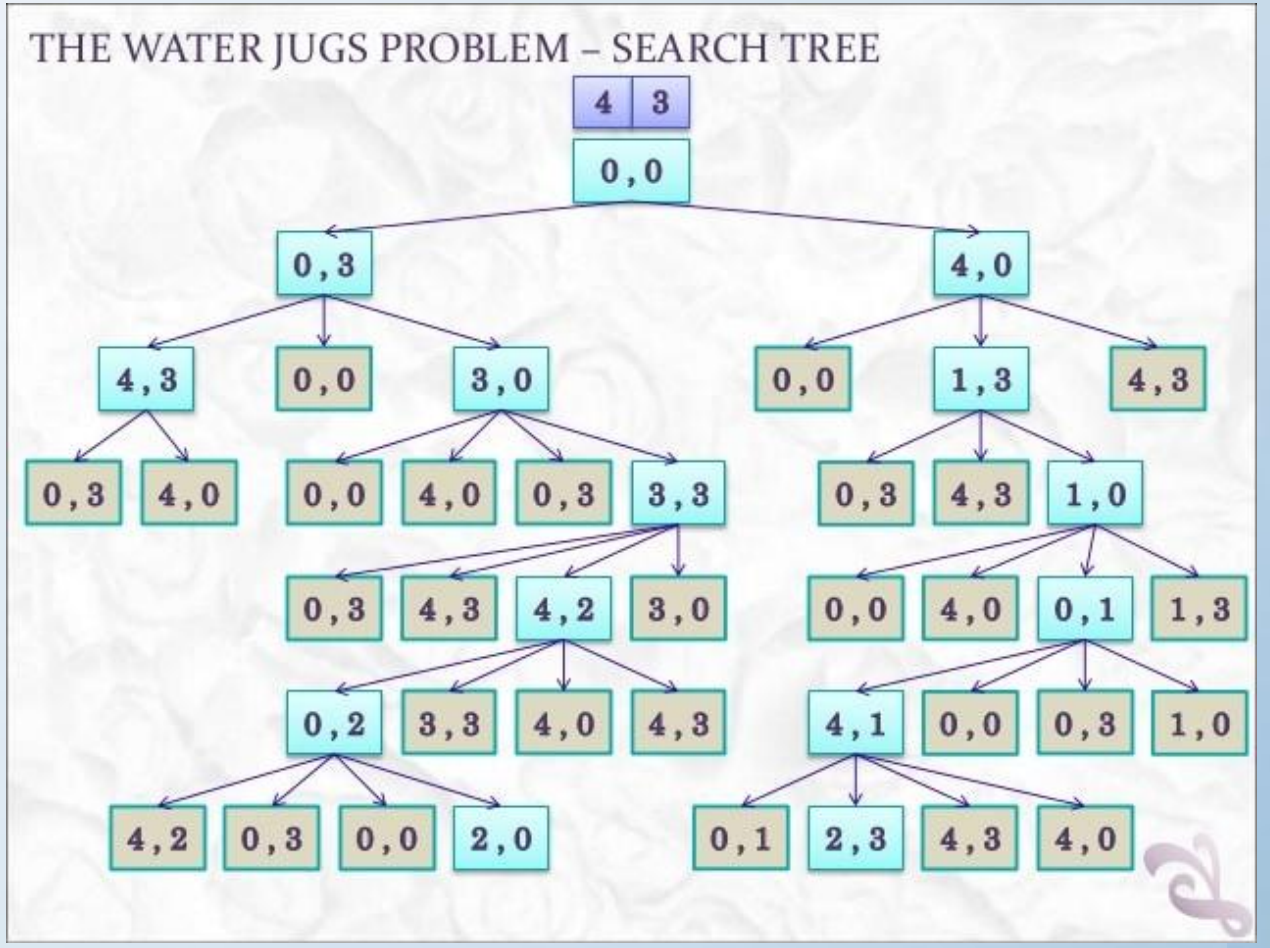
Solution for water jug problem

Gallons in the 4-gallon jug	Gallons in the 3-gallon	Rule Applied
0	0	
0	3	2
3	0	9
3	3	2
4	2	7
0	2	5 or 12
2	0	9 or 11

One solution to the water jug problem.

Water Jugs (4L ,3L)





Production System

Production system consists of:

i. A set of rules, each consisting of a left side (**pattern**) that determines the applicability of rules and the right side that describes the **operations** to be performed.

ii. One or more **knowledge/databases** that contain whatever information is appropriate for the particular task.

iii. A **control strategy** that specifies the order in which rules will be compared to the database a way of resolving the conflicts that arise if several rules matches at once.

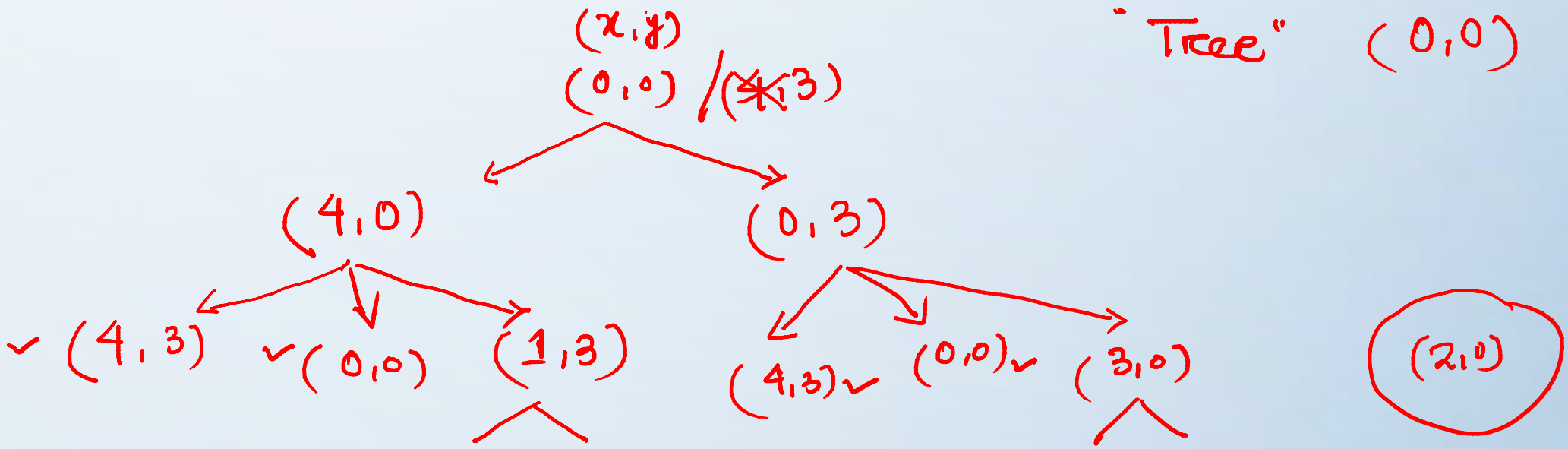
iv. A **rule applier**.

Control Strategies

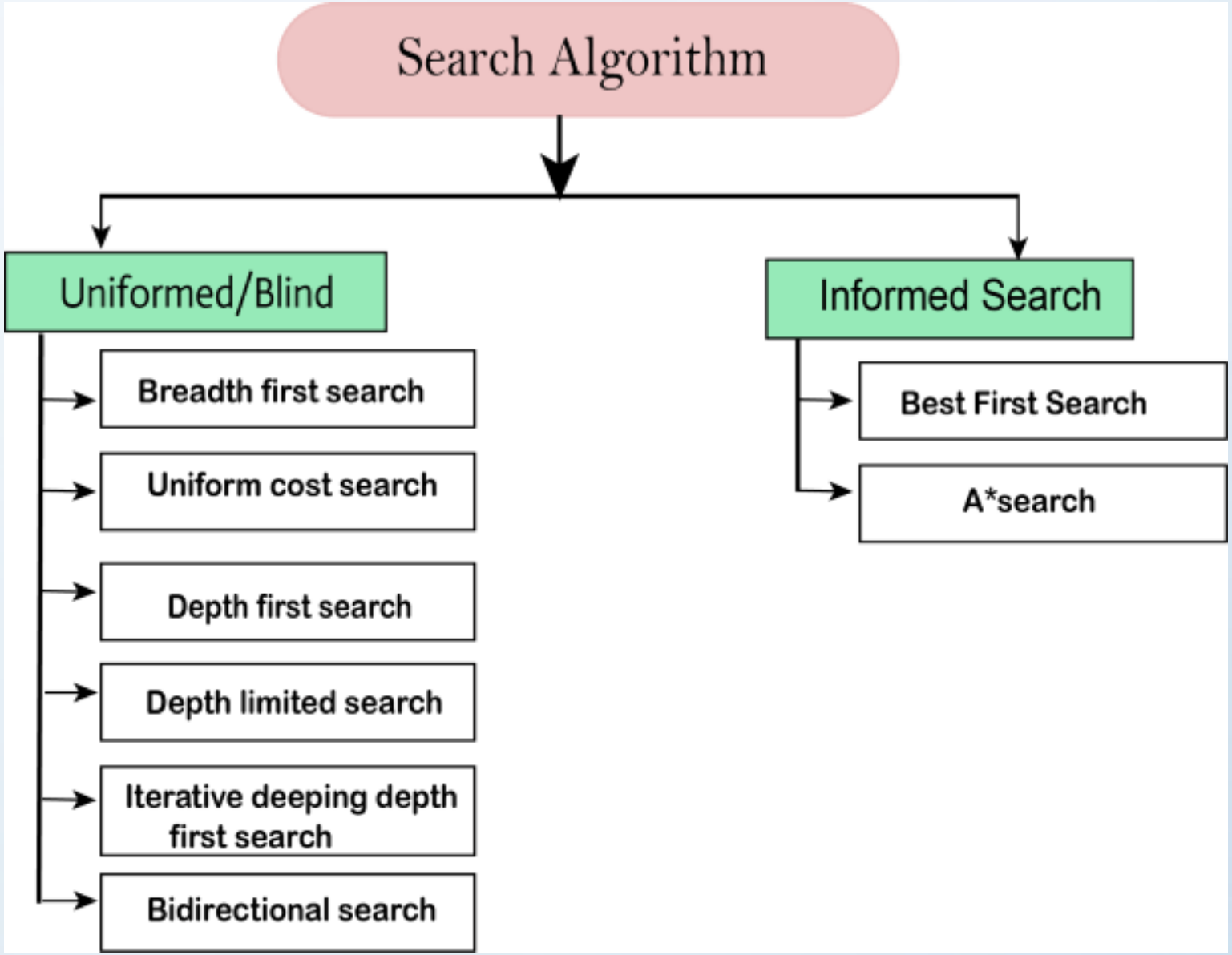
We need to know which rules to be applied next to decide the next state during a search problem. Sometime more than one rule may apply its left side to match the current state:

i. The first requirement of a good control strategy is that it causes motion.

ii. The second requirement of a good control strategy is that it to be systematic.



Search Algorithms in AI



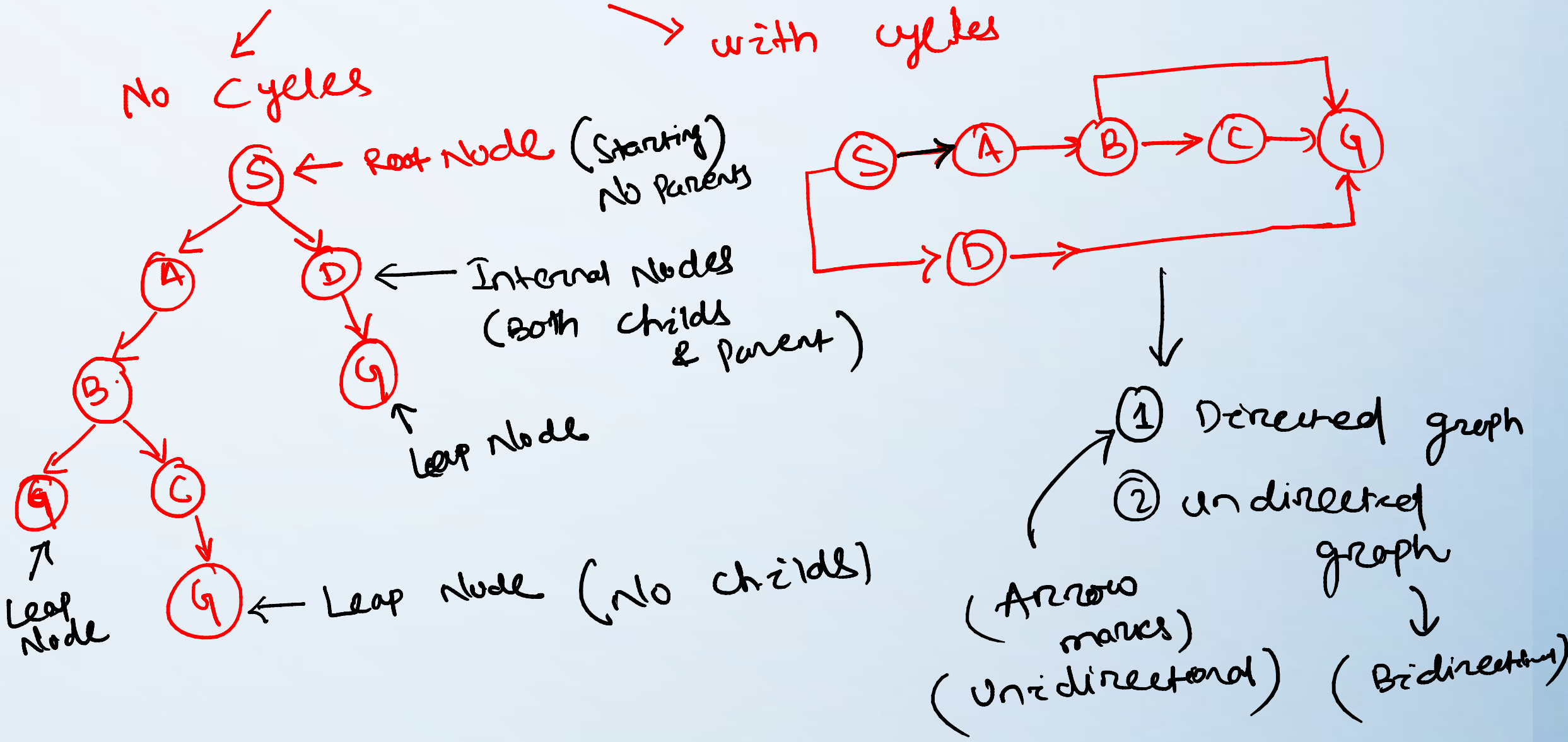
Search strategies

- A search strategy is defined by picking the order of node expansion

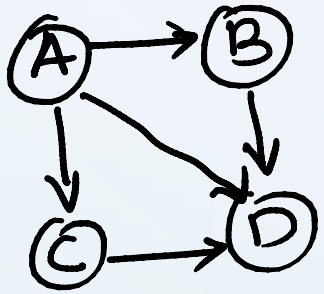
- Strategies are evaluated along the following dimensions:
  - ✓ completeness: does it always find a solution if one exists?
  - ✓ time complexity: number of nodes generated
  - ✓ space complexity: maximum number of nodes in memory
  - ✓ optimality: does it always find a least-cost solution?
  - ✓ systematicity: does it visit every search node at most once?

- Time and space complexity are measured in terms of
  - ✓ ***b***: maximum branching factor of the search tree
  - ✓ ***d***: depth of the least-cost solution
  - ✓ ***m***: maximum depth of the state space (may be ∞)

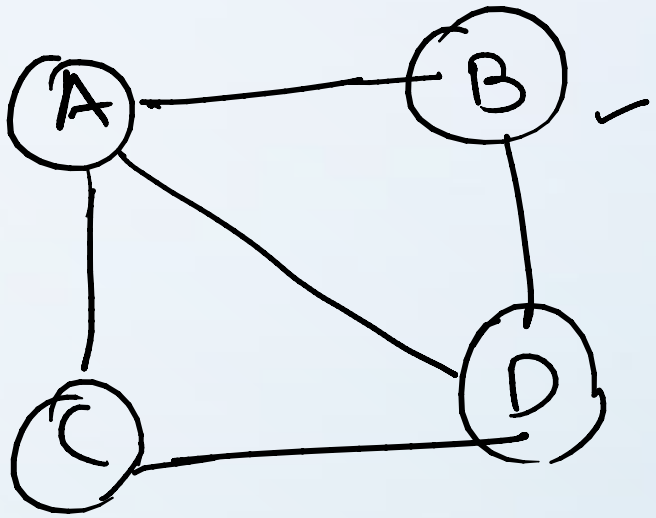
Search Tree vs Graph



Graph & Adjacency matrix :-

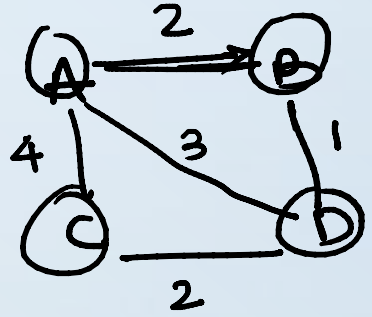


	A	B	C	D
A	0	1	1	1
B	0	0	0	1
C	0	0	0	1
D	0	0	0	0



	A	B	C	D
A	0	1	1	1
B	1	0	0	1
C	1	0	0	1
D	1	1	1	0

"Uniform cost" = 1



Uninformed search strategies

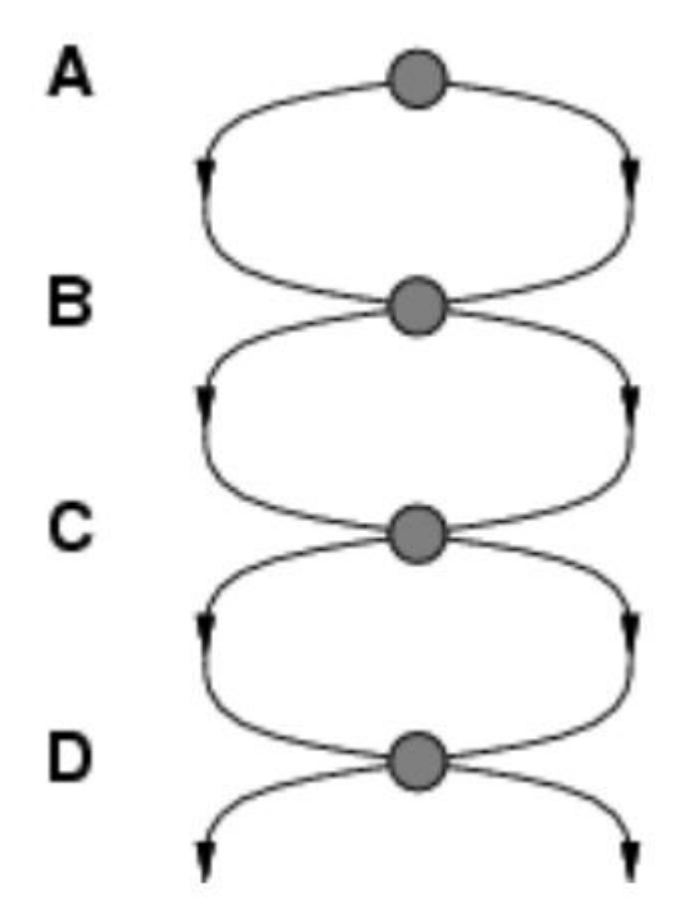
- Uninformed search is a class of general-purpose search algorithms which operates in brute force-way.
- Uninformed search algorithms do not have additional information about state or search space other than how to traverse the tree, so it is also called blind search.

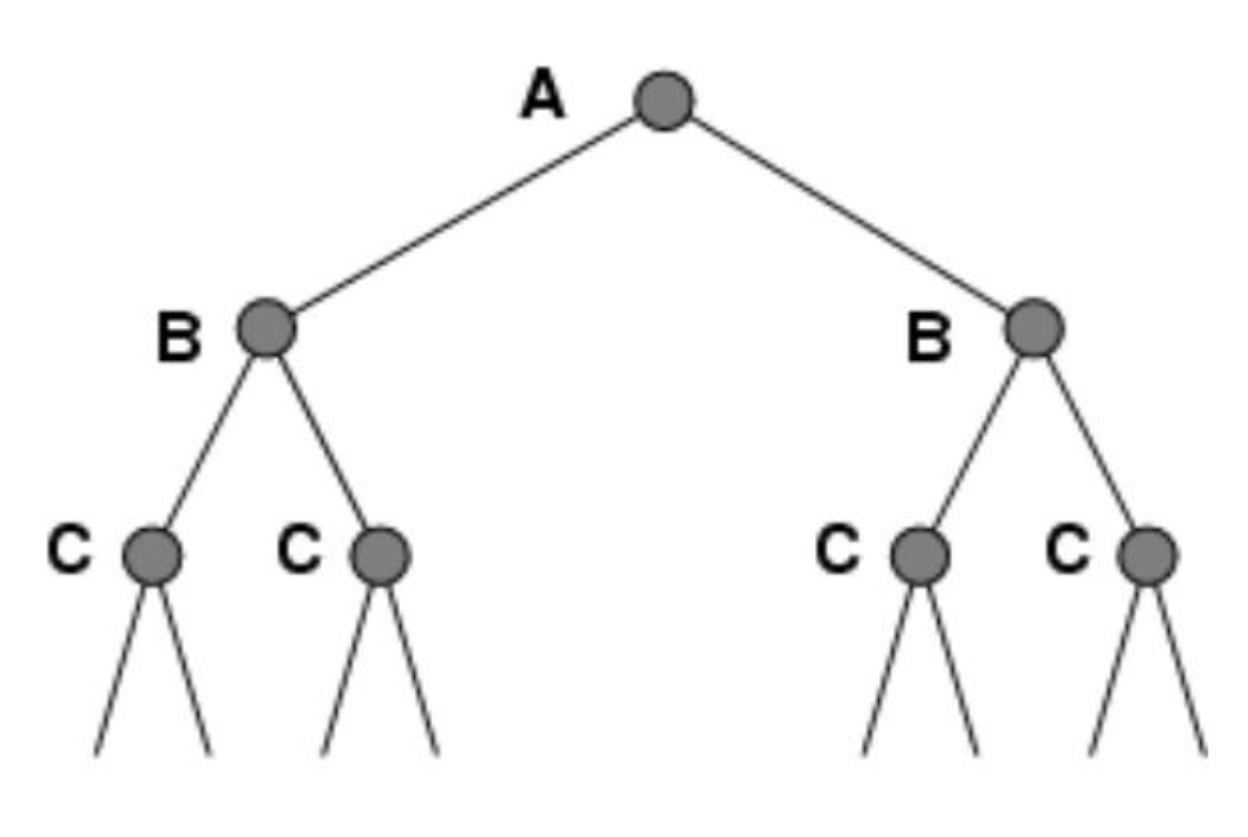


- ✓ Breadth-first search
- ✓ Depth-first search
- ✓ Depth-limited search
- ✓ Iterative deepening search
- ✓ Bidirectional Search

Repeated states

Failure to detect repeated states can turn a linear problem into an exponential one!



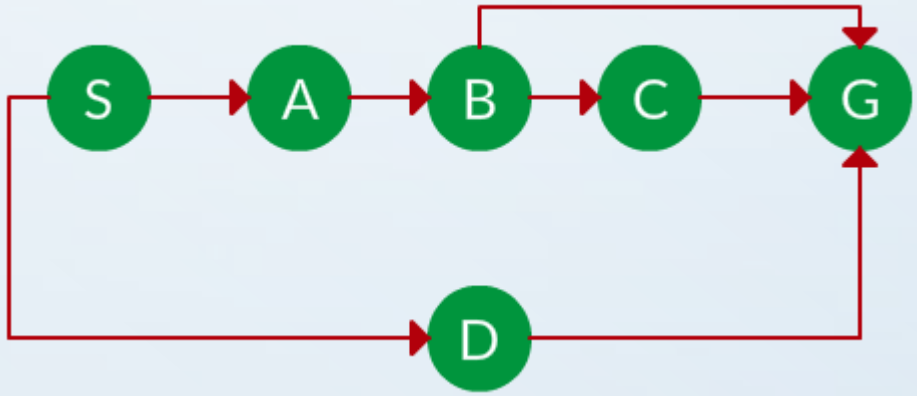


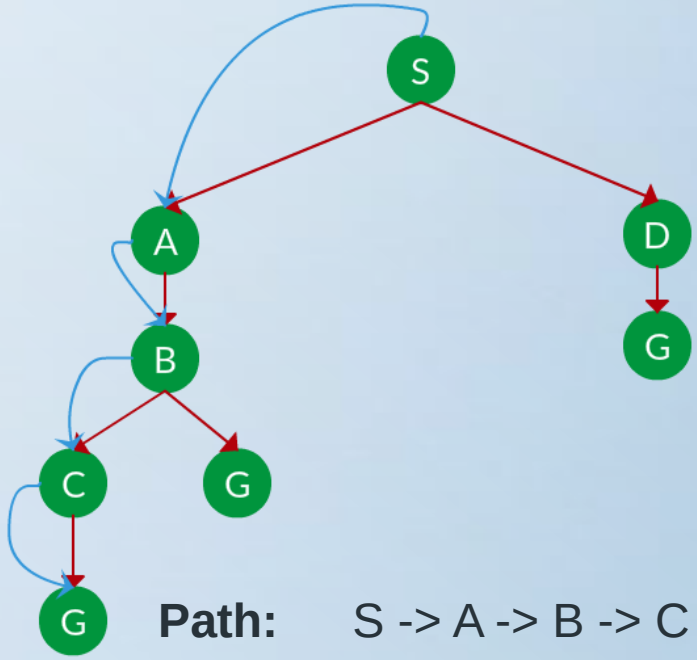
Depth First Search

Depth-first search (DFS) is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking.

Example:

Which solution would DFS find to move from node S to node G if run on the graph below?





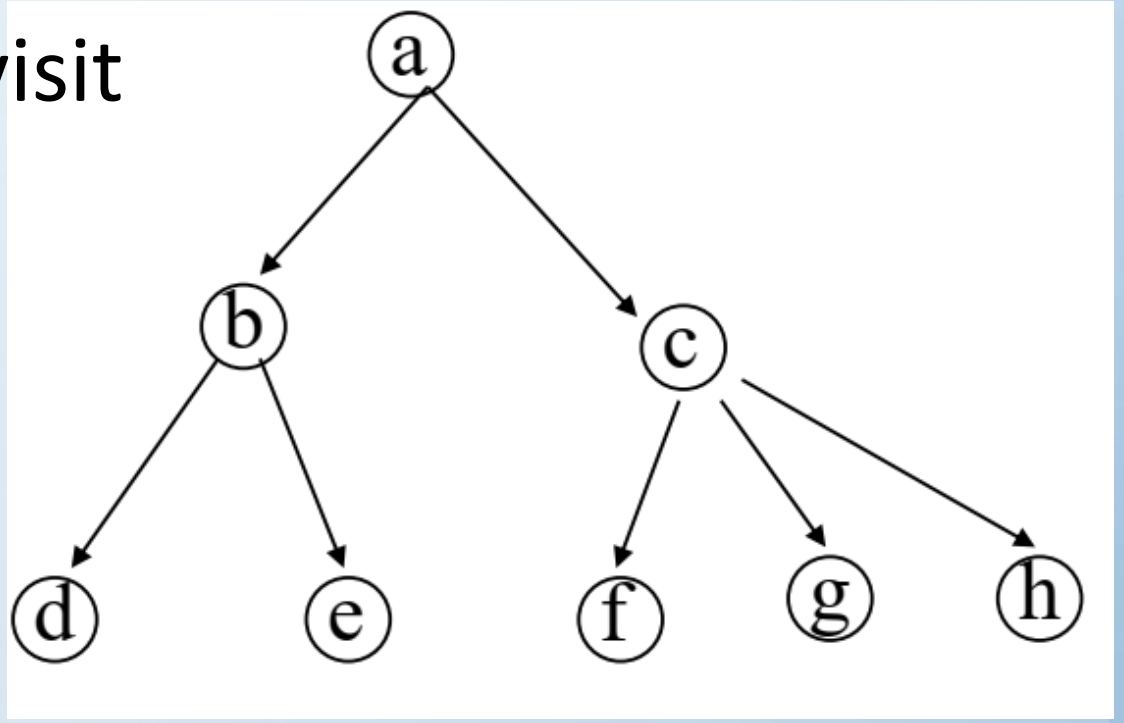
Path: S -> A -> B -> C -> G

Depth First Search

Depth-first search (DFS) is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking.

- Maintain stack of nodes to visit

- Evaluation
  - ✓ Complete? **NO**
  - ✓ Time Complexity? $O(b^m)$
  - ✓ Space Complexity? $O(bm)$



Depth First Search (LIFO)

Pseudocode

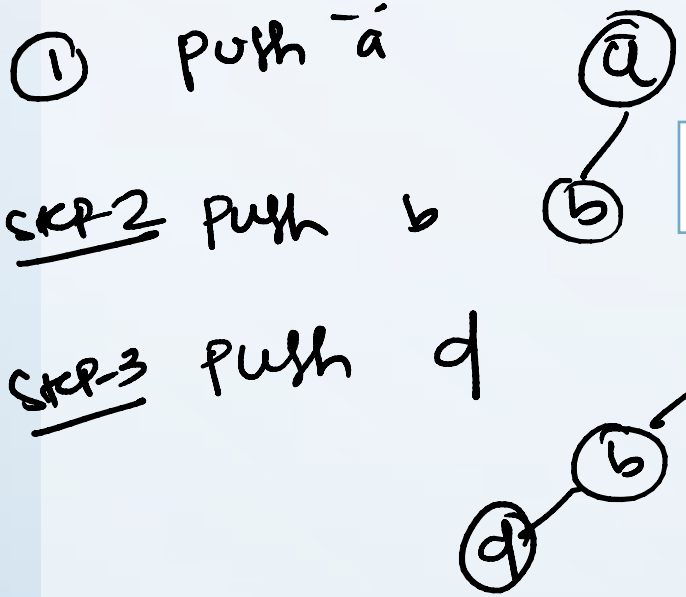
```
DFS(G,v)   ( v is the vertex where the search starts )
      Stack S := {};   ( start with an empty stack )
      for each vertex u, set visited[u] := false;
      push S, v;
      while (S is not empty) do
        u := pop S;
        if (not visited[u]) then
          visited[u] := true;
          for each unvisited neighbour w of uu
            push S, w;
        end if
      end while
    END DFS()
```

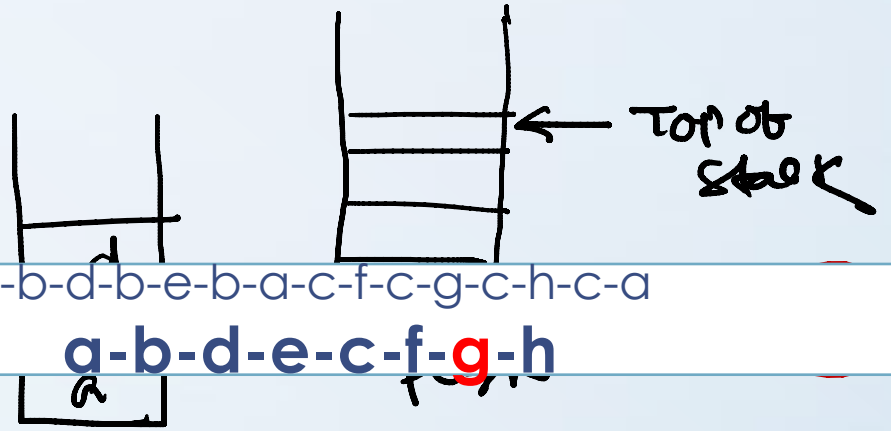
DFS Algorithm Pseudocode

```
procedure DFS(G,v):
    for v ∈ V:
        explored[v] ← false
    end for
    for all v ∈ V do
        if not explored[v]:
            DFS-visit(v)
end procedure

procedure DFS-visit(v):
    explored[v] ← true
    for u ∈ adj(v):
        if not explored[u]:
            DFS-visit(u)
        end if
    end for
end procedure
```

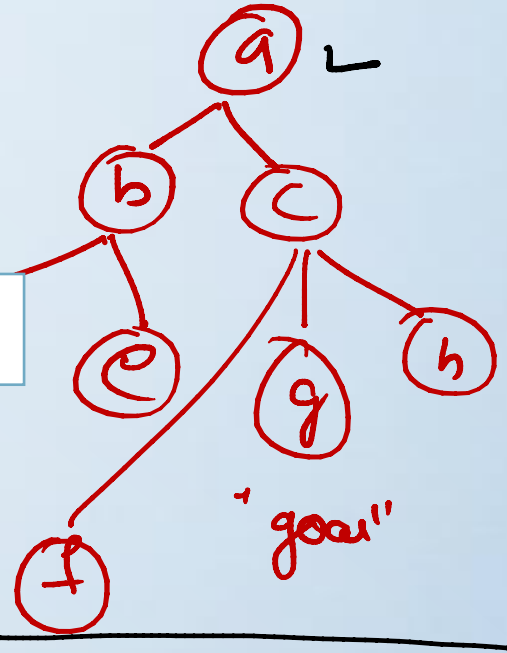
DFS and Stack



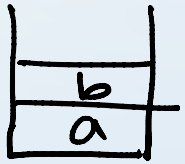


a-b-d-b-e-b-a-c-f-c-g-c-h-c-a

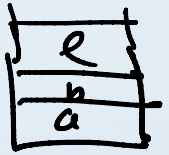
a-b-d-e-c-f-g-h



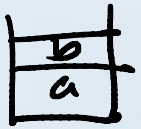
step-04 pop d



step-05 push e

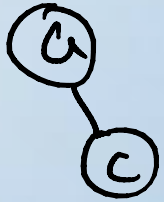


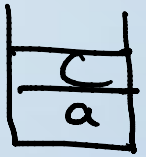
step-06 pop e



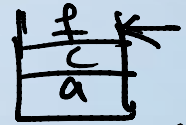
step-07 pop b

step-08 push c





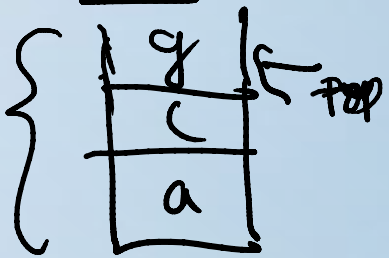
step-09 push f





step-10 :- pop f

step-11 :- push g

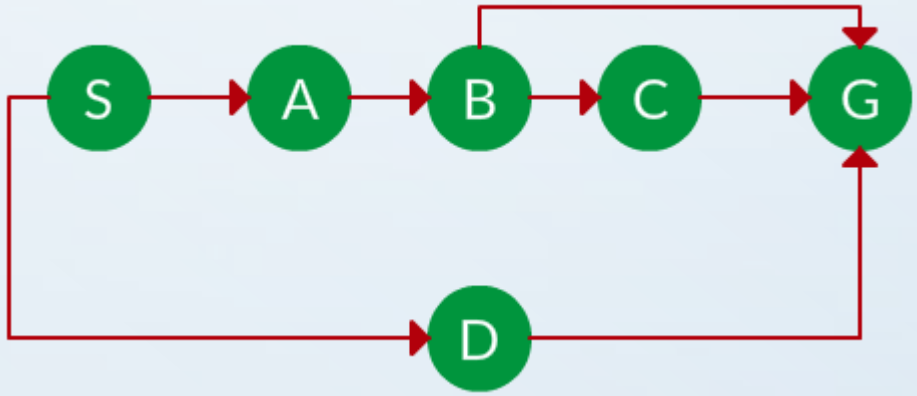


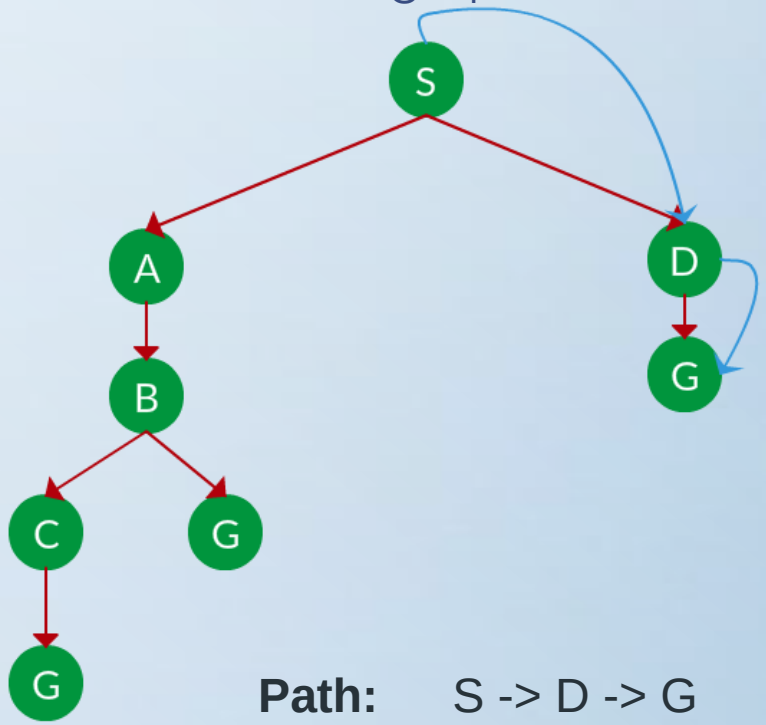
Breadth First Search: shortest first

Breadth-first search (BFS) is an algorithm for traversing or searching tree or graph data structures. It starts at the tree root, and explores all of the neighbor nodes at the present depth prior to moving on to the nodes at the next depth level.

Example:

Which solution would BFS find to move from node S to node G if run on the graph below?





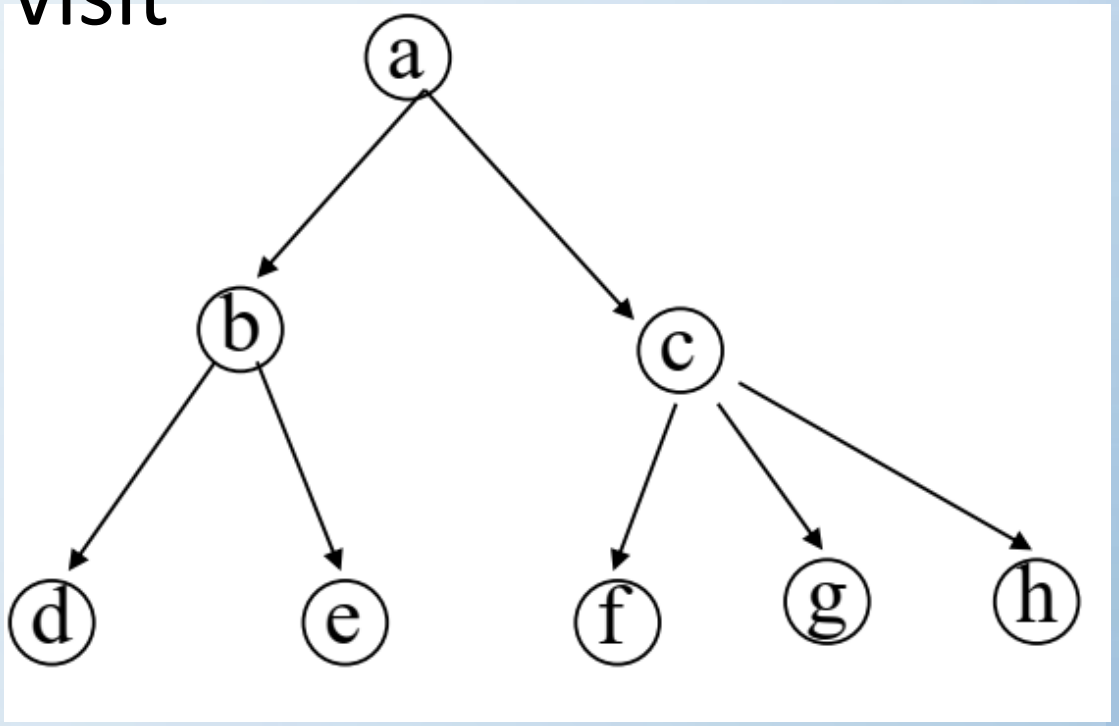
Path: S -> D -> G

Breadth First Search: shortest first

Breadth-first search (BFS) is an algorithm for traversing or searching tree or graph data structures. It starts at the tree root, and explores all of the neighbor nodes at the present depth prior to moving on to the nodes at the next depth level.

- Maintain queue of nodes to visit
- Evaluation
  - ✓ Complete? **Yes**
  - ✓ Time Complexity? $O(b^d)$
  - ✓ Space Complexity? $O(b^d)$
  - ✓ Optimality

$$Yes, if\ stepcost = 1$$



BFS and Queue (FIFO)

Breadth-First Search Algorithm Pseudocode

Here's the pseudocode to implement the Breadth-First Search Algorithm:

```
1   Input: s as the source node
2
3   BFS (G, s)
4   let Q be queue.
5   Q.enqueue( s )
6
7   mark s as visited
8   while ( Q is not empty)
9   v = Q.dequeue( )
10
11  for all neighbors w of v in Graph G
12  if w is not visited
13  Q.enqueue( w )
14  mark w as visited
```

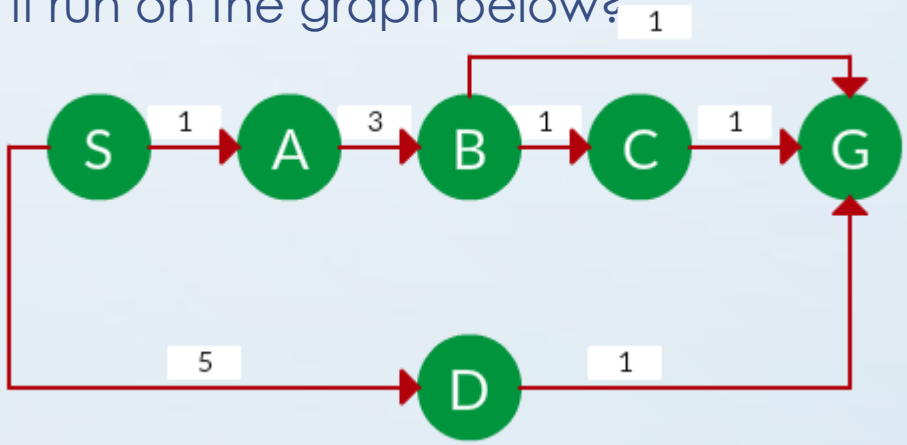
Uniform Cost Search: cheapest first

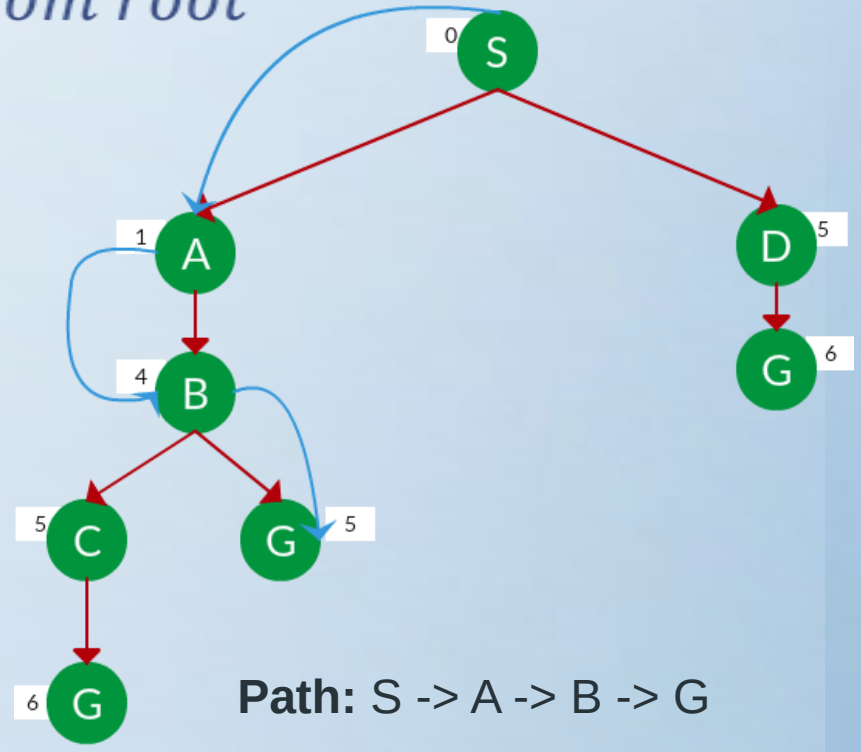
- UCS is different from BFS and DFS because here the costs come into play. In other words, traversing via different edges might not have the same cost. The goal is to find a path where the cumulative sum of costs is the least.

- Cost of a node is defined as:

$$cost(node) = cumulative\ cost\ of\ all\ nodes\ from\ root$$
$$cost(root) = 0$$

Example:
Which solution would BFS find to move from node S to node G if run on the graph below?



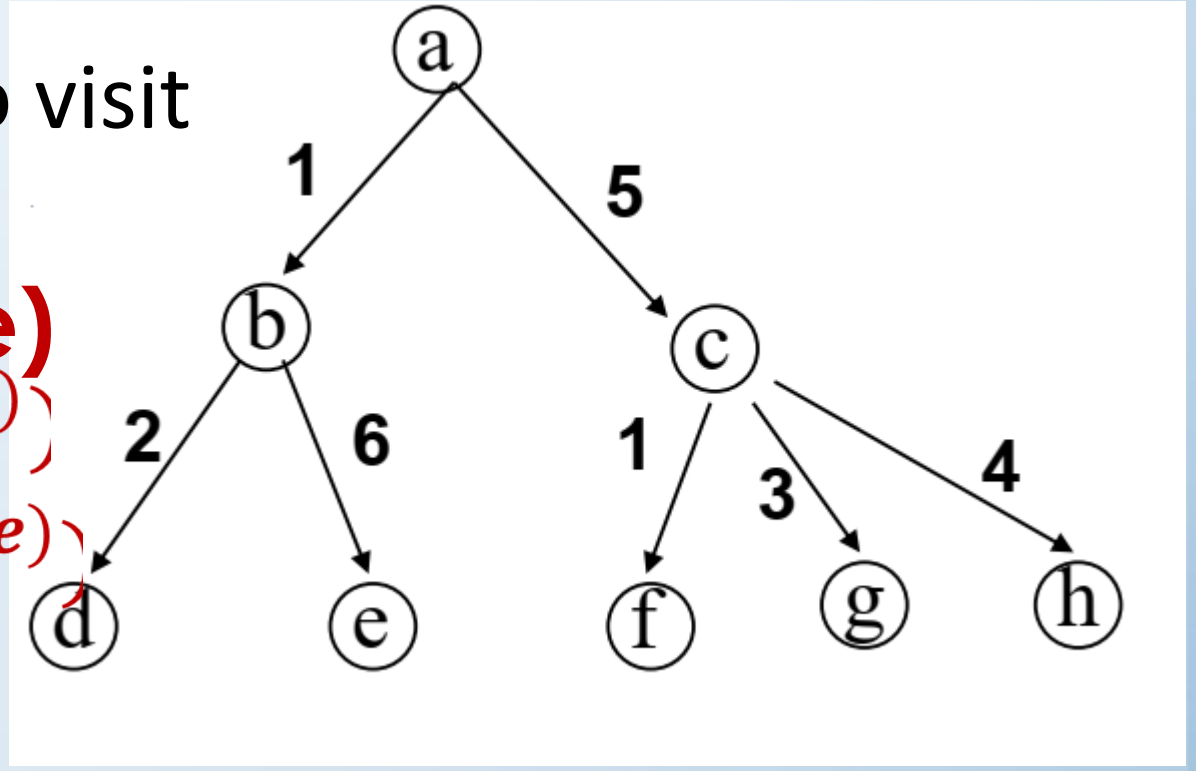


Path: S -> A -> B -> G

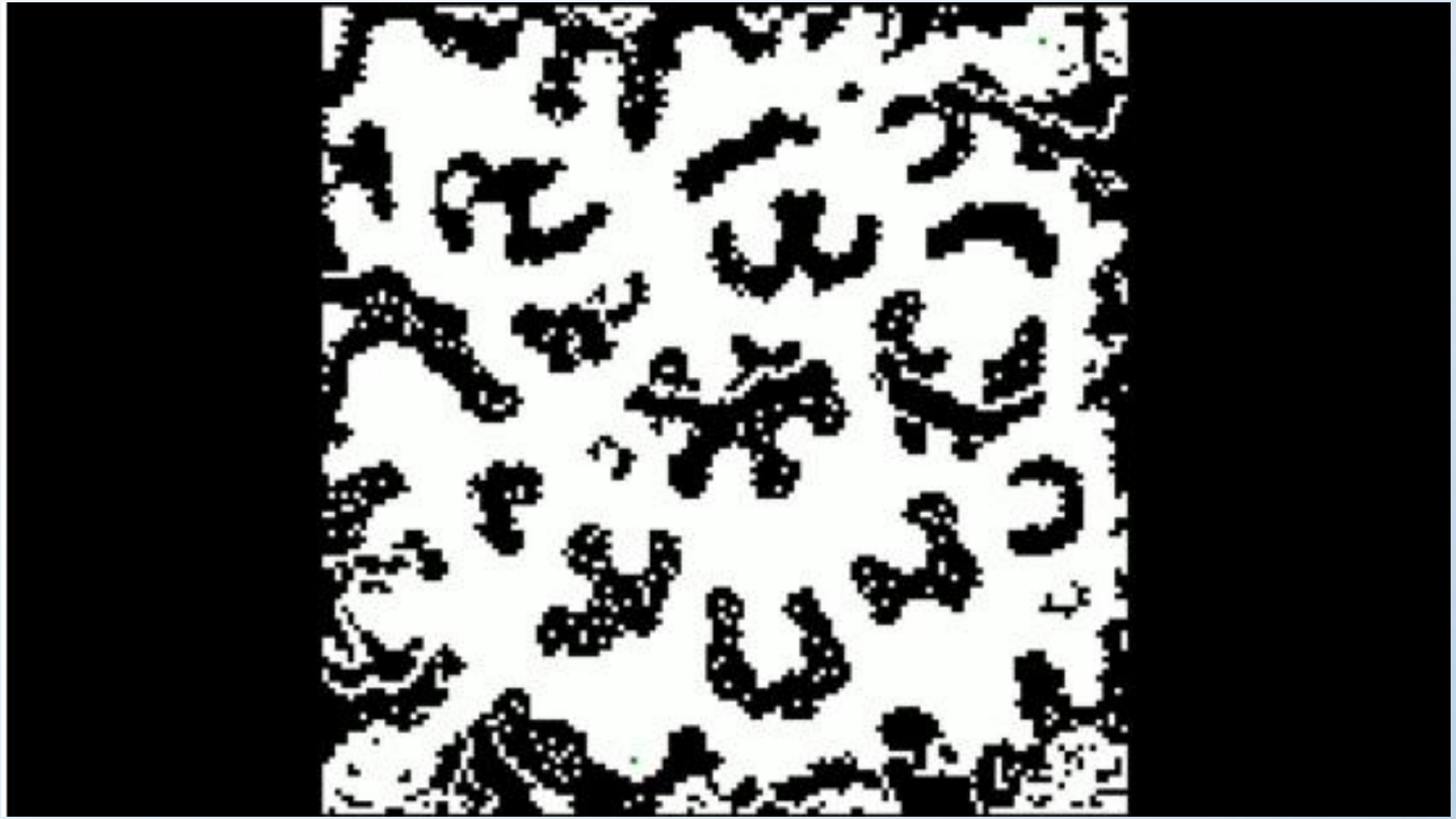
Uniform Cost Search: cheapest first

- UCS is different from BFS and DFS because here the costs come into play. In other words, traversing via different edges might not have the same cost. The goal is to find a path where the cumulative sum of costs is the least.

- Maintain queue of nodes to visit
- Evaluation
  - ✓ Complete? **Yes (If b finite)**
  - ✓ Time Complexity? $O(b^{(c*/e)})$
  - ✓ Space Complexity? $O(b^{(c*/e)})$
  - ✓ Optimality Yes

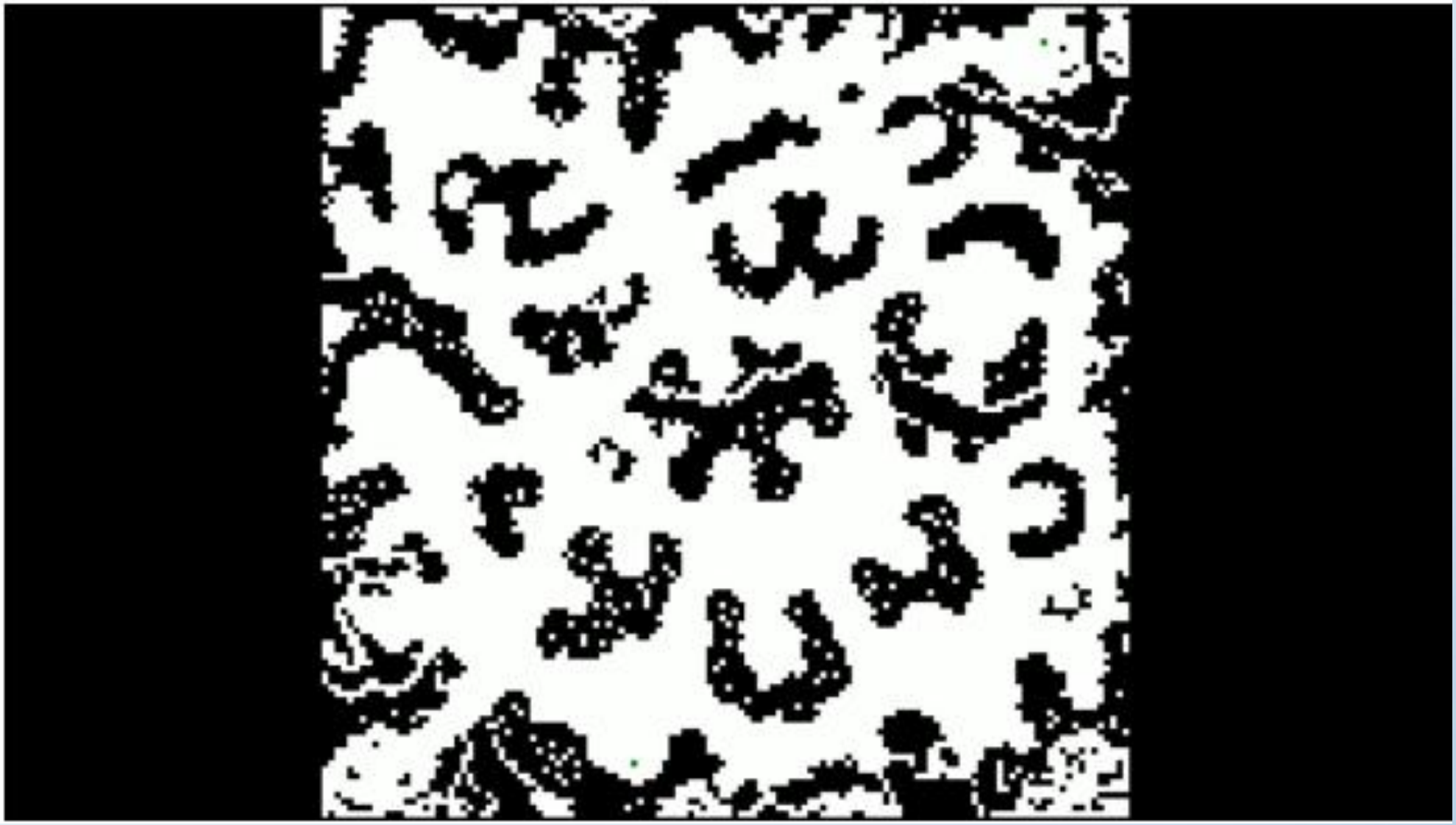


DFS: Example



http://www.youtube.com/watch?v=dtoFAvtVE4U

UCS: Example



http://www.youtube.com/watch?v=z6lUnb9ktkE

Time vs Memory

Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

Time and memory requirement for BFS. The number shown assumed branch factor b = 10, 1 million nodes/sec; 1000bytes/node.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Time	b^d	b^d	b^m	b^l	b^d	$b^{d/2}$
Space	b^d	b^d	bm	bl	bd	$b^{d/2}$
Optimal?	Yes	Yes	No	No	Yes	Yes
Complete?	Yes	Yes	No	Yes, if $l > d$	Yes	Yes

CSA2001

FUNDAMENTALS IN AI AND ML

Module-2

Searching for a (shortest / least cost) path to goal state(s)

Search through the state space.

We will consider search techniques that use an explicit search tree that is generated by the initial state + successor function.

```
initialize (initial node)
Loop
        choose a node for expansion
          according to strategy
        goal node? done
        expand node with successor function
```

Tree-search algorithms

Basic idea:

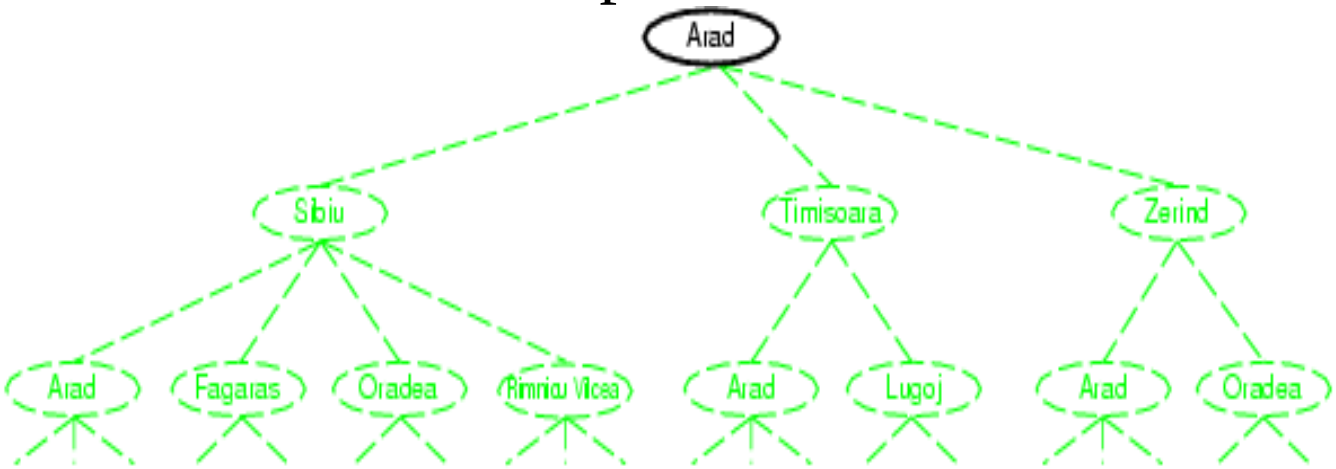
- **simulated exploration of state space by generating successors of already-explored states (a.k.a. ~ expanding states)**

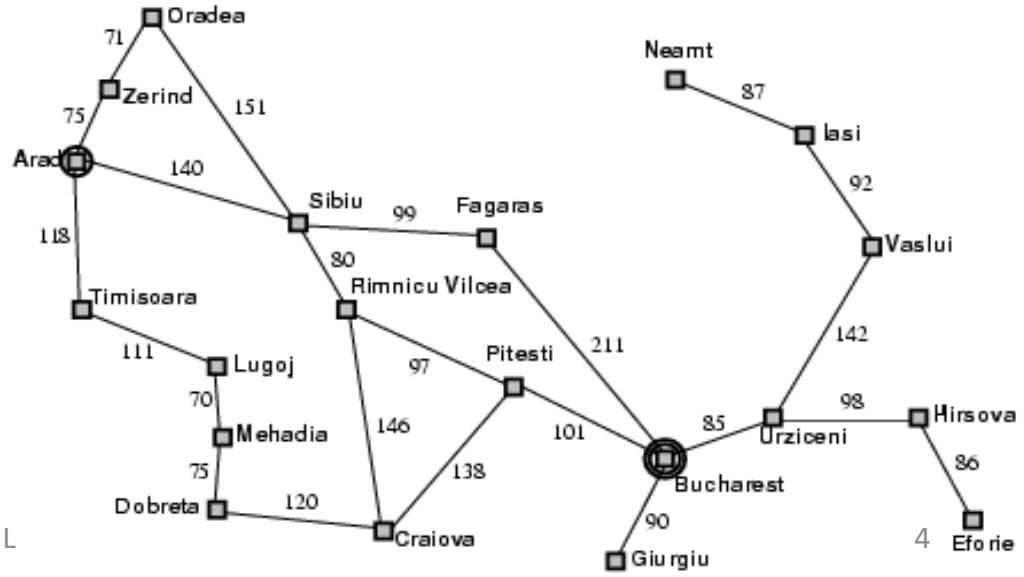
```
function TREE-SEARCH( problem, strategy) returns a solution, or failure
    initialize the search tree using the initial state of problem
    loop do
        if there are no candidates for expansion then return failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state then return the corresponding solution
        else expand the node and add the resulting nodes to the search tree
```

Note: 1) Here we only check a node for possibly being a goal state, after we select the node for expansion.
2) A "node" is a data structure containing state + additional info (parent node, etc.

Tree-search algorithm- Example

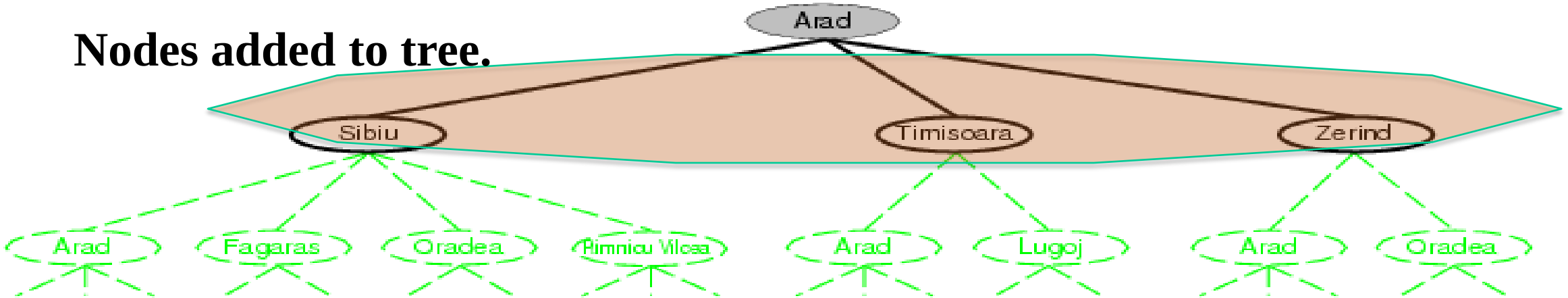
Node selected for expansion.

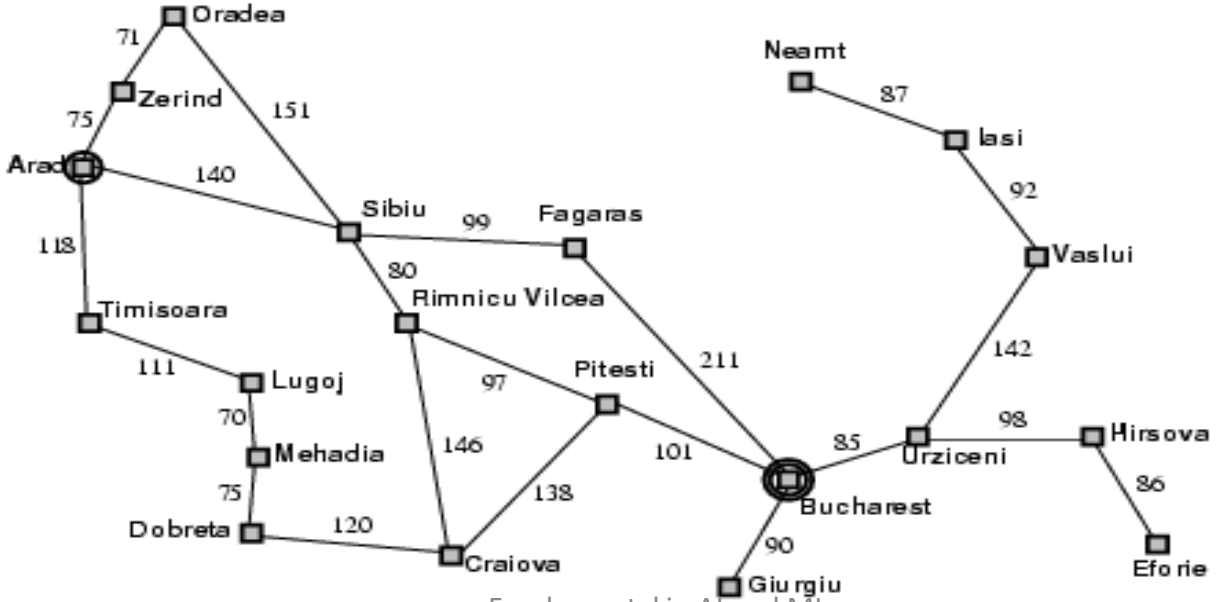




Tree-search algorithm- Example

Nodes added to tree.

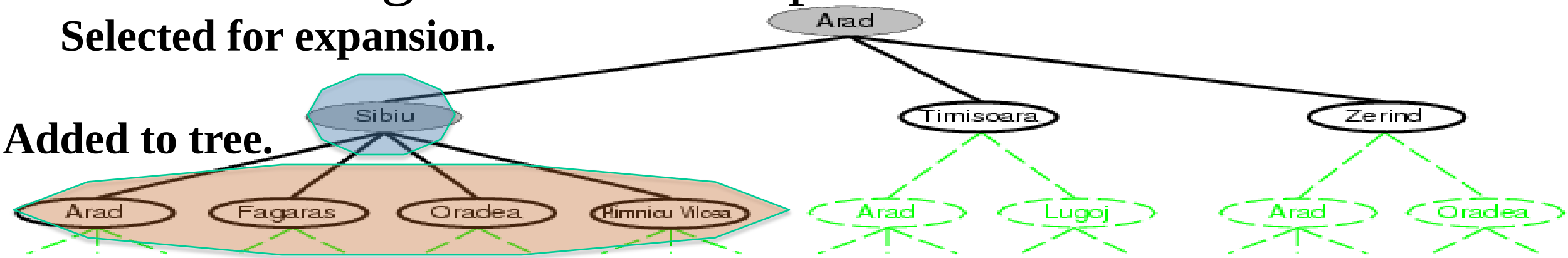




Tree-search algorithm- Example

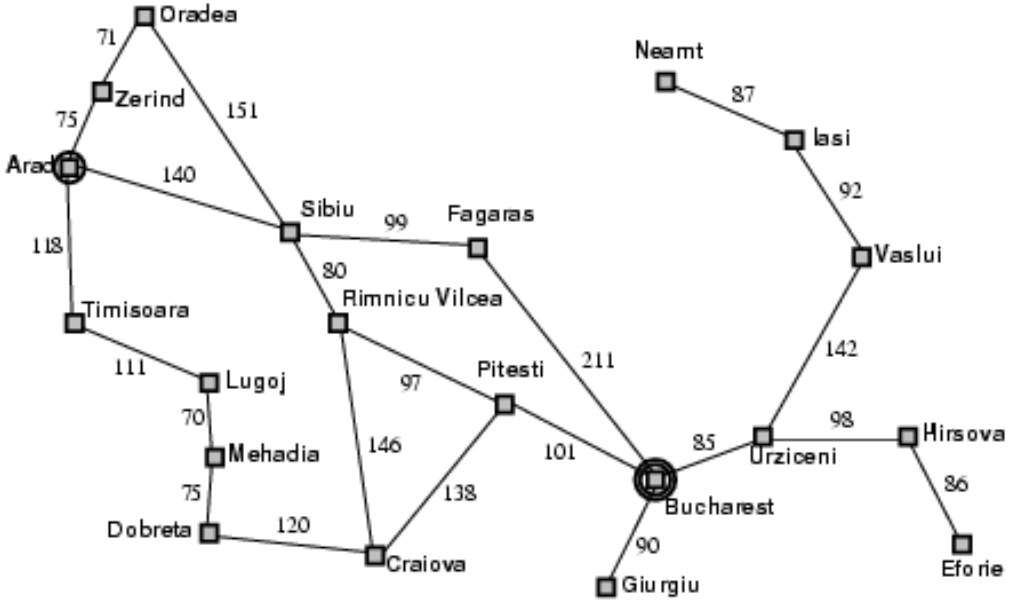
Selected for expansion.

Added to tree.

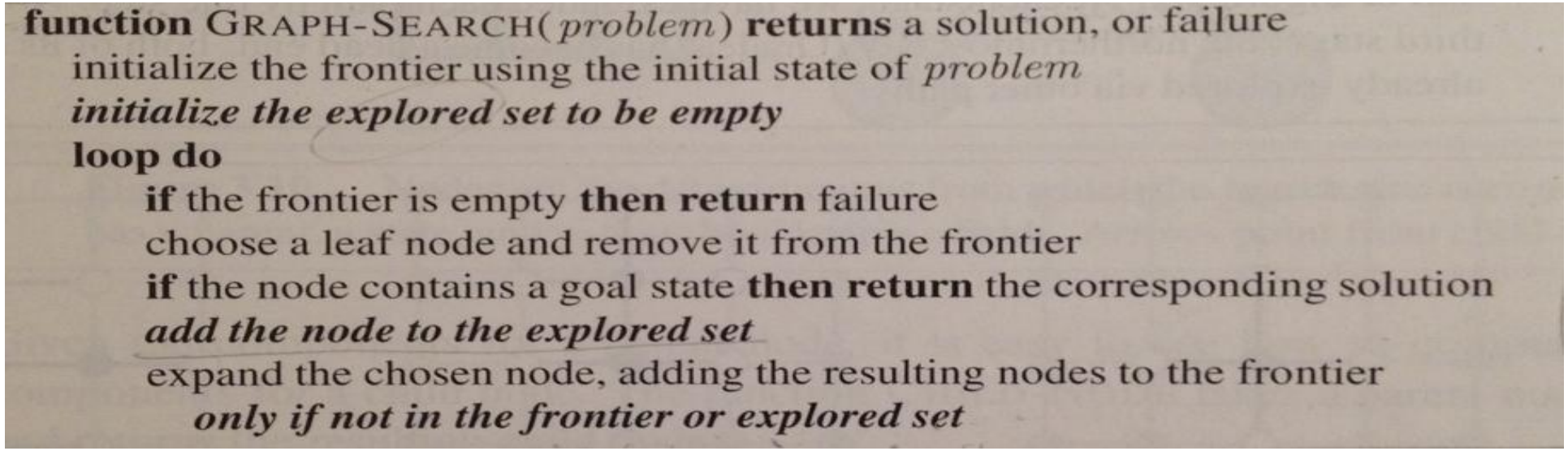


Note: Arad added (again) to tree! (reachable from Sibiu)

Not necessarily a problem, but in Graph-Search, we will avoid this by maintaining an "explored" list.



Graph-search



```
function GRAPH-SEARCH(problem) returns a solution, or failure
    initialize the frontier using the initial state of problem
    initialize the explored set to be empty
    loop do
        if the frontier is empty then return failure
        choose a leaf node and remove it from the frontier
        if the node contains a goal state then return the corresponding solution
        add the node to the explored set
        expand the chosen node, adding the resulting nodes to the frontier
            only if not in the frontier or explored set
```

Note:

1) Uses "explored" set to avoid visiting already explored states.

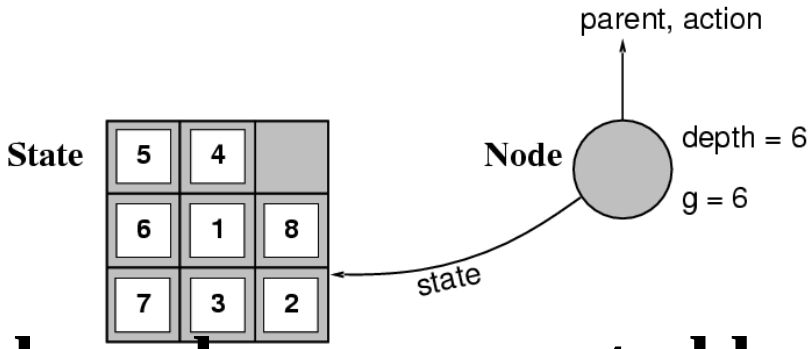
2) Uses "frontier" set to store states that remain to be explored and expanded.

3) *However, with eg uniform cost search, we need to make a special check when node (i.e. state) is on frontier. Details later.*

Implementation: states vs. nodes

A state is a --- representation of --- a physical configuration.

A node is a data structure constituting part of a search tree includes state, tree parent node, action (applied to parent), path cost (initial state to node) *g(x)*, depth



Fringe is the collection of nodes that have been generated but not (yet) expanded. Each node of the fringe is a leaf node.

The Expand function creates new nodes, filling in the various fields and using the SuccessorFn of the problem to create the corresponding states.

Implementation: General Tree Search

```
function TREE-SEARCH( problem, fringe) returns a solution, or failure
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← REMOVE-FRONT(fringe)
        if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
        fringe ← INSERTALL(EXPAND(node, problem), fringe)

function EXPAND( node, problem) returns a set of nodes
    successors ← the empty set
    for each action, result in SUCCESSOR-FN[problem](STATE[node]) do
        s ← a new NODE
        PARENT-NODE[s] ← node;  ACTION[s] ← action;  STATE[s] ← result
        PATH-COST[s] ← PATH-COST[node] + STEP-COST(node, action, s)
        DEPTH[s] ← DEPTH[node] + 1
        add s to successors
    return successors
```

Search Strategies

A search strategy is defined by picking the order of node expansion.

Strategies are evaluated along the following dimensions:

- **completeness: does it always find a solution if one exists?**
- **time complexity: number of nodes generated**
- **space complexity: maximum number of nodes in memory**
- **optimality: does it always find a least-cost solution?**

Time and space complexity are measured in terms of

- ***b:* maximum branching factor of the search tree**
- ***d:* depth of the least-cost solution**
- ***m*: maximum depth of the state space (may be ∞)**

Uninformed Search Strategies

Uninformed (blind) search strategies use only the information available in the problem definition:

- **Breadth-first search**
- **Uniform-cost search**
- **Depth-first search**
- **Depth-limited search**
- **Iterative deepening search**
- **Bidirectional search**

Key issue: type of queue used for the fringe of the search tree (collection of tree nodes that have been generated but not yet expanded)

Breadth-First Search

Expand shallowest unexpanded node.

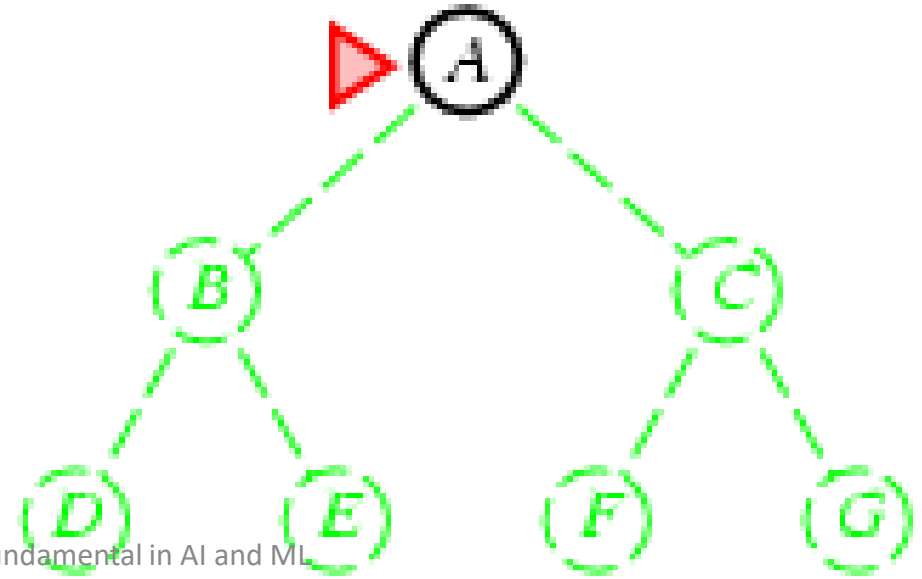
Implementation:

- ***fringe* is a FIFO queue, i.e., new nodes go at end**

  (First In First Out queue.)

Fringe queue: <A>

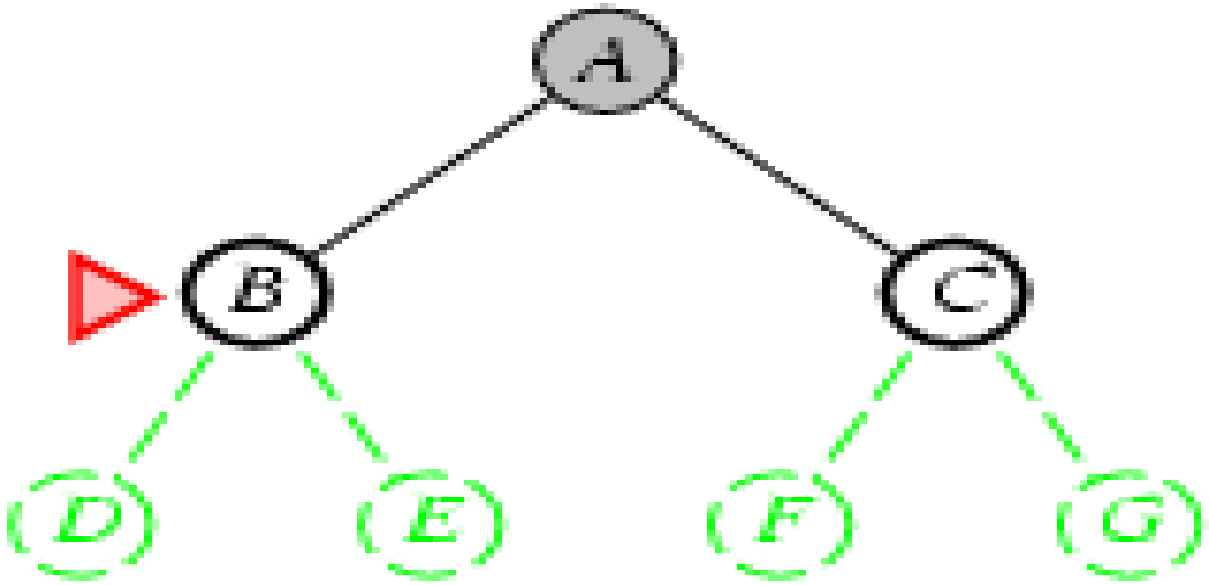
Select A from queue and expand.



Gives <B, C>

Breadth-First Search

Queue: <B, C>

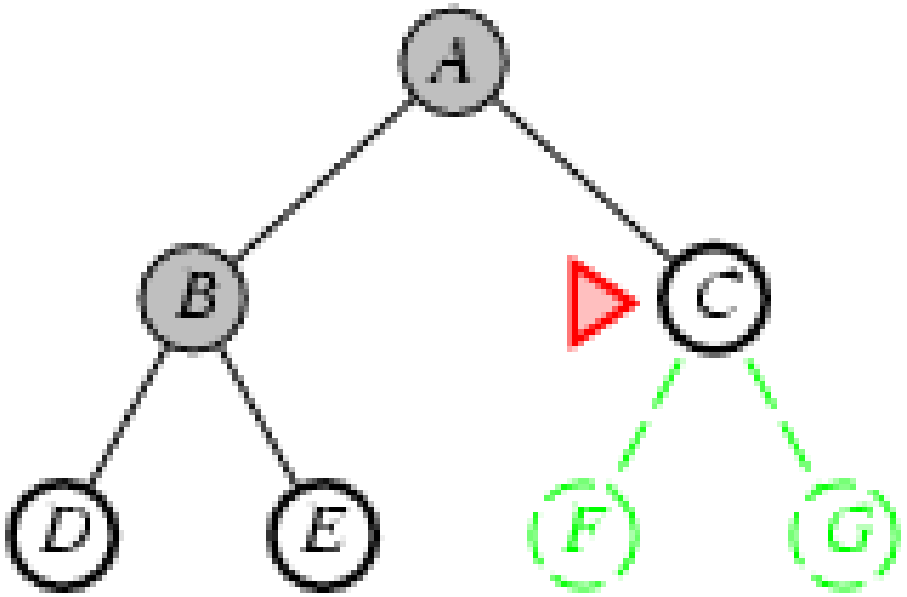


Select B from front, and expand.

Put children at the end.

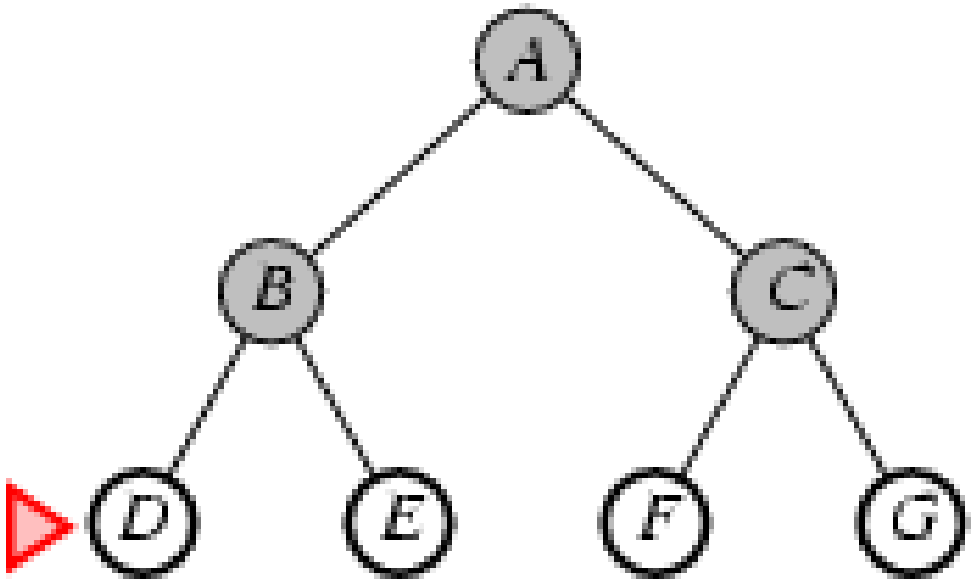
Gives <C, D, E>

Breadth-First Search



Fringe queue: <C, D, E>

Breadth-First Search



Fringe queue: <D, E, F, G>

Assuming no further children, queue becomes <E, F, G>, <F, G>, <G>, <>. Each time node checked for goal state.

Breadth-First Search

```
function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure
    node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    frontier ← a FIFO queue with node as the only element
    explored ← an empty set
    loop do
        if EMPTY?(frontier) then return failure
        node ← POP(frontier)  /* chooses the shallowest node in frontier */
        add node.STATE to explored
        for each action in problem.ACTIONS(node.STATE) do
            child ← CHILD-NODE(problem, node, action)
            if child.STATE is not in explored or frontier then
                if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
                frontier ← INSERT(child, frontier)
```

Properties of breadth-first search

Complete? Yes (if b is finite)
Time? $1+b+b^2+b^3+\ldots+b^d + b(b^d-1) = O(b^{d+1})$
Space? $O(b^{d+1})$ (keeps every node in memory;
needed also to reconstruct soln. path)

Optimal soln. found?

Yes (if all step costs are identical)

Space is the bigger problem (more than time)

- *b:* maximum branching factor of the search tree
- *d:* depth of the least-cost solution

Note: check for goal only when node is expanded.

Time and Space requirement for Breadth First Search with bf = 10

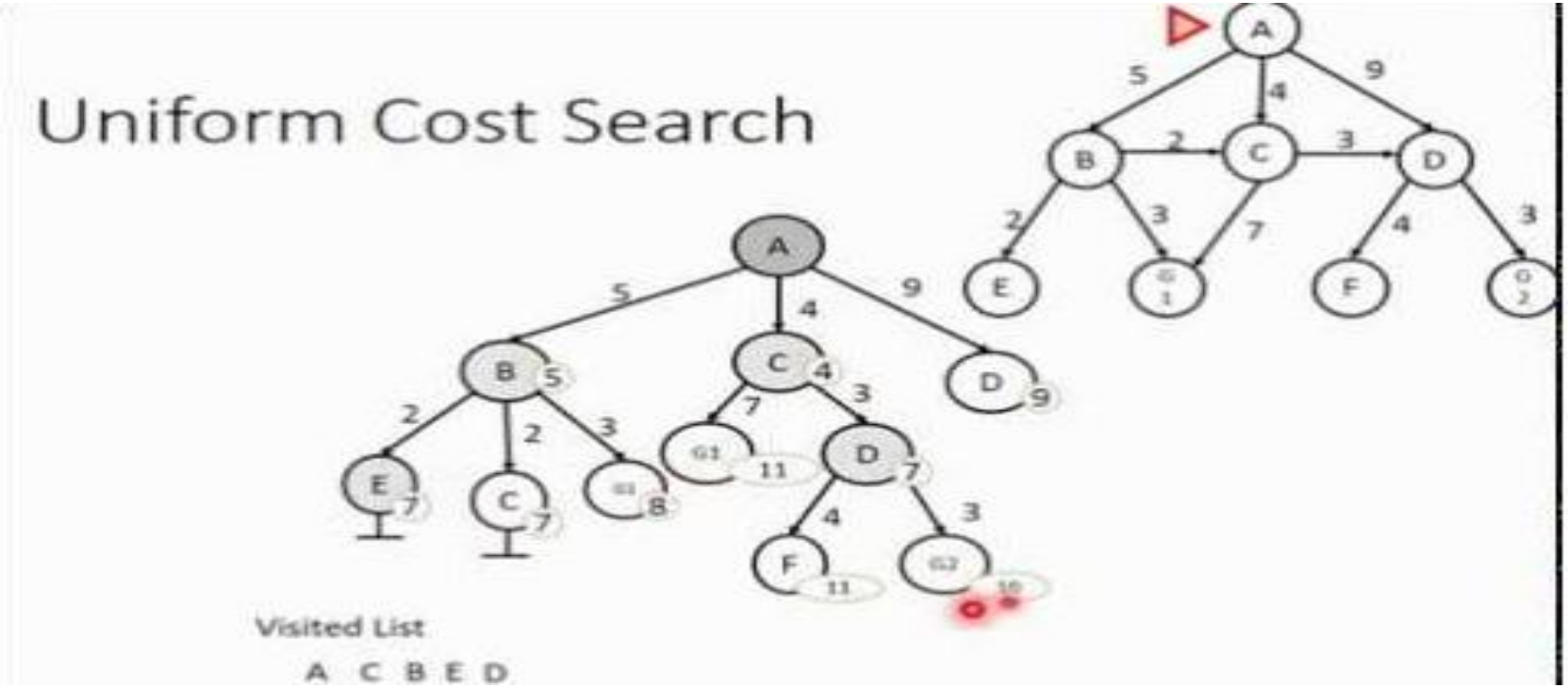
Depth	Nodes	Time	Memory
2	110	.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

Figure 3.13 Time and memory requirements for breadth-first search. The numbers shown assume branching factor $b = 10$; 1 million nodes/second; 1000 bytes/node.

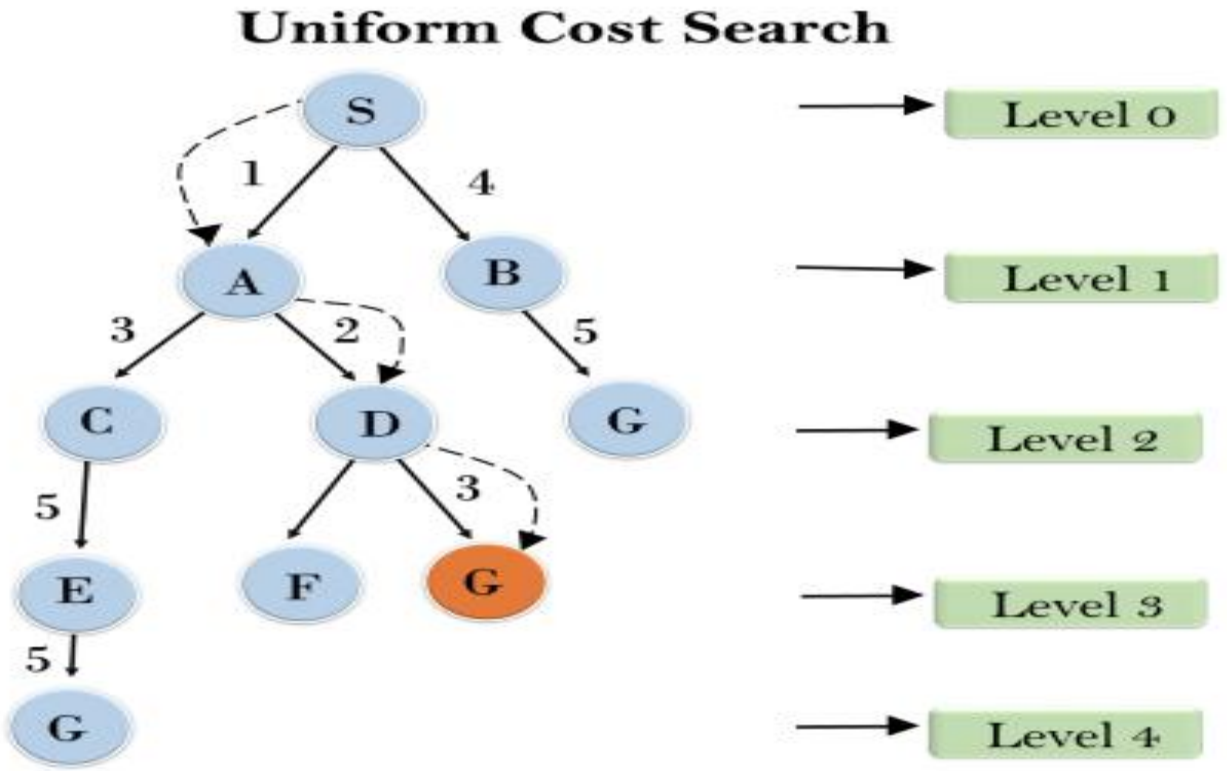
Uniform Cost Search

```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure
    node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
    frontier ← a priority queue ordered by PATH-COST, with node as the only element
    explored ← an empty set
    loop do
        if EMPTY?(frontier) then return failure
        node ← POP(frontier)  /* chooses the lowest-cost node in frontier */
        if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
        add node.STATE to explored
        for each action in problem.ACTIONS(node.STATE) do
            child ← CHILD-NODE(problem, node, action)
            if child.STATE is not in explored or frontier then
                frontier ← INSERT(child, frontier)
            else if child.STATE is in frontier with higher PATH-COST then
                replace that frontier node with child
```

Uniform Cost Search



Uniform Cost Search



Uniform Cost Search

Expand least-cost (of path to) unexpanded node

(e.g. useful for finding shortest path on map)

Implementation:

- ***fringe* = queue *ordered by path cost***

g – cost of reaching a node

Complete? Yes, if step cost $\geq \varepsilon$ **(>0)**

Time? # of nodes with $g \leq$ cost of optimal solution (C*), $O(b^{(1+\lfloor C^*/\varepsilon \rfloor)})$

Space? # of nodes with $g \leq$ cost of optimal solution, $O(b^{(1+\lfloor C^*/\varepsilon \rfloor)})$

Optimal? Yes – nodes expanded in increasing order of $g(n)$

Note: Some subtleties (e.g. checking for goal state).

See p 84 R&N. Also, next slide.

Uniform Cost Search

Two subtleties: (bottom p. 83 Norvig)

1) **Do goal state test, only when a node is selected for expansion.**

(Reason: Bucharest may occur on frontier with a longer than optimal path. It won't be selected for expansion yet. Other nodes will be expanded first, leading us to uncover a shorter path to Bucharest.

2) Graph-search alg. says "don't add child node to frontier if already on explored list or already on frontier." BUT, child may give a shorter path to a state already on frontier. Then, we need to modify the existing node on frontier with the shorter path.

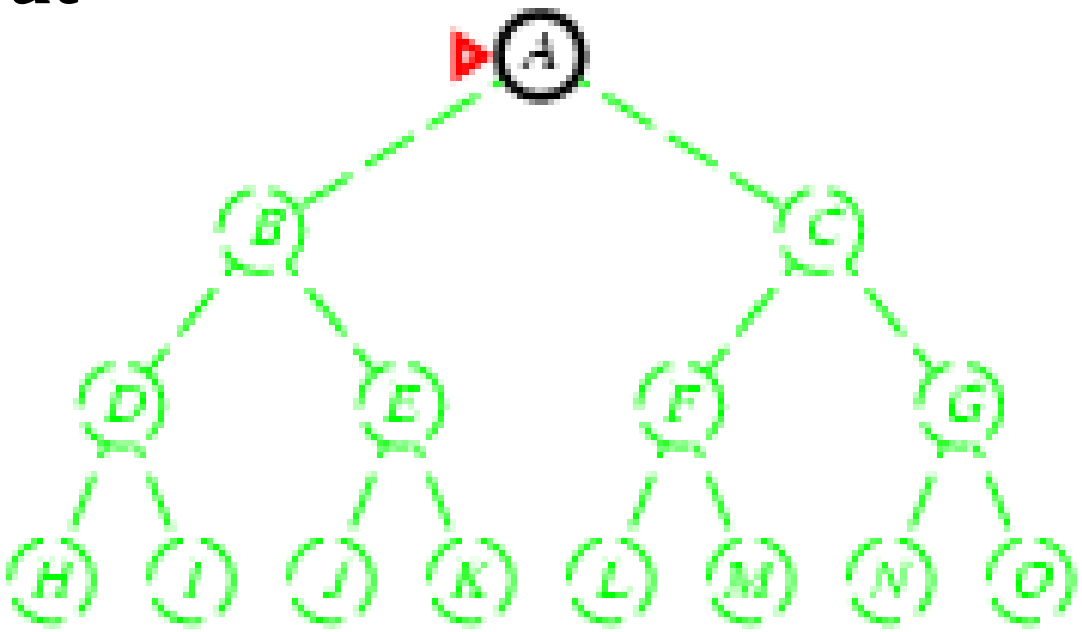
Depth-First Search

"Expand deepest unexpanded node"

Implementation:

- ***fringe* = LIFO queue, i.e., put successors at front ("push on stack")**

  Last In First Out



Fringe stack:

A

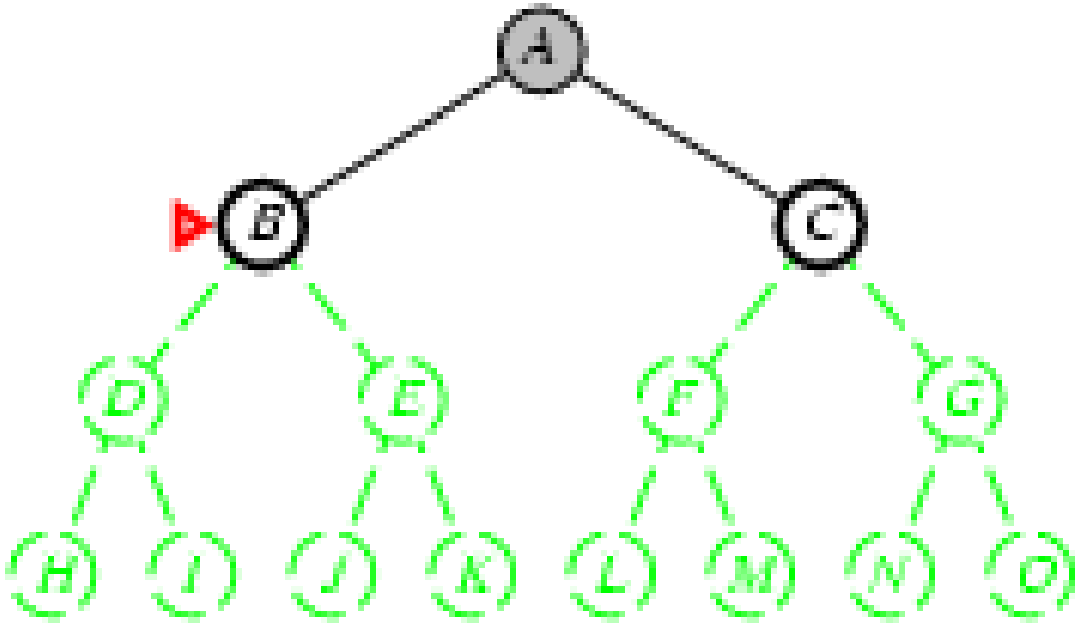
Expanding A, gives stack:

B

C

So, B next.

Depth-First Search



Expanding B, gives stack:

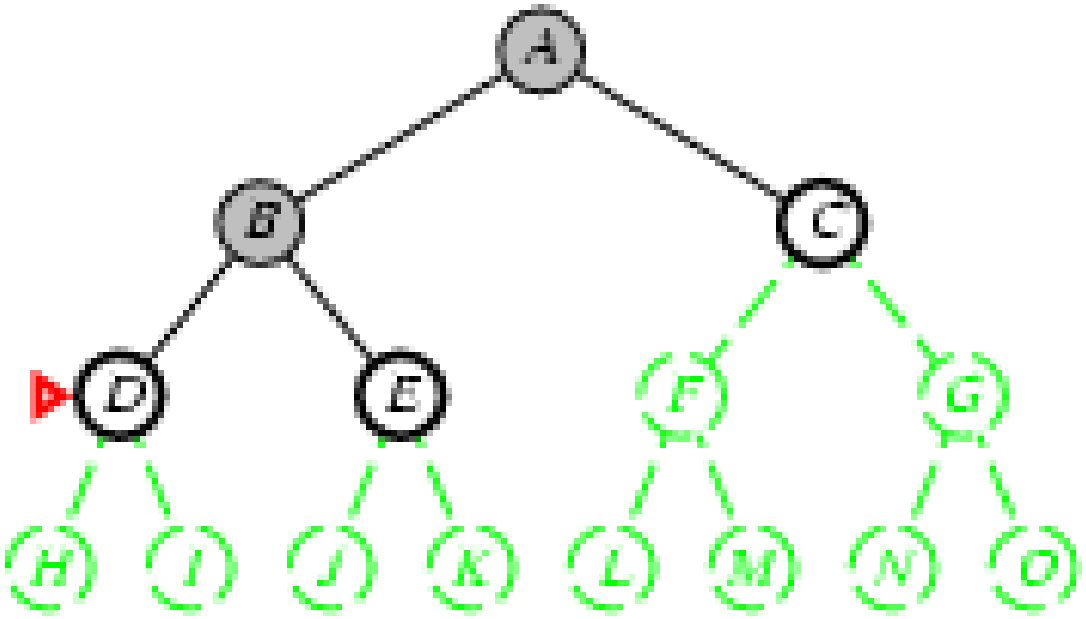
D

E

C

So, D next.

Depth-First Search

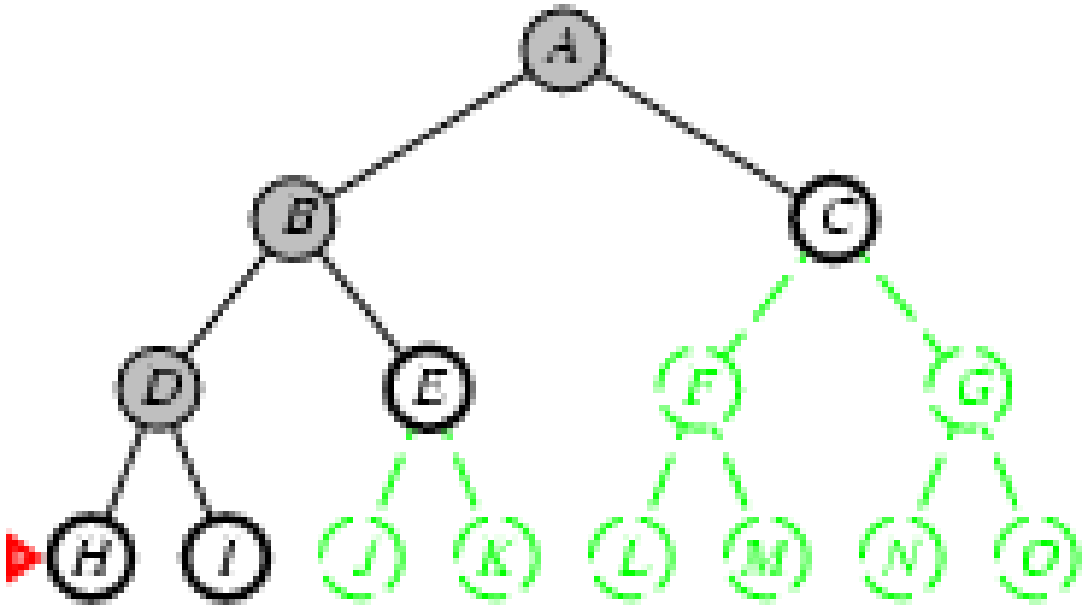


Expanding D, gives stack:

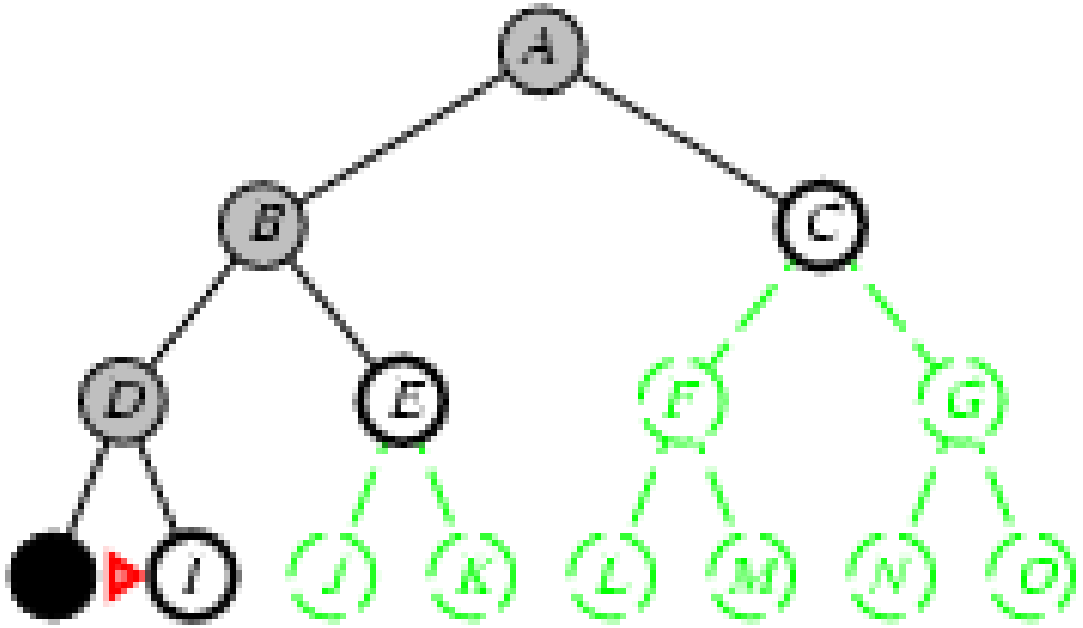
H
I
E
C

So, H next. etc.

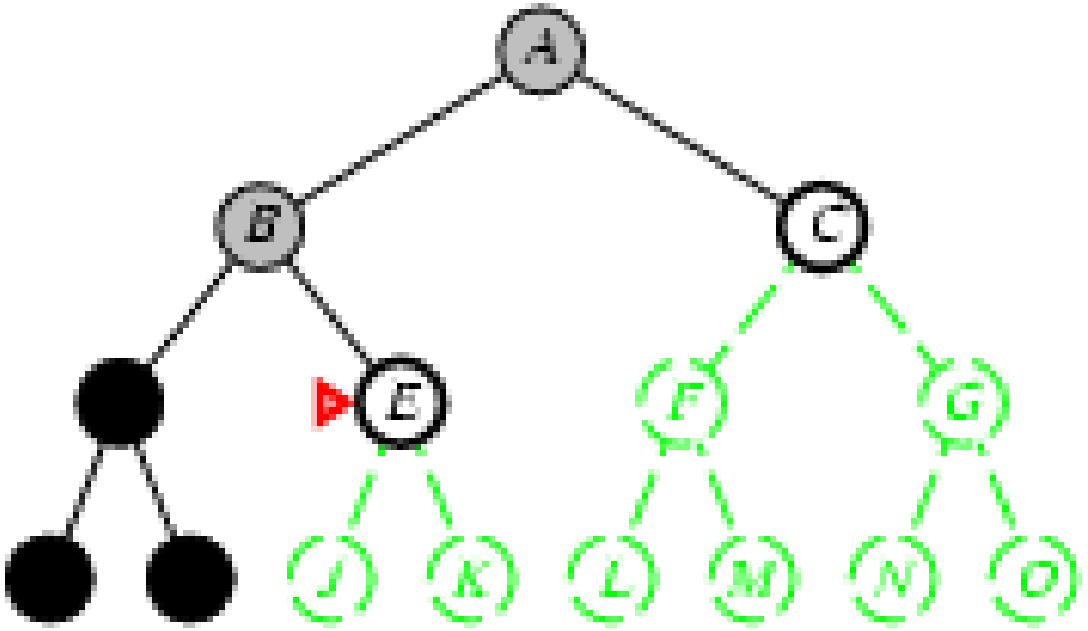
Depth-First Search



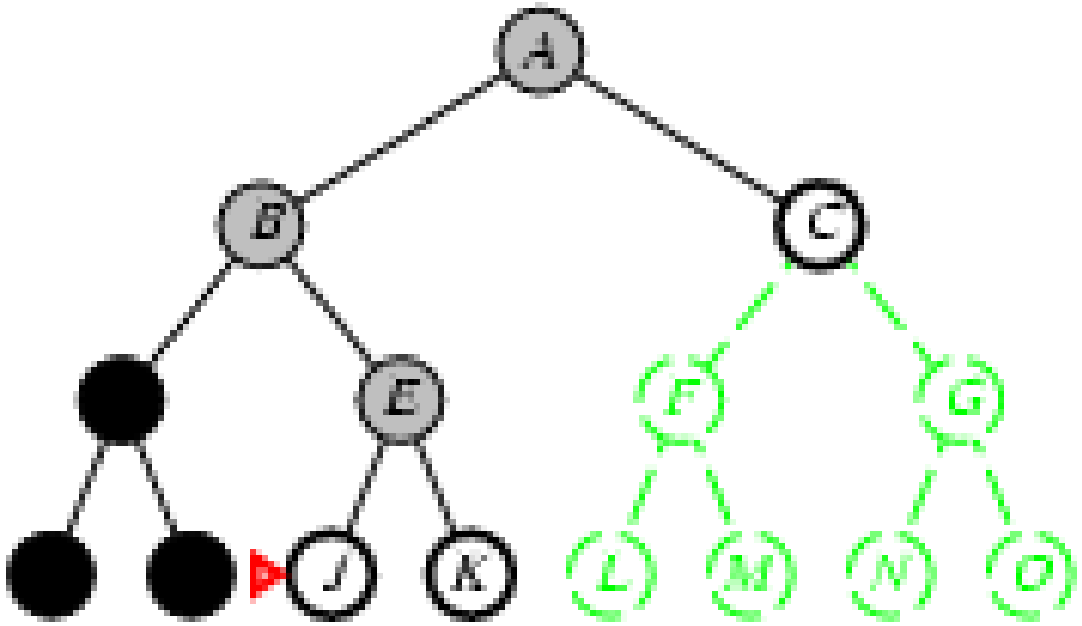
Depth-First Search



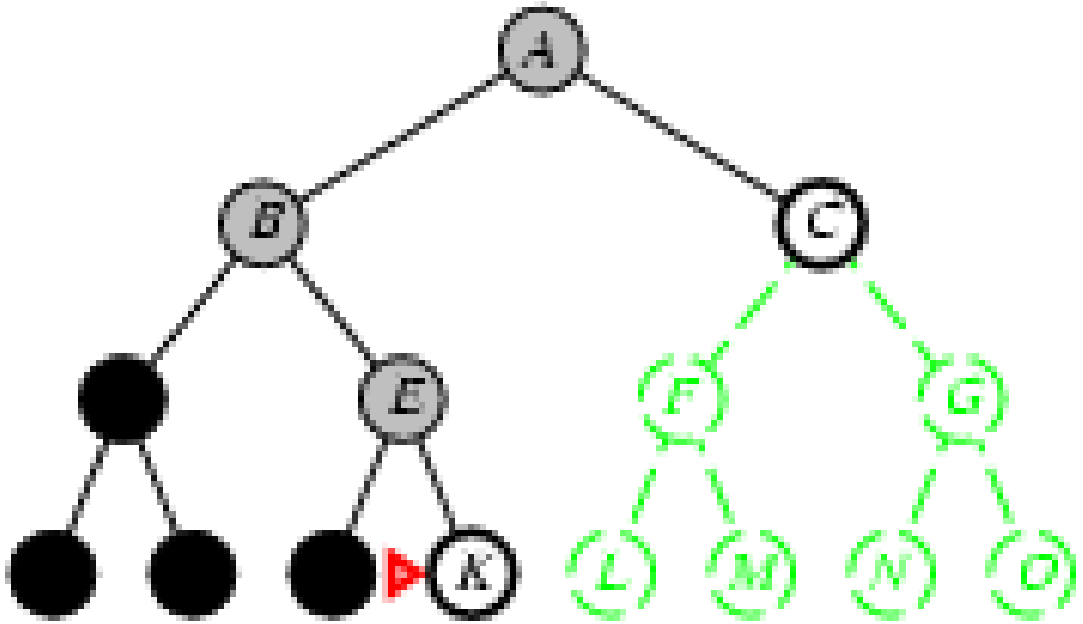
Depth-First Search



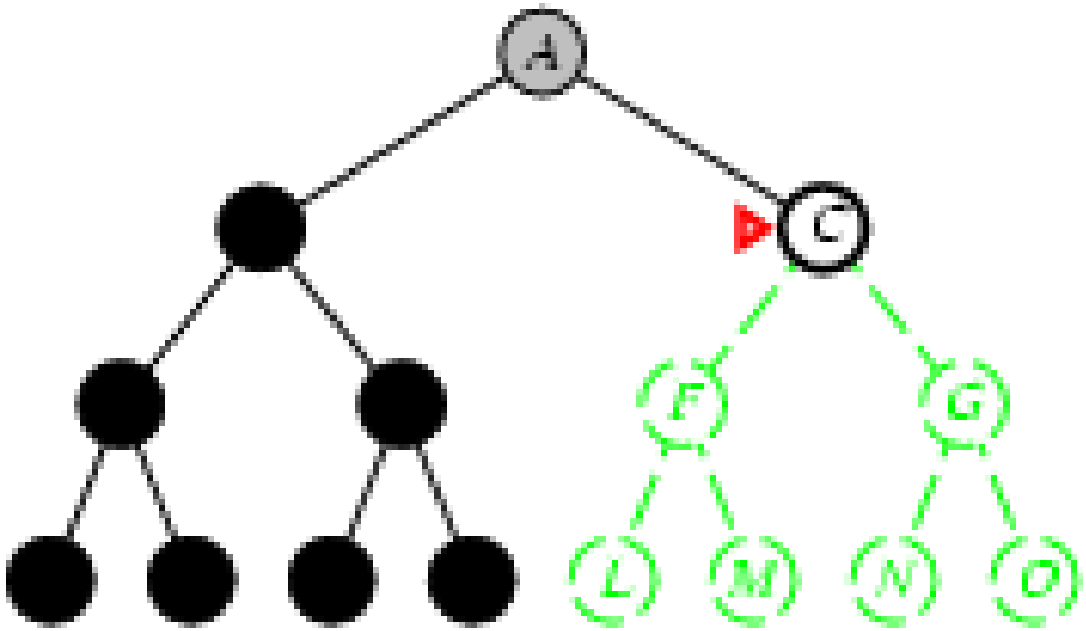
Depth-First Search



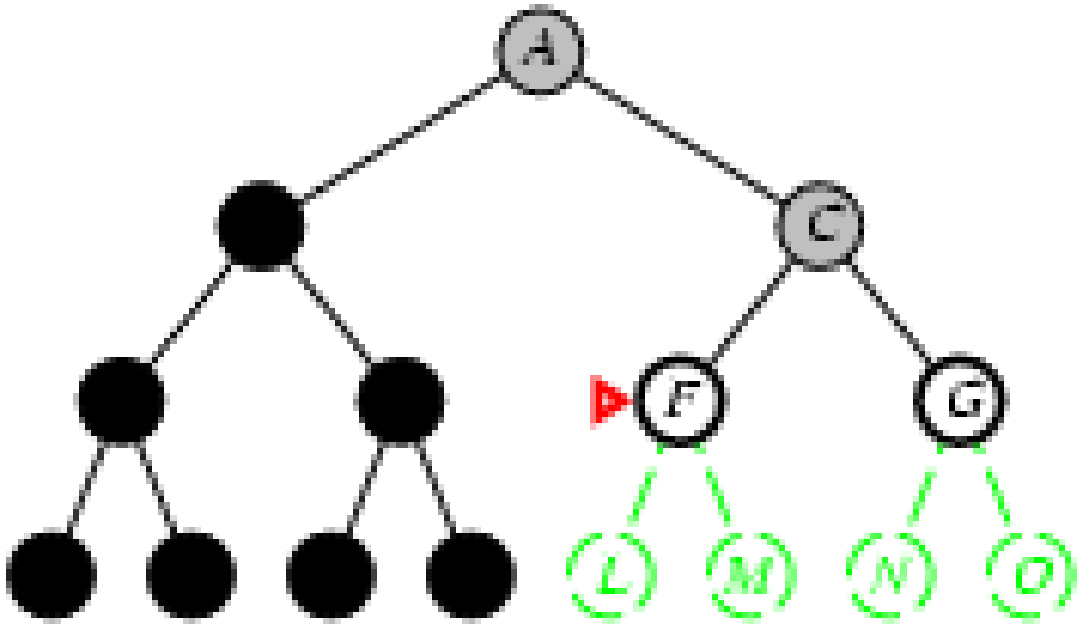
Depth-First Search



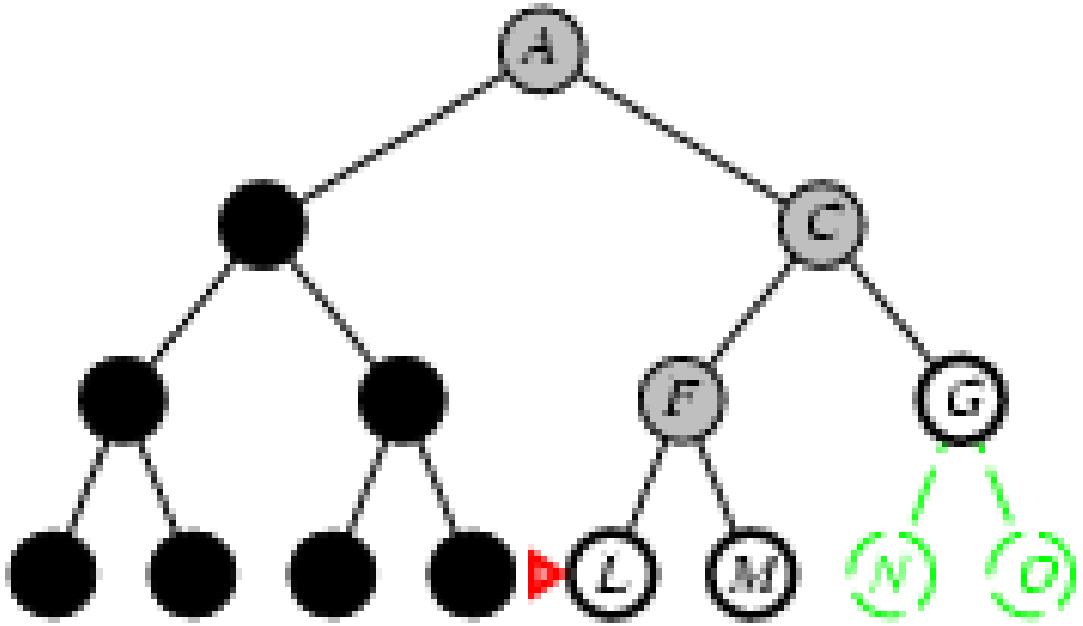
Depth-First Search



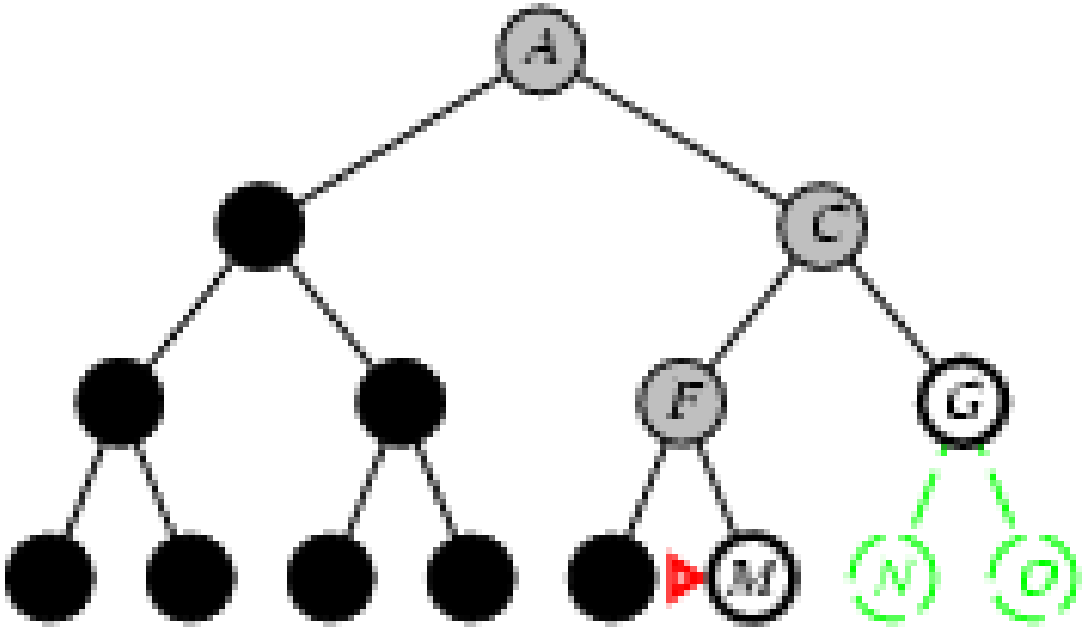
Depth-First Search



Depth-First Search



Depth-First Search



Properties of Depth-First Search

Complete? No: fails in infinite-depth spaces, spaces with loops
Modify to avoid repeated states along path
complete in finite spaces

Note: Can also reconstruct soln. path from single stored branch.

Time? $O(b^m)$**: bad if** m **is much larger than** d

- but if solutions are dense, may be much faster than breadth-first

Space? No

***O(bm)*, i.e., linear space!**

Guarantee that opt. soln. is found?

b: **max. branching factor of the search tree**
d: **depth of the shallowest (least-cost) soln.**
m: maximum depth of state space

Depth-Limited Search

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns a solution, or failure/cutoff
  return RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns a solution, or failure/cutoff
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  else if limit = 0 then return cutoff
  else
      cutoff_occurred? ← false
      for each action in problem.ACTIONS(node.STATE) do
          child ← CHILD-NODE(problem, node, action)
          result ← RECURSIVE-DLS(child, problem, limit − 1)
          if result = cutoff then cutoff_occurred? ← true
          else if result ≠ failure then return result
      if cutoff_occurred? then return cutoff else return failure
```

Complexity Analysis

Completeness	If l < d, incomplete
Time Complexity	$O(b^l)$
Space Complexity	O(bl)
Optimality	If l > d, non optimal

Iterative deepening search

```
function ITERATIVE-DEEPENING-SEARCH( problem) returns a solution, or fail-
ure
    inputs: problem, a problem
    for depth ← 0 to ∞ do
        result ← DEPTH-LIMITED-SEARCH( problem, depth)
        if result ≠ cutoff then return result
```

Iterative deepening search

```
function ITERATIVE-DEEPENING-SEARCH( problem) returns a solution, or failure
    inputs: problem, a problem
    for depth ← 0 to ∞ do
        result ← DEPTH-LIMITED-SEARCH( problem, depth)
        if result ≠ cutoff then return result
```

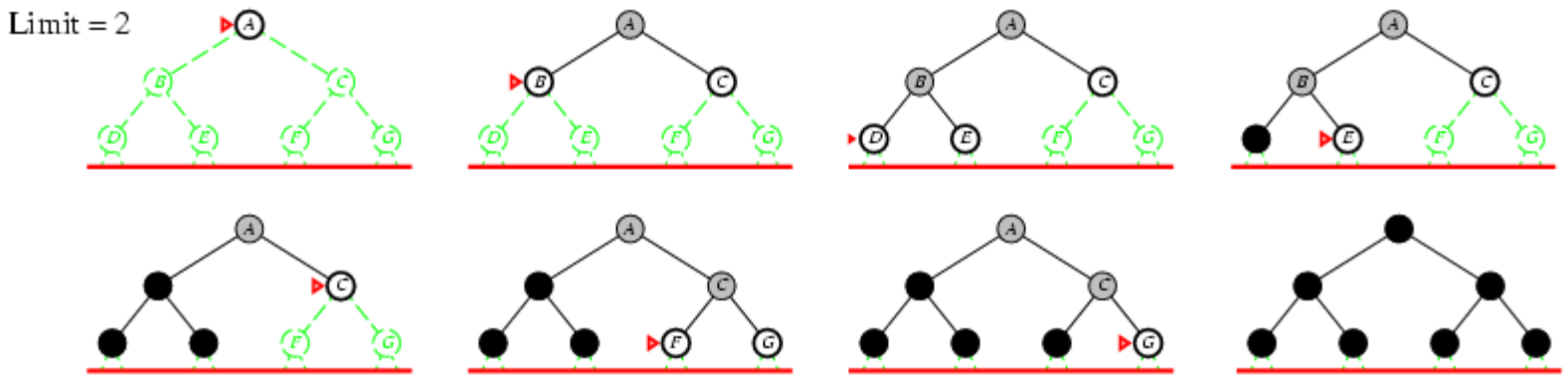
Iterative deepening search



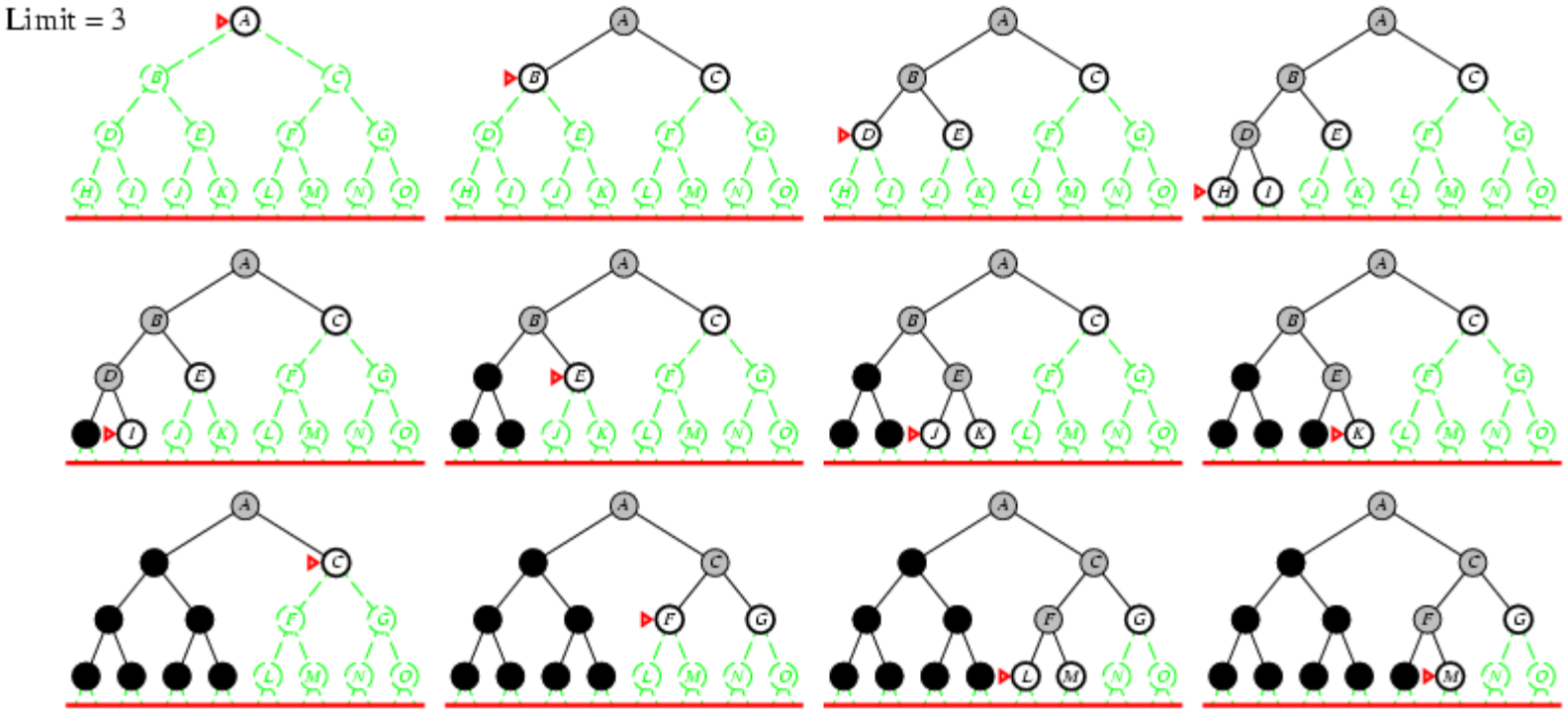
Iterative deepening search



Iterative deepening search l =2



Iterative deepening search l =3 Why would one do that?



Why would one do that?

Combine good memory requirements of depth-first with the completeness of breadth-first when branching factor is finite and is optimal when the path cost is a non-decreasing function of the depth of the node.

Idea was a breakthrough in game playing. All game tree search uses iterative deepening nowadays. What's the added advantage in games?

"Anytime" nature.

Iterative deepening search

Iterative deepening is the preferred uninformed search method when there is a large search space and the depth of the solutionis not known.

Number of nodes generated in an iterative deepening search to depth

***d* with branching factor *b*:** **Looks quite wasteful, is it?**

$$N_{IDS} = d\,b^1 + (d-1)b^2 + \ldots + 3b^{d-2} + 2b^{d-1} + 1b^d$$

Nodes generated in a breadth-first search with branching factor *b*:

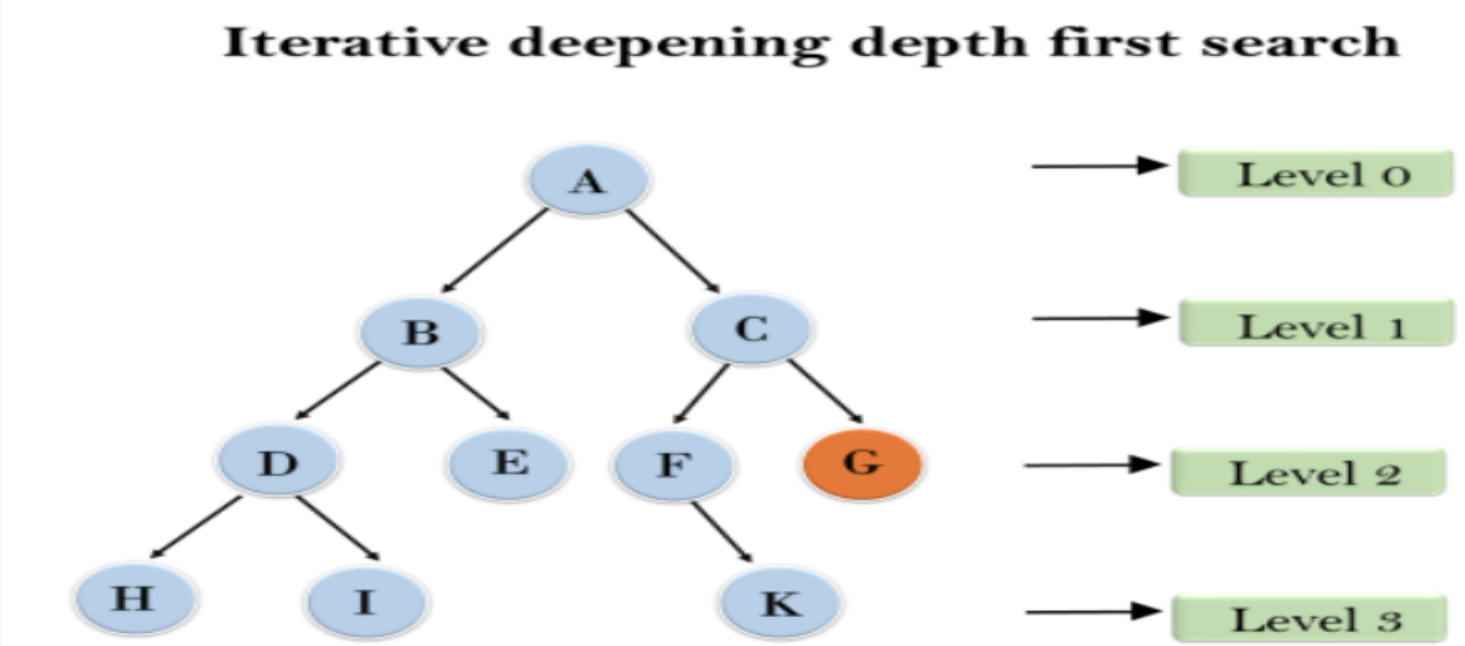
$$N_{BFS} = b^1 + b^2 + \ldots + b^{d-2} + b^{d-1} + b^d$$

For *b = 10, d = 5*,



- $N_{BFS} = 10 + 100 + 1{,}000 + 10{,}000 + 100{,}000 = 111{,}110$
- $N_{IDS} = 50 + 400 + 3{,}000 + 20{,}000 + 100{,}000 = 123{,}456$

Iterative deepening search



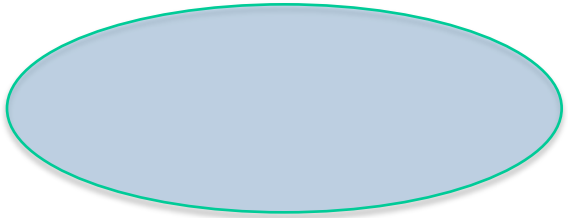
1'st Iteration-----> A
2'nd Iteration----> A, B, C
3'rd Iteration------>A, B, D, E, C, F, G
4'th Iteration------>A, B, D, H, I, E, C, F, K, G
In the fourth iteration, the algorithm will find the goal node.

Iterative deepening search

Complete? **Yes**

(*b* finite)

Time? $d\,b^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$



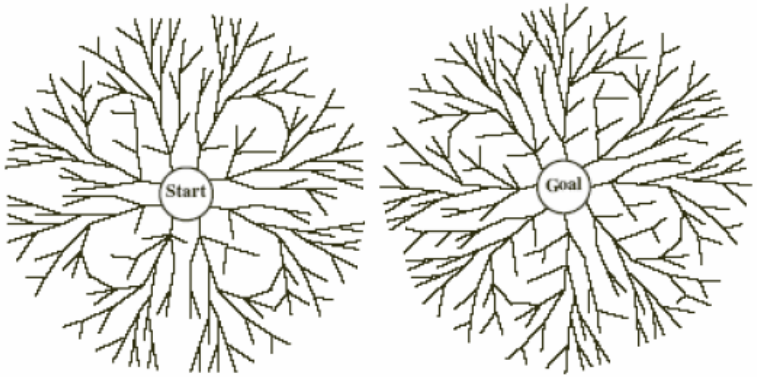
Space? ***O(bd)***

Optimal? **Yes, if step costs identical**

Bidirectional Search

- **Simultaneously:**
  - **Search forward from start**
  - **Search backward from the goal**

  Stop when the two searches meet.



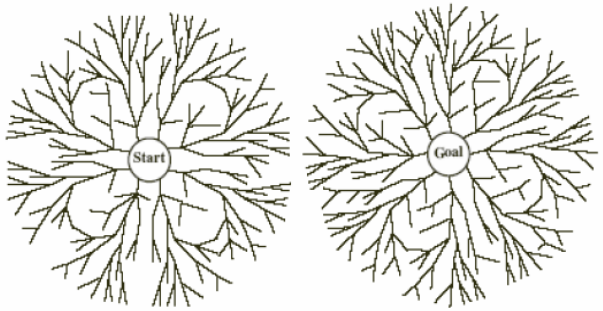
- **If branching factor = b in each directio with solution at depth d**

  only $O(2\ b^{d/2}) = O(2\ b^{d/2})$

Aside: The predecessor of a node should be easily computable (i.e., actions are easily reversible).

Bi directional Search

- **Checking a node for membership in the other search tree can be done in constant time (hash table)**



- **Key limitations:**

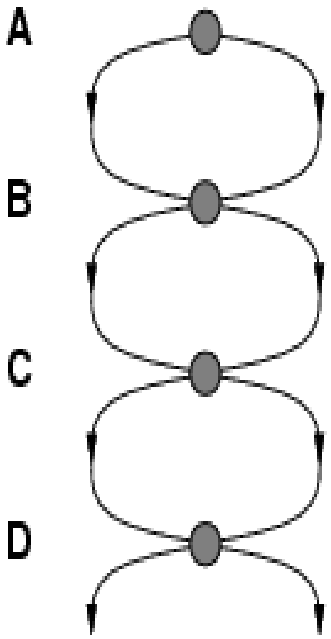
  Space $O(b^{d/2})$

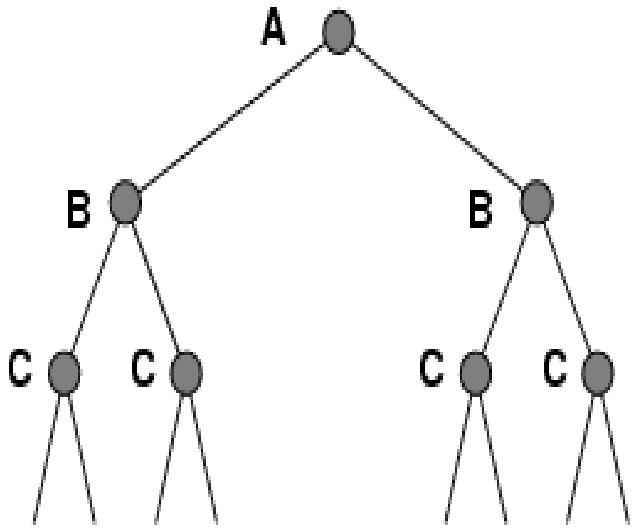
  Also, how to search backwards can be an issue (e.g., in Chess)? What's tricky?

  Problem: lots of states satisfy the goal; don't know which one is relevant.

Aside: The predecessor of a node should be easily computable (i.e., actions are easily reversible).

Repeated States





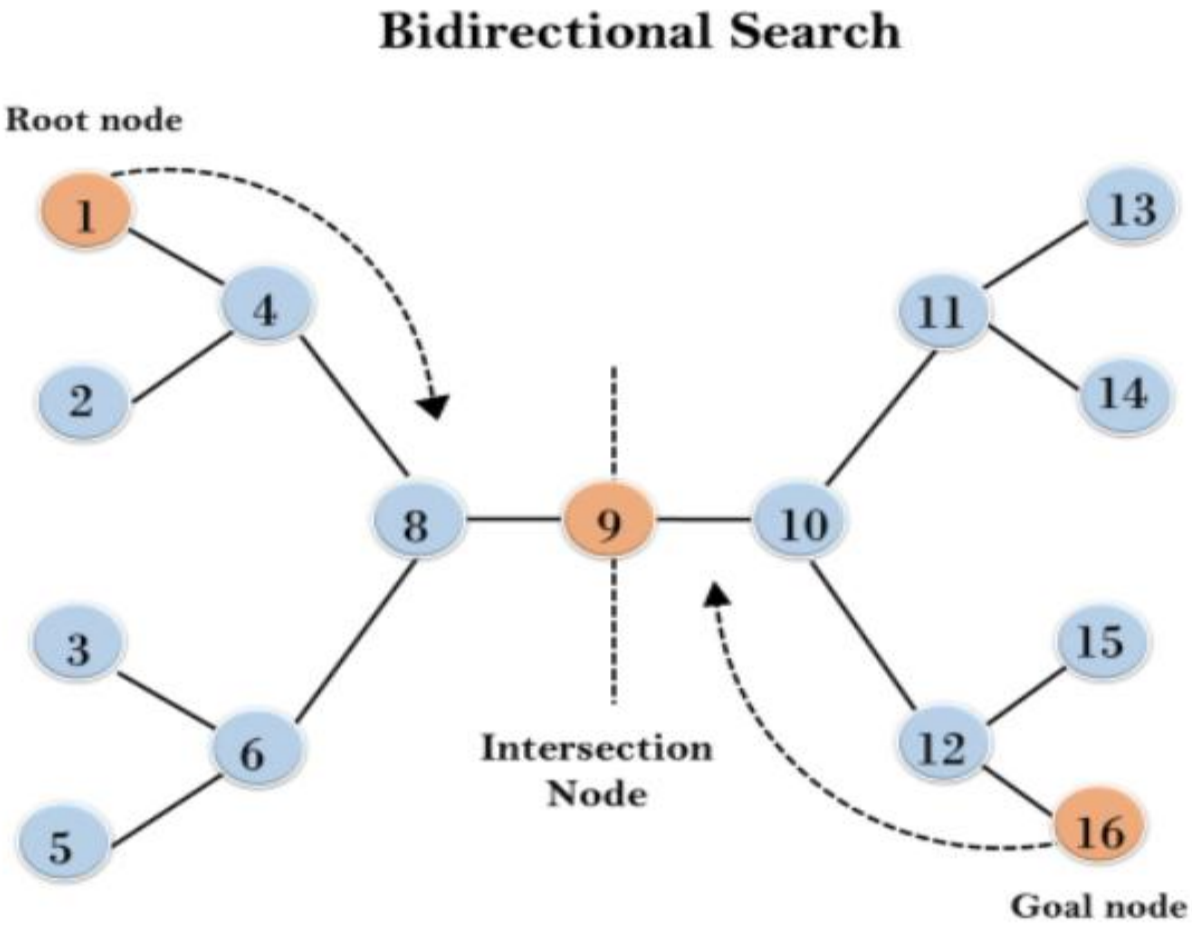
Failure to detect repeated states can turn linear problem into an exponential one!

Don't return to parent node
Don't generate successor = node's parent
Don't allow cycles
Don't revisit state
Keep every visited state in memory!
$O(b^d)$ (can be expensive)

Problems in which actions are reversible (e.g., routing problems or sliding-blocks puzzle). Also, in eg Chess; uses hash tables to check for repeated states. Huge tables 100M+ size but very useful.

See Tree-Search vs. Graph-Search. Need to be careful to maintain (path) optimality and completeness.

Bi directional Search - Example



In the below search tree, bidirectional search algorithm is applied. This algorithm divides one graph/tree into two sub-graphs. It starts traversing from node 1 in the forward direction and starts from goal node 16 in the backward direction.

The algorithm terminates at node 9 where two searches meet.

Bi directional Search

Completeness: Bidirectional Search is complete if we use BFS in both searches.

Time Complexity: Time complexity of bidirectional search using BFS is **$O(b^d)$**.

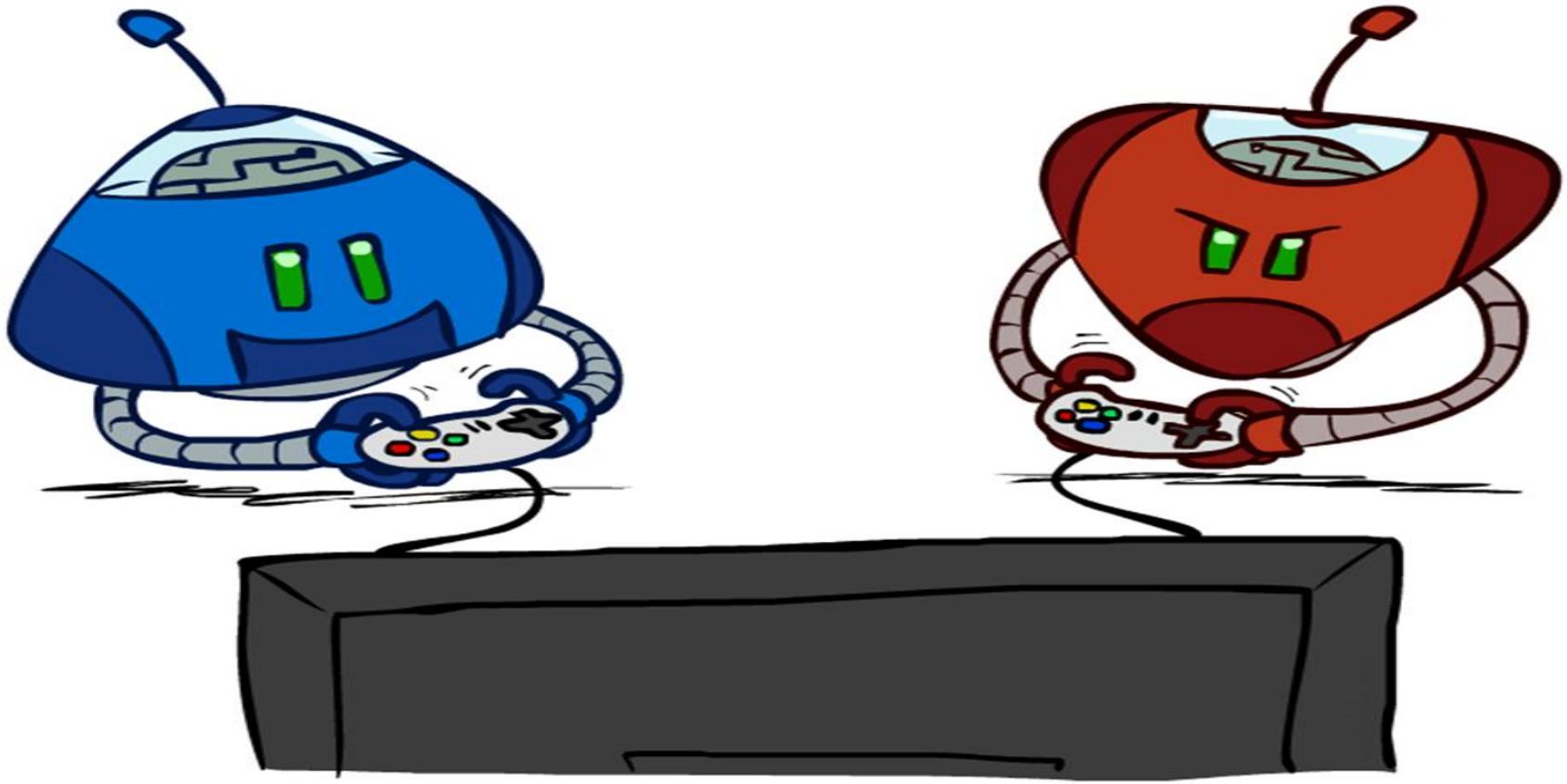
Space Complexity: Space complexity of bidirectional search is **$O(b^d)$**.

Optimal: Bidirectional search is Optimal.

Summary: General Uninformed Search

- **Original search ideas in AI where inspired by studies of human problem solving in, eg, puzzles, math, and games, but a great many AI tasks now require some form of search (e.g. find optimal agent strategy; active learning; constraint reasoning; NP-complete problems require search).**
- **Problem formulation usually requires abstracting away real-world details to define a state space that can feasibly be explored.**
- **Variety of uninformed search strategies**
- **Iterative deepening search uses only linear space and not much more time than other uninformed algorithms.**
- **Avoid repeating states / cycles.**

Games: Minimax and Alpha-Beta Pruning



Outline:

1. Overview
2. Minimax for Zero-Sum Games
3. α-β Pruning



Types of Games:

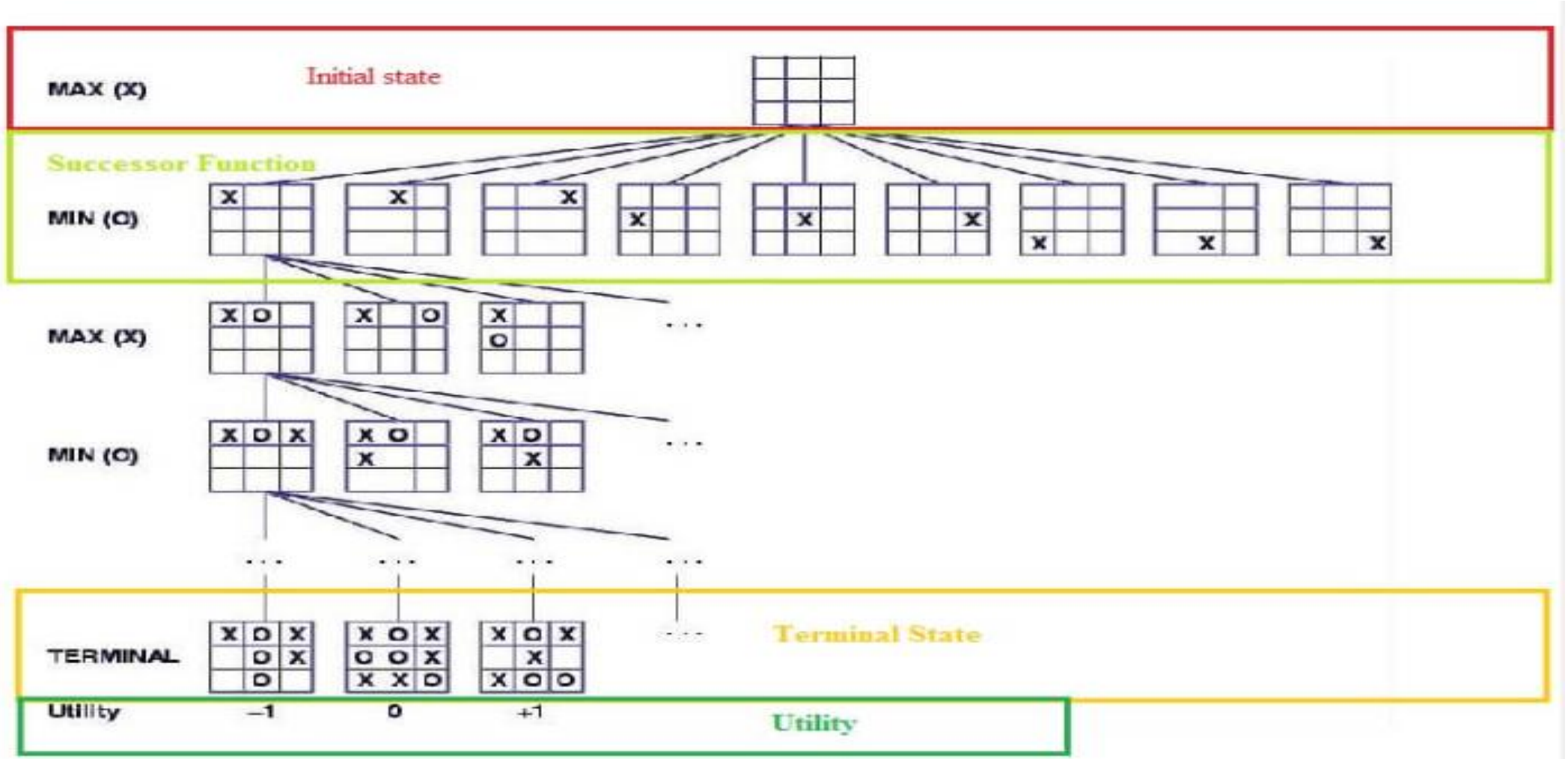
Game = task environment with > 1 agent

To Know:

1. Deterministic or stochastic?
2. Perfect information (fully observable)?
3. Two, three, or more players?
4. Teams or individuals?
5. Turn-taking or simultaneous?
6. Zero sum?

Output: algorithms for calculating a ***contingent plan*** (a.k.a. strategy or policy) which recommends a move for every possible eventuality

Example of a Game-Tree



Standard Games

Rational Agent→ A rational player
Assume all future moves will be optimal

Game formulation:

1. Initial state: s_0
2. Players: Player(s) indicates whose move it is
3. Actions: Actions(s) for player on move
4. Transition model: Result(s,a)
5. Terminal test: Terminal-Test(s)
6. Terminal values: Utility(s,p) for player p
7. Or just Utility(s) for player making the decision at root

Minimax :Like DFS
Time: $O(b^m)$
Space: $O(bm)$

For chess, b » 35, m » 100

Exact solution is completely infeasible

Humans can't do this either, so how do we play chess?

Zero-Sum Games





Zero-Sum Games

Agents have ***opposite*** utilities

Pure competition:

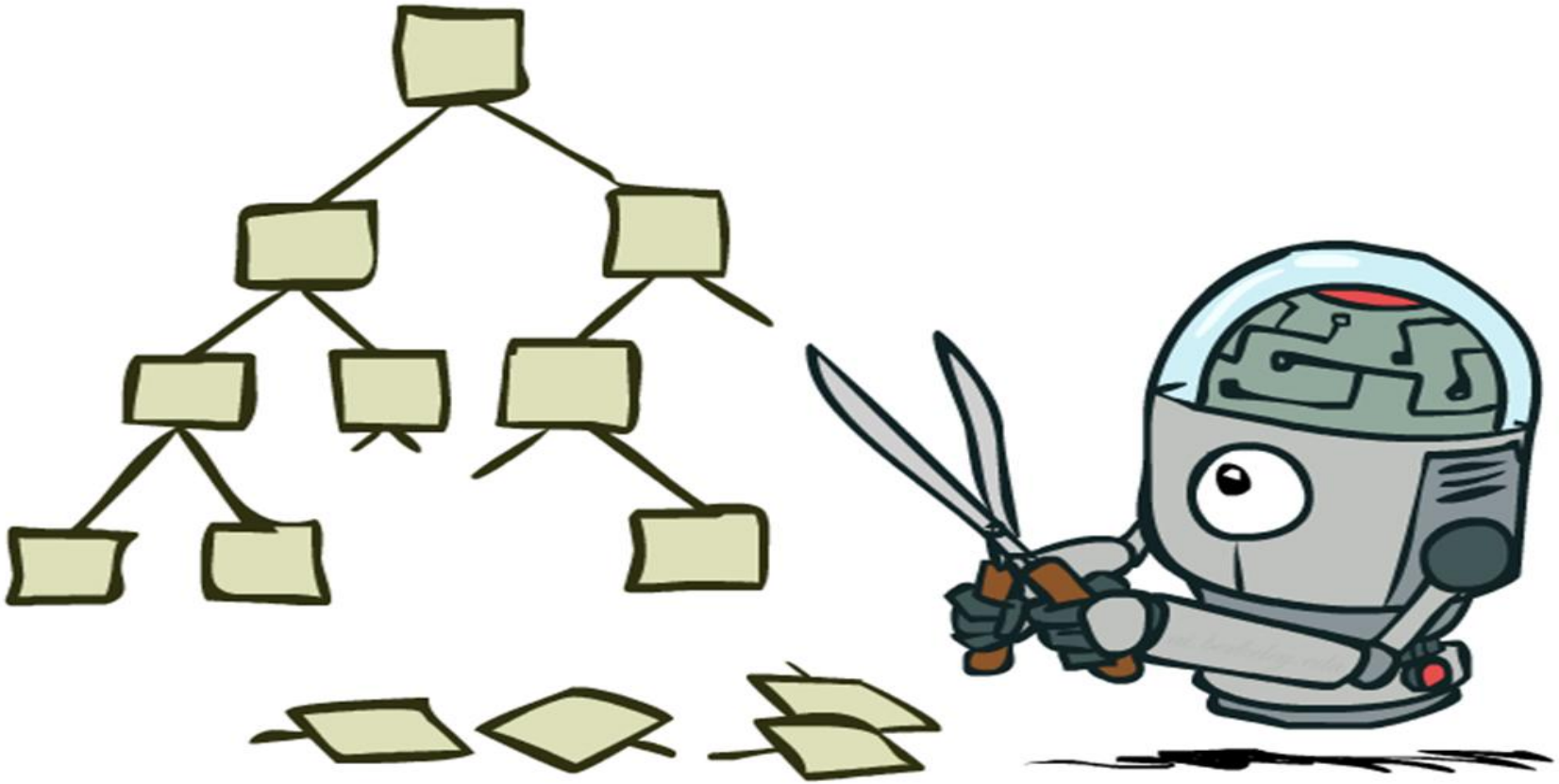
One ***maximizes***, the other ***minimizes***

General-Sum Games

Agents have ***independent*** utilities

Cooperation, indifference, competition, shifting alliances, and more are all possible

Game Tree Complexity



Saviour
Alpha-beta
Pruning

Alpha-Beta Pruning

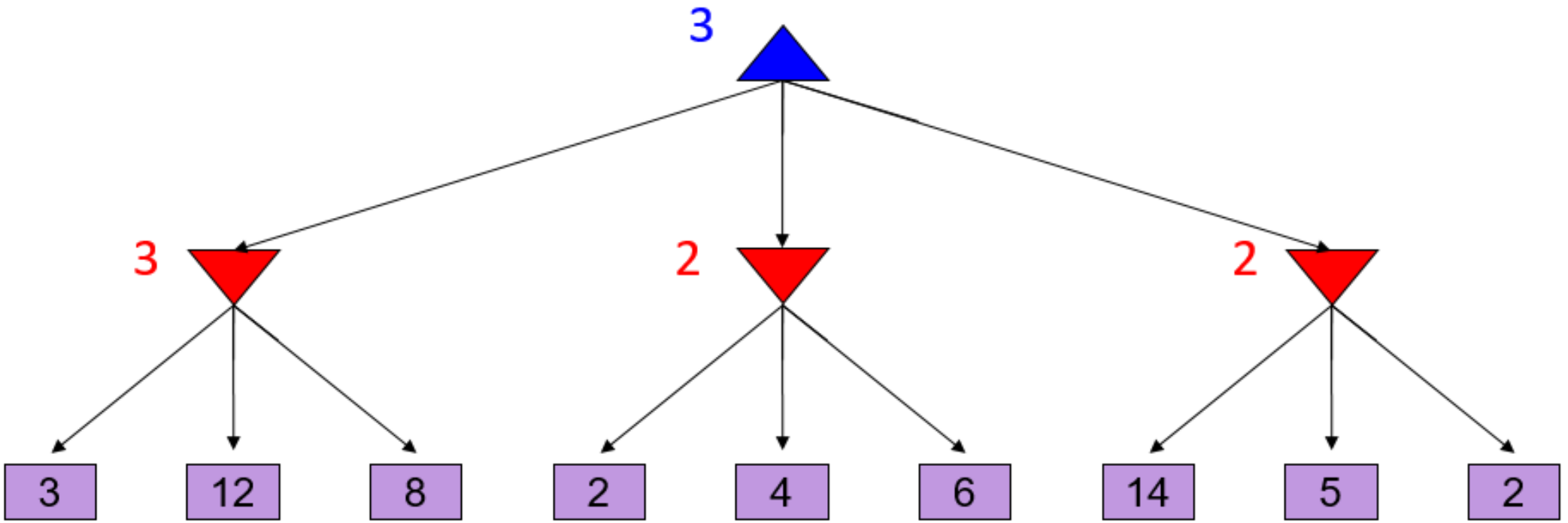
Principle:

In any game tree for any node **n** , if there is a node **m** either at the parent node of **n** or any where further up, then n will never be reached in the actual play!!!!!!

Alpha bound of a node: Max current value of all Max ancestors of the node.Exploration of a min node is stopped when its value equals or falls below alpha.

Beta bound of a node: Min current value of all Min ancestors of the node.Exploration of a max node is stopped when its value equals or exceeds beta.

Minimax Example

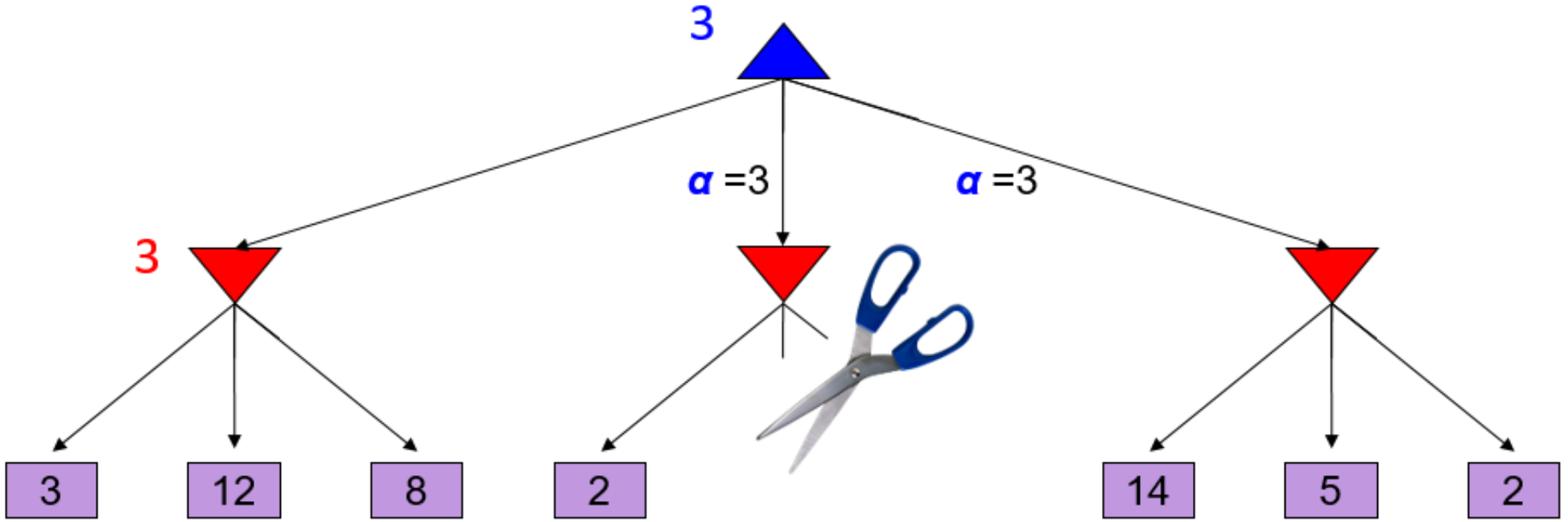


What can we Prune ?

α = best option so far from any MAX node on this path

For chess: only 35^{50} instead of 35^{100}!!

Yaaay!!!!!



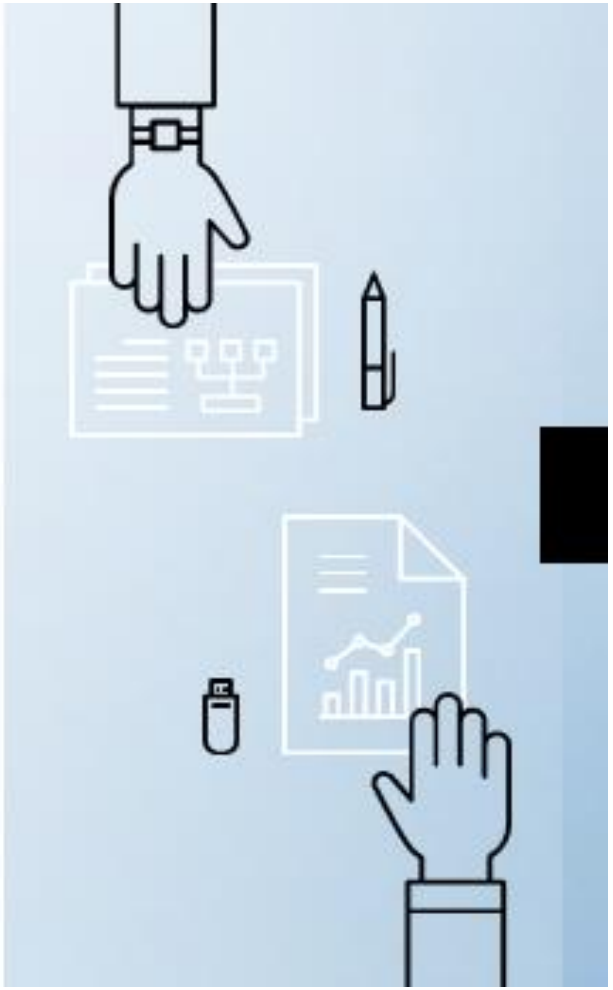
- ***The order of generation matters***: more pruning is possible if good moves come first

Why game playing ?

Why Game playing?

1. It's a good reasoning problem, formal and nontrivial.

2. Direct comparison with humans and other computer programs is easy.

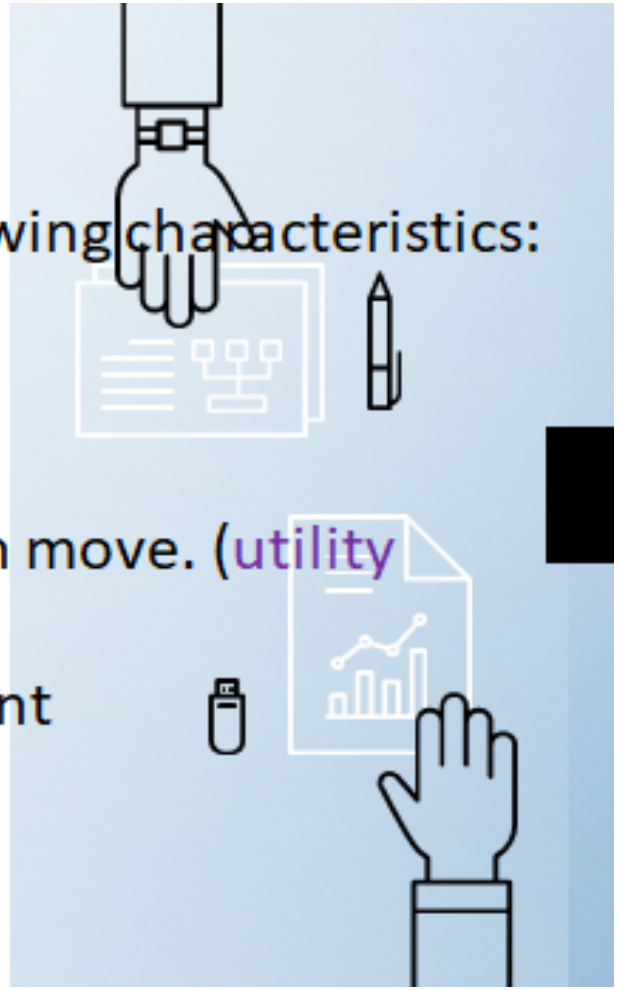


Characteristics of Game playing ?

Characteristics of Game playing

Mainly games of strategy with the following characteristics:

1. Sequence of moves to play
2. Rules that specify possible moves
3. Rules that specify a payment for each move. (utility function)
4. Objective is to maximize your payment



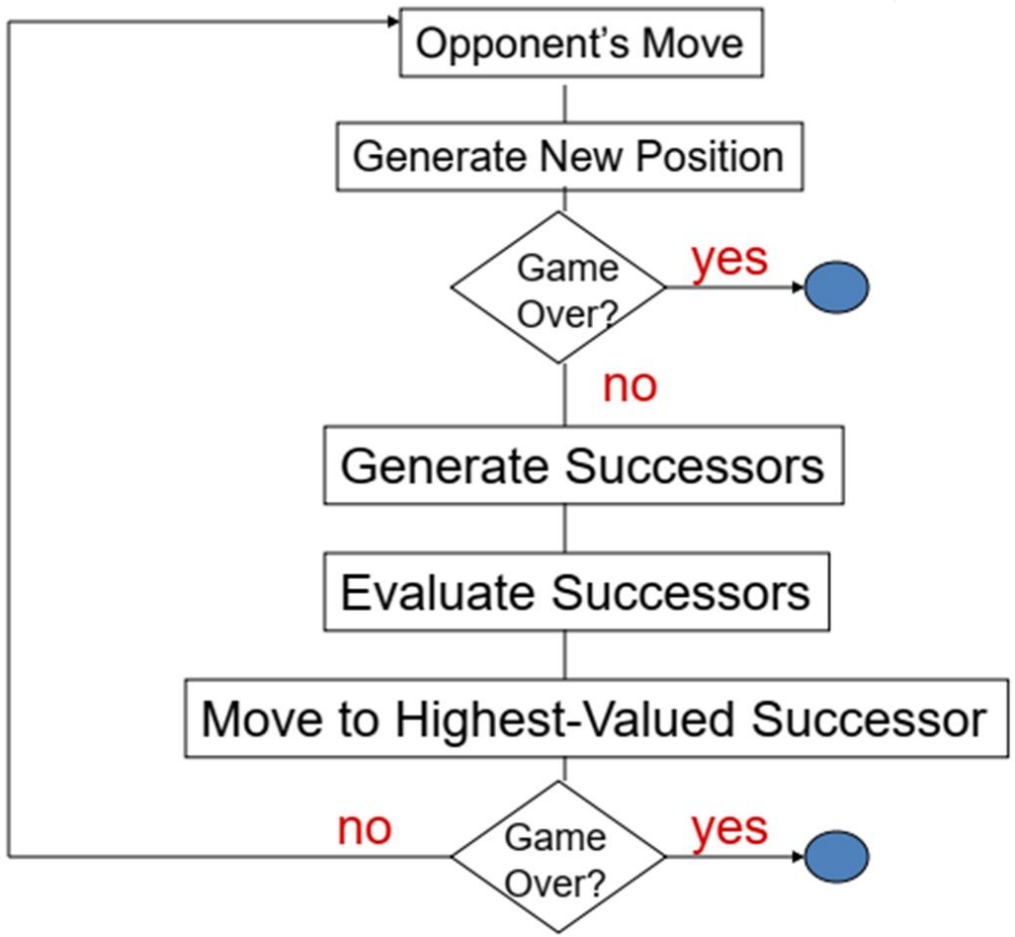
Games vs. Search Problems

✓ Two agent search strategy under competitive environments.

• Unpredictable opponent → specifying a move for every possible opponent reply.

• Time limits → unlikely to find goal, must approximate.

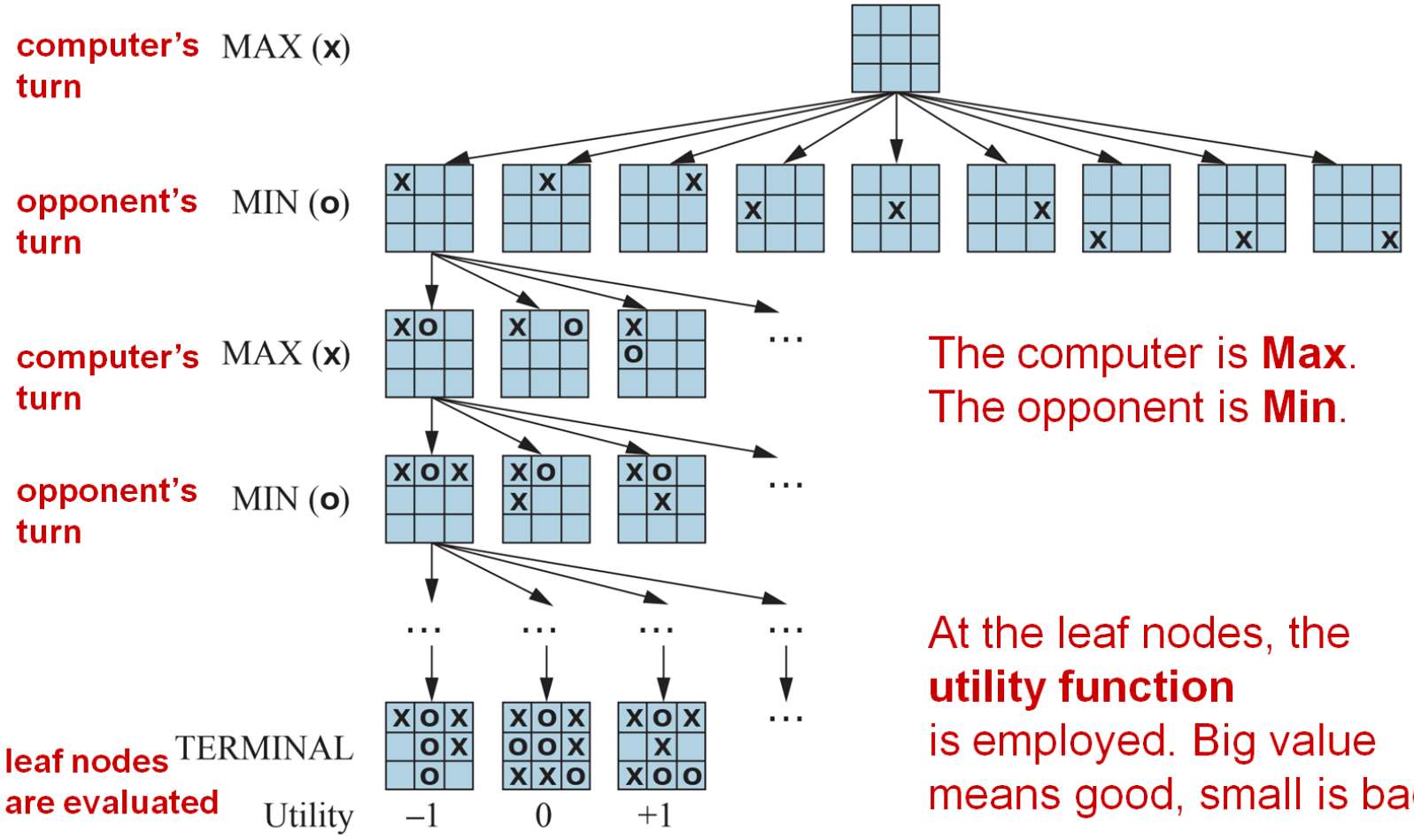
Two Player game



Games as Adversarial Search

- States:
  - ✓ board configurations.
- Initial state:
  - ✓ the board position and which player will move.
- Successor function:
  - ✓ returns list of (move, state) pairs, each indicating a legal move and the resulting state.
- Terminal test:
  - ✓ determines when the game is over.
- Utility function:
  - ✓ gives a numeric value in terminal states.
    (e.g., -1, 0, +1 for loss, tie, win)

Two player Game Tree (Example)



The computer is **Max**.
The opponent is **Min**.

At the leaf nodes, the **utility function** is employed. Big value means good, small is bad

Mini-Max Algorithm

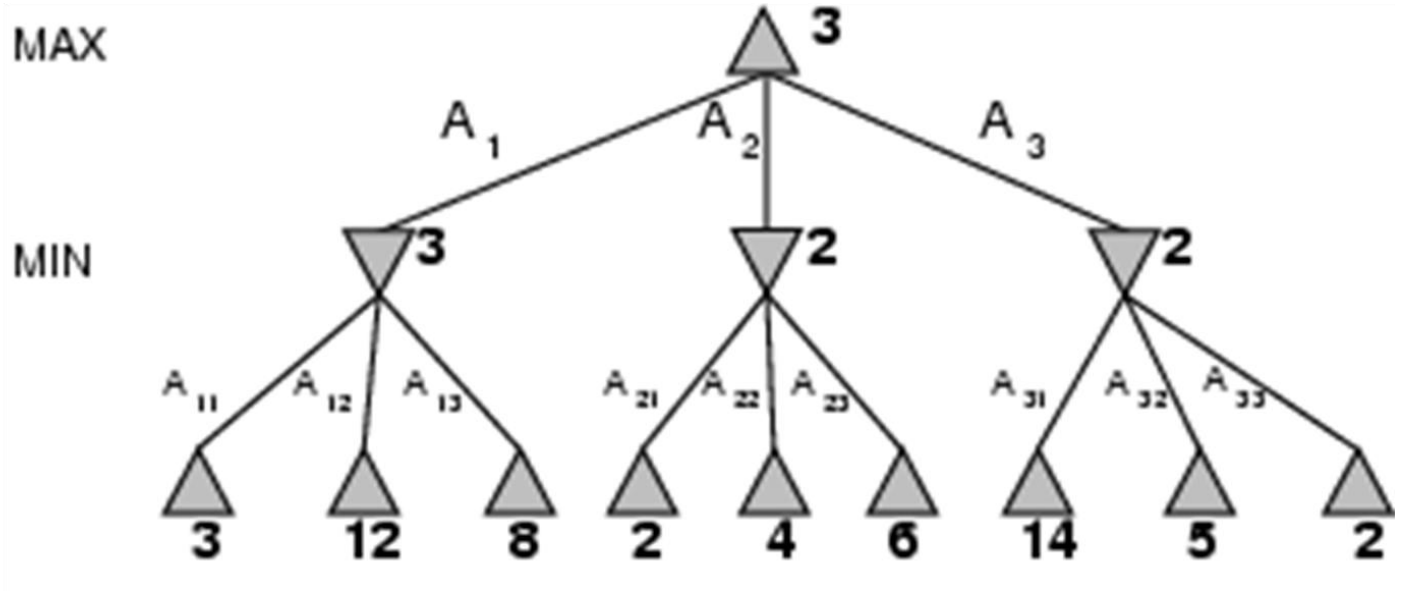
- move: a move by both players
- ply: a half-move

- utility function: the function applied to leaf nodes
- backed-up value
  - ✓ of a max-position: the value of its largest successor
  - ✓ of a min-position: the value of its smallest successor

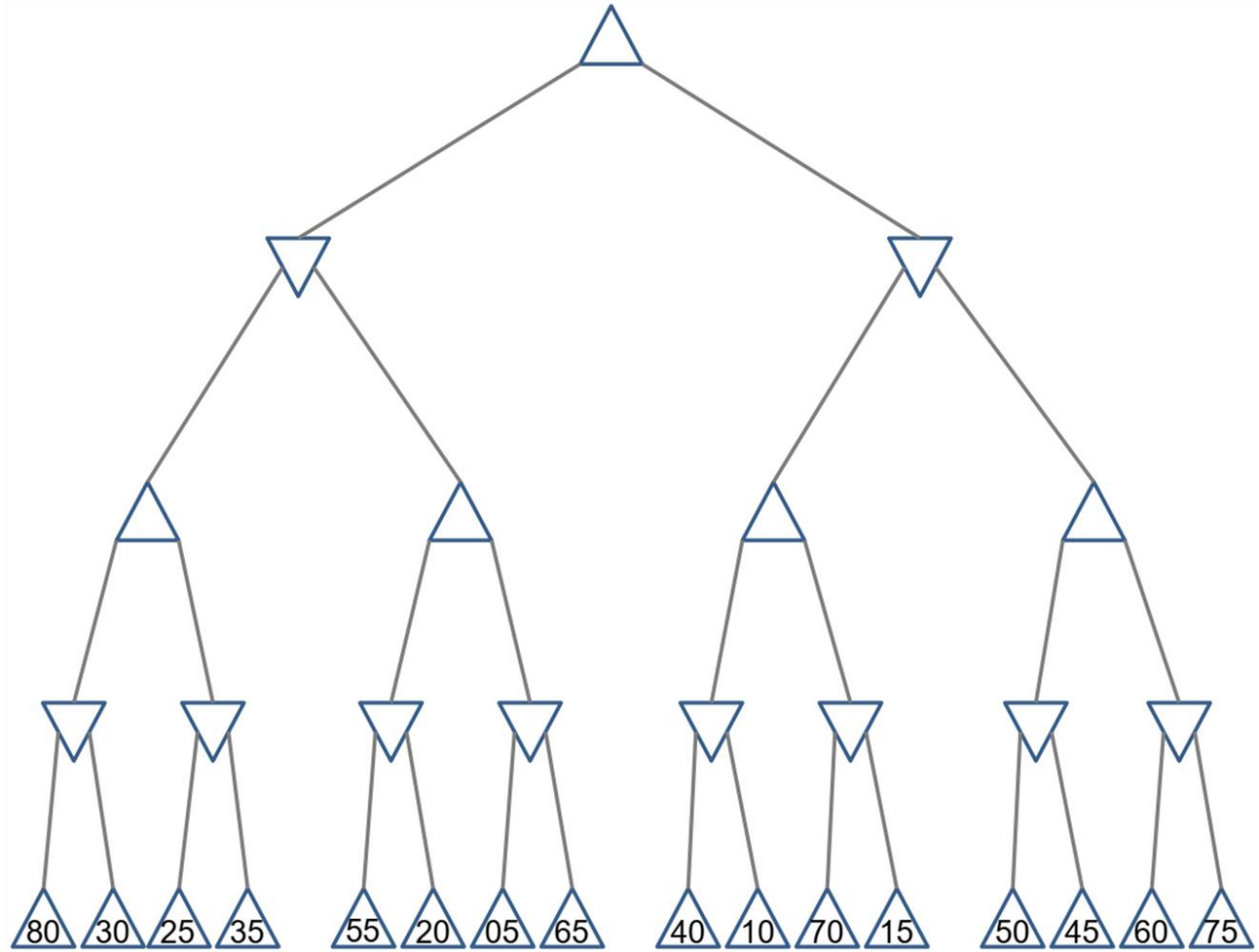
- minimax procedure: search down several levels; at the bottom level apply the utility function, back-up values all the way up to the root node, and that node selects the move.

Mini Max Toy Game

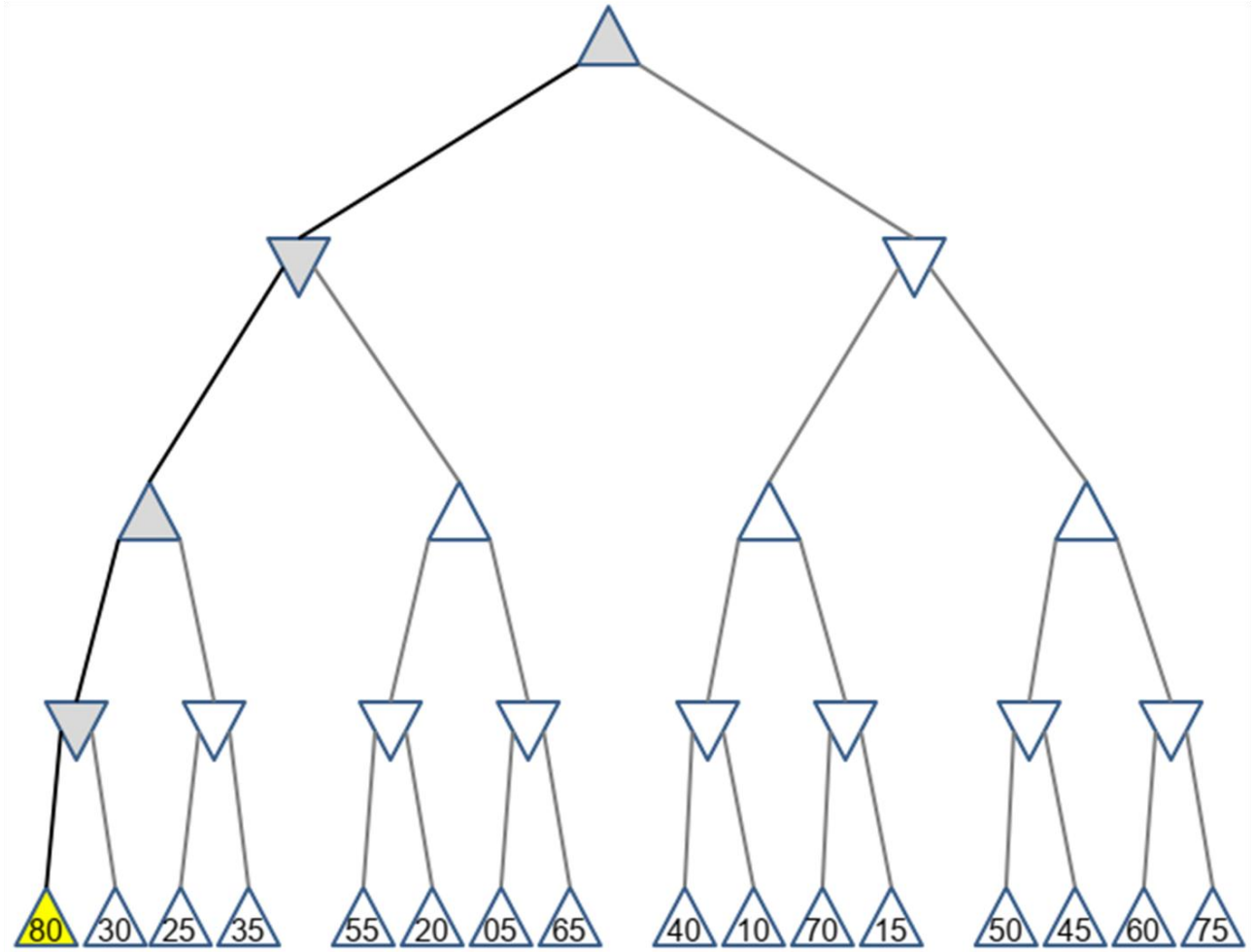
- Perfect play for deterministic games
- Idea: choose move to position with highest minimax value = best achievable payoff against best play
- E.g., 2-ply game:



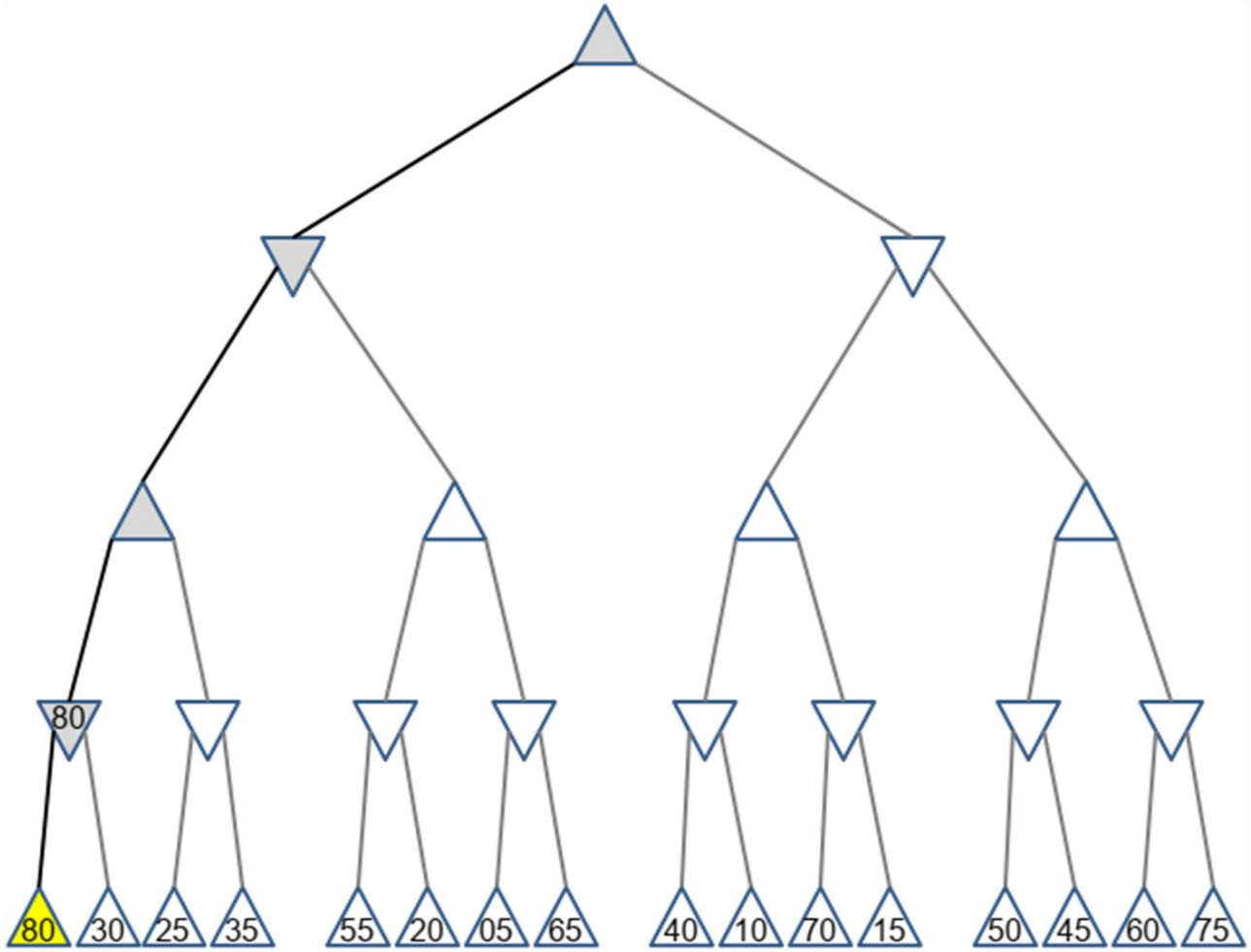
Mini-Max Search Tree



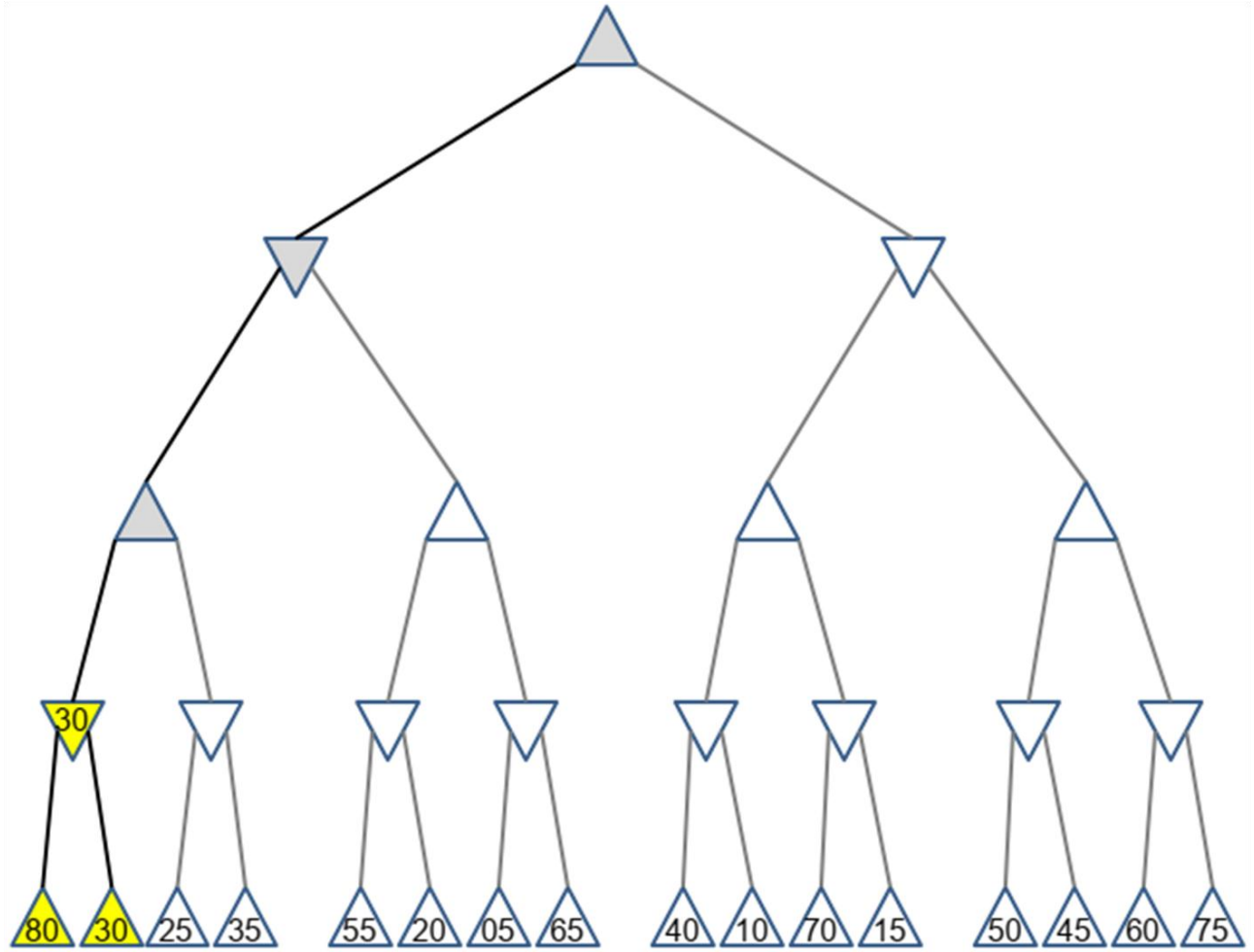
Mini-Max Search Tree



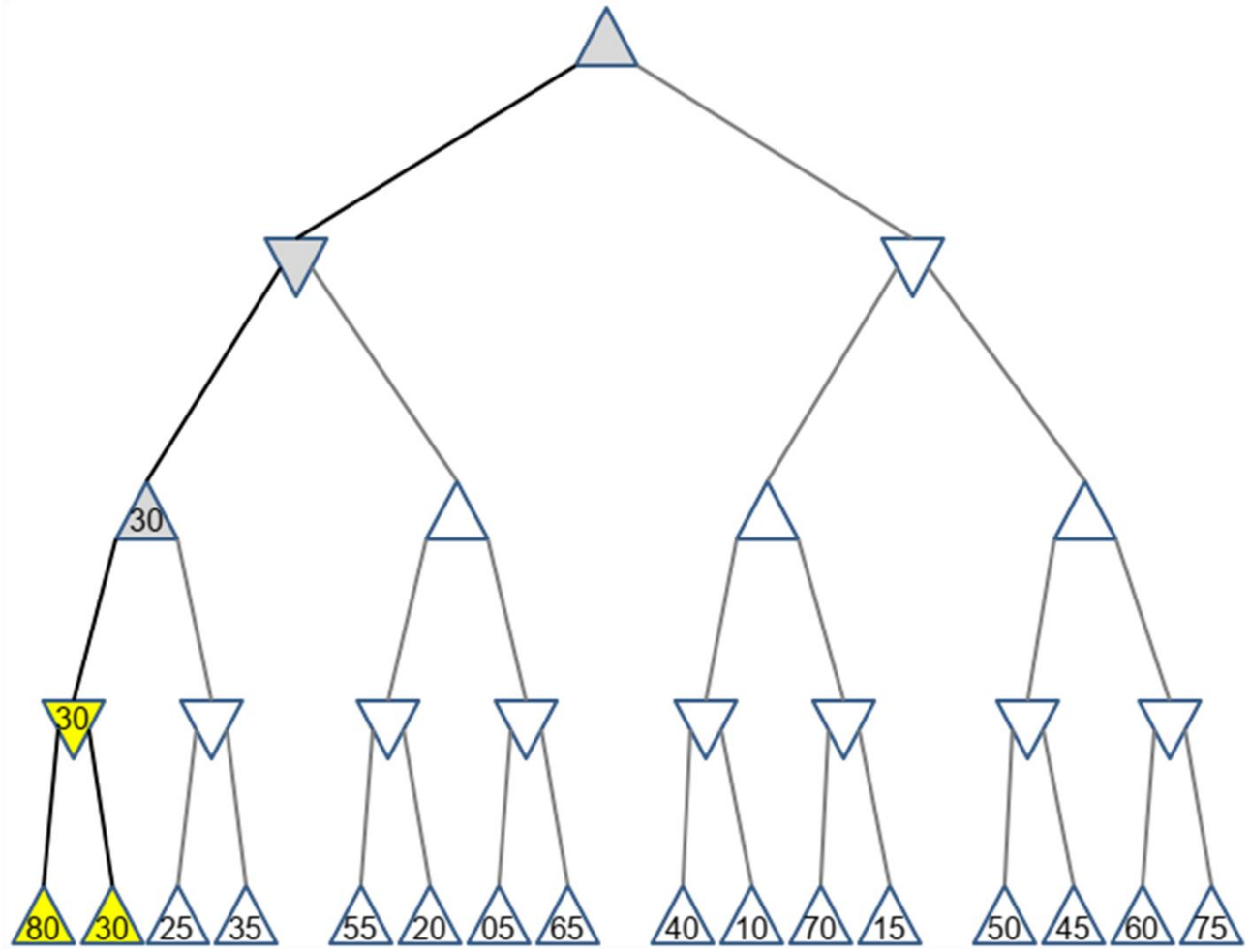
Mini-Max Search Tree



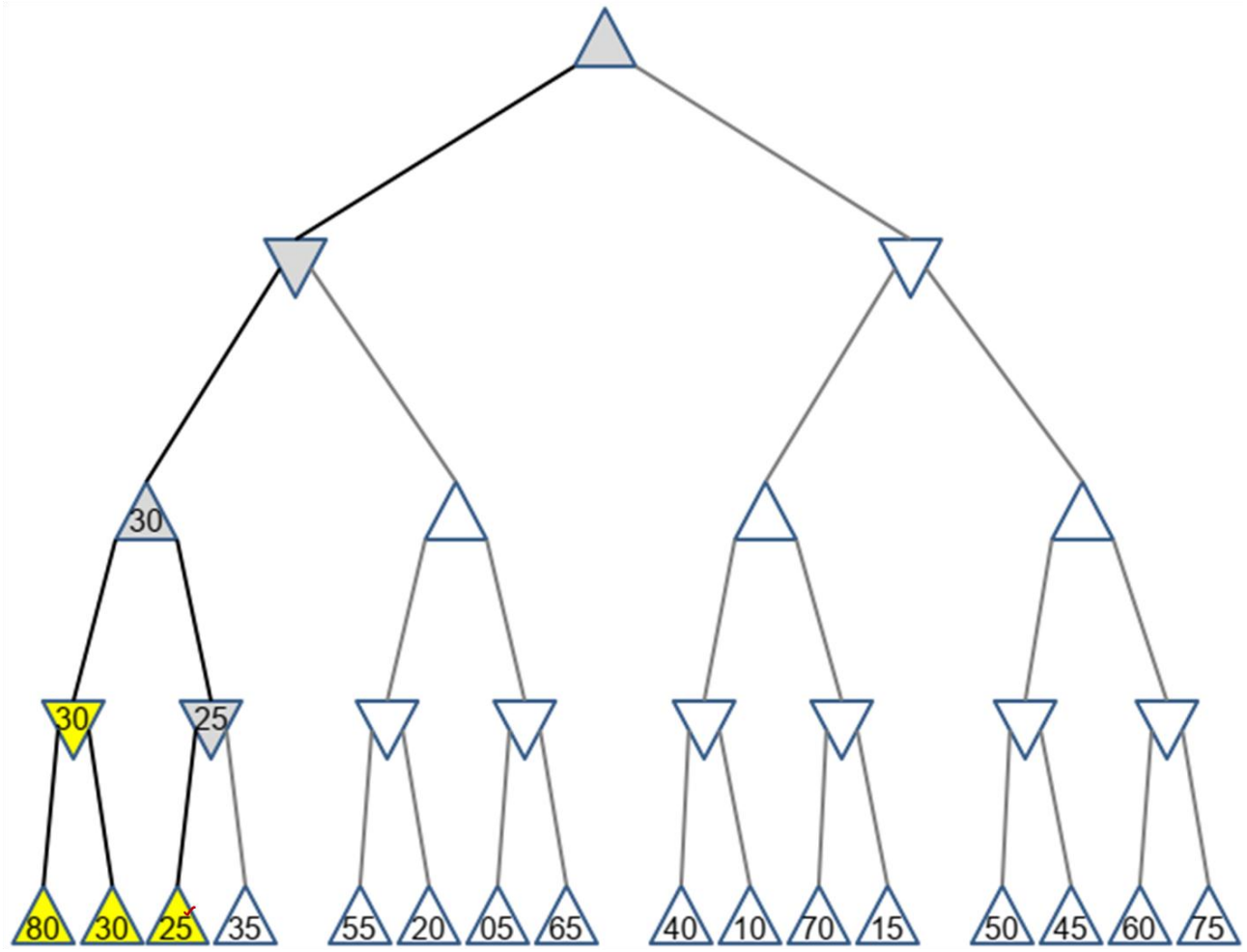
Mini-Max Search Tree



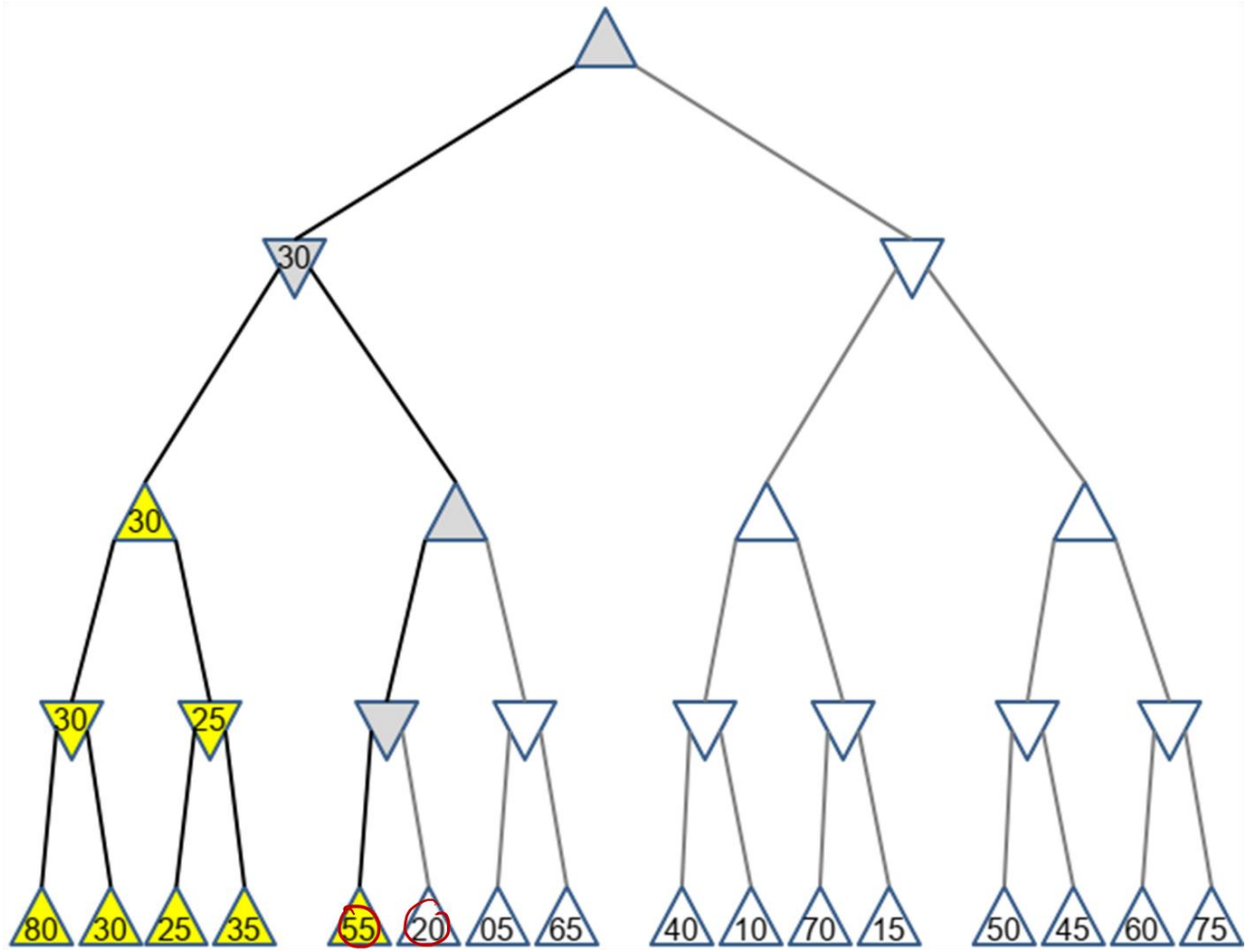
Mini-Max Search Tree



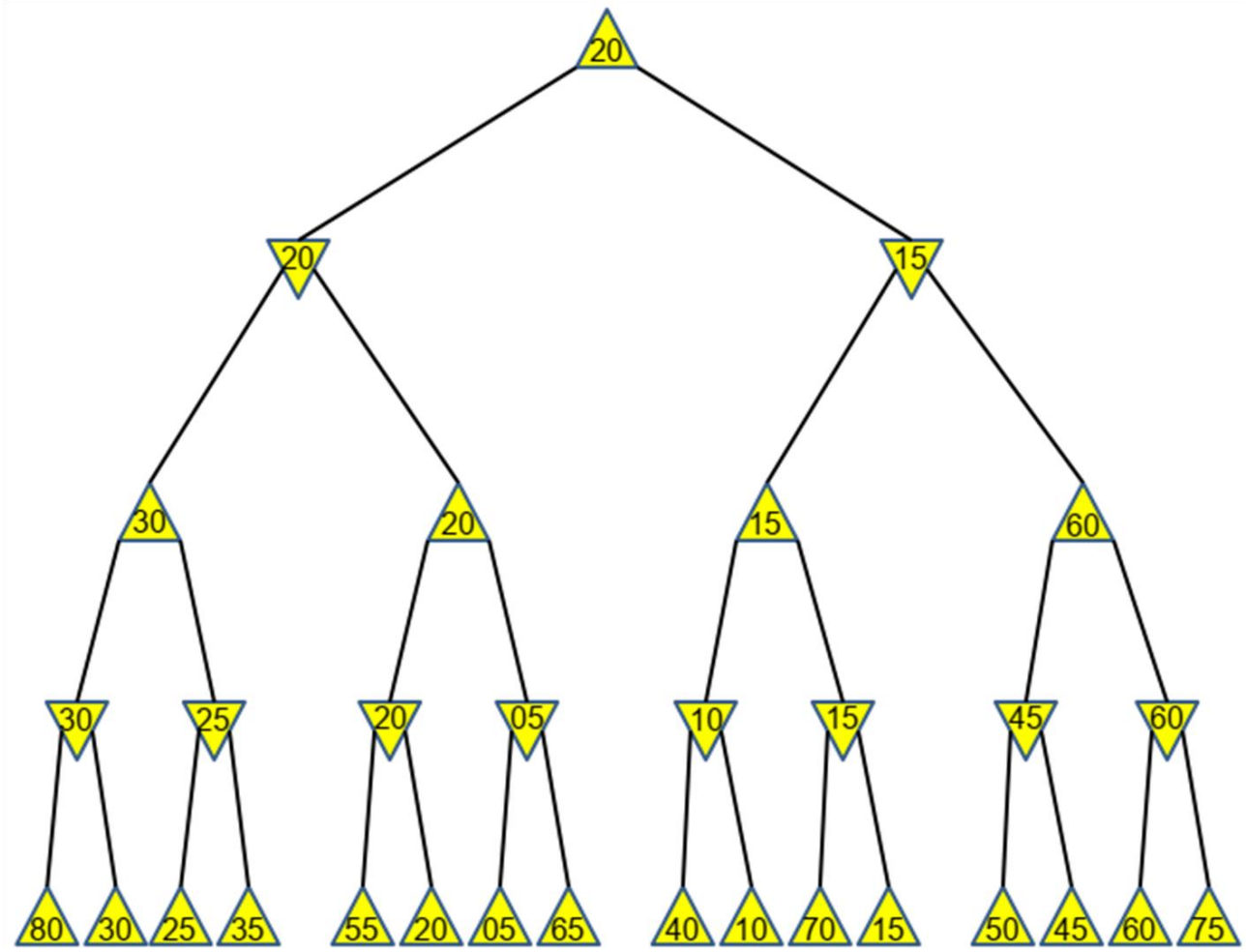
Mini-Max Search Tree



Mini-Max Search Tree



Mini-Max Search Tree

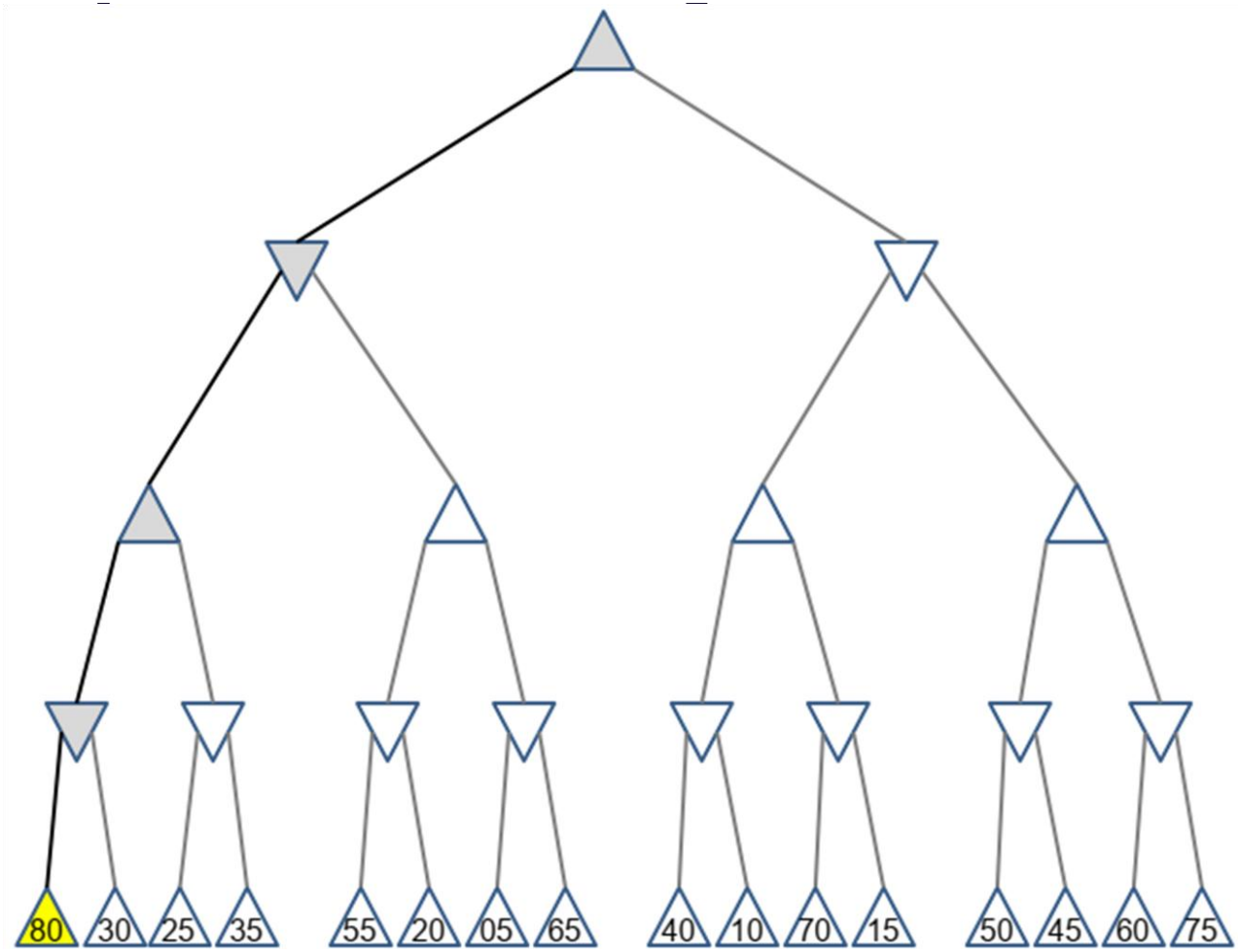


What kind of search tree for Mini-Max algorithm?

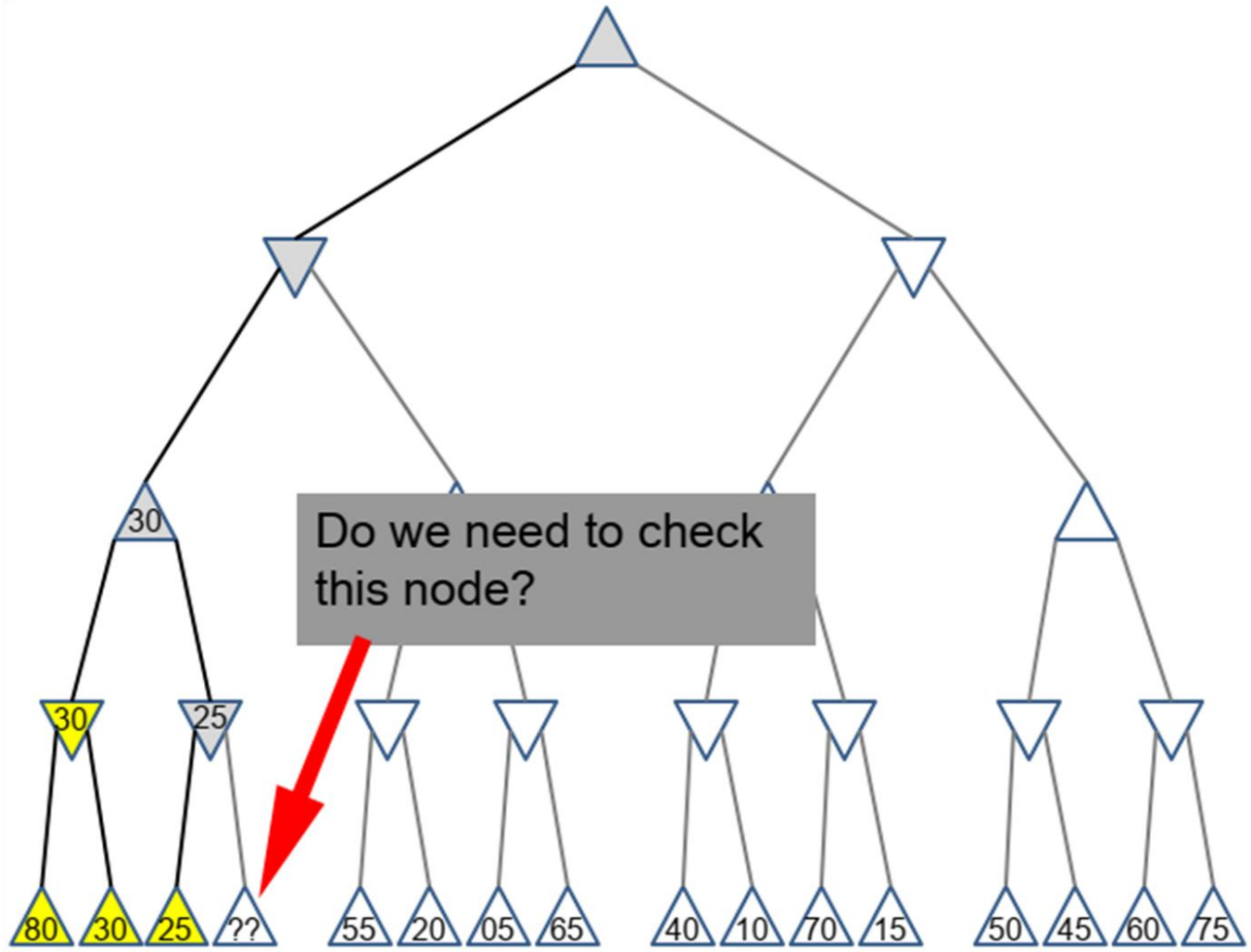
Properties of Mini-Max

- Complete?
  - Yes (if tree is finite)
- Optimal?
  - Yes (against an optimal opponent)
  - No (does not exploit opponent weakness against suboptimal opponent)
- Time complexity?
  - $O(b^m)$
- Space complexity?
  - $O(bm)$ (depth-first exploration)

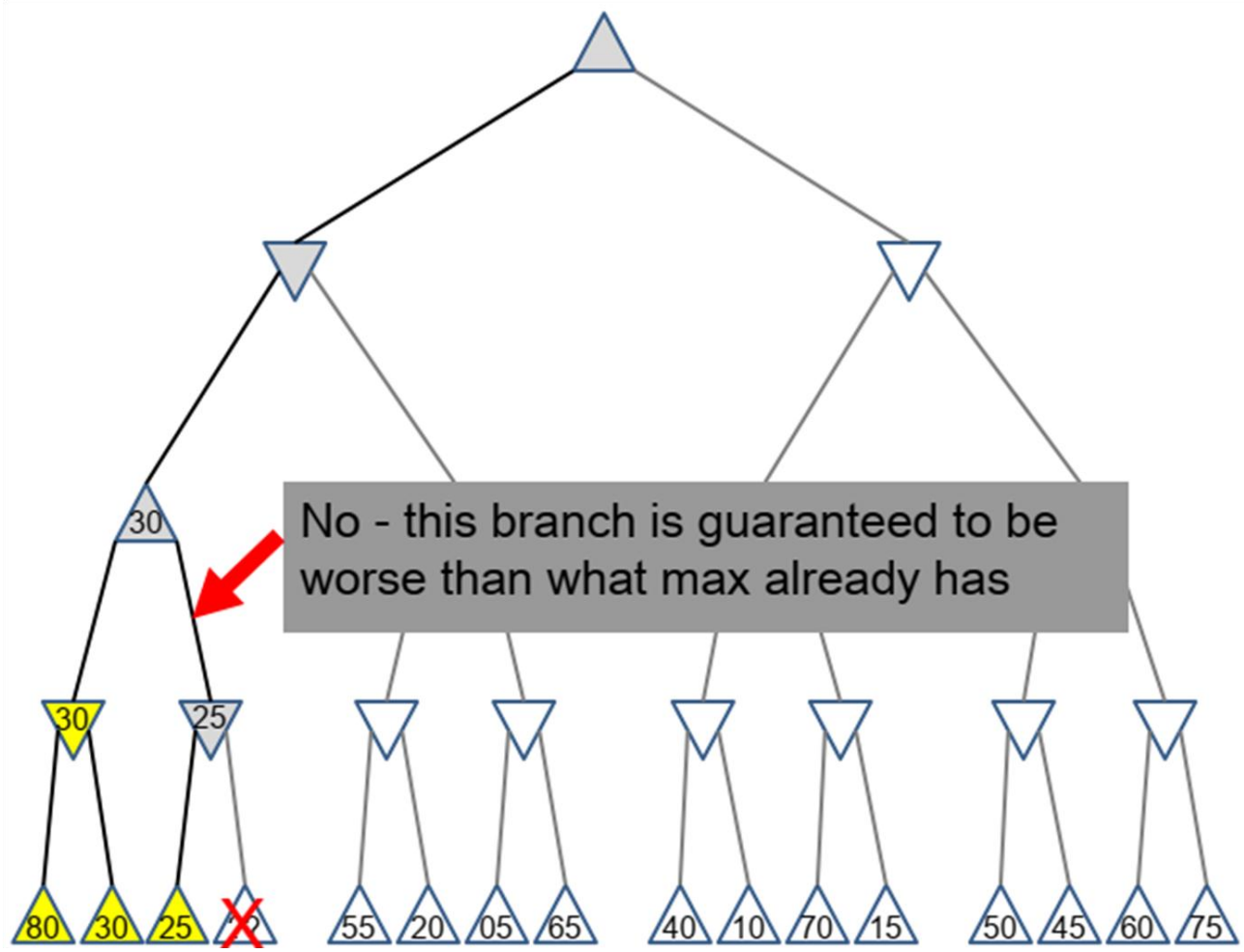
Why expand Unnecessary nodes ?



Mini-Max Search Tree



Mini-Max Search Tree



Alpha Beta

- The alpha-beta procedure can speed up a depth-first minimax search.

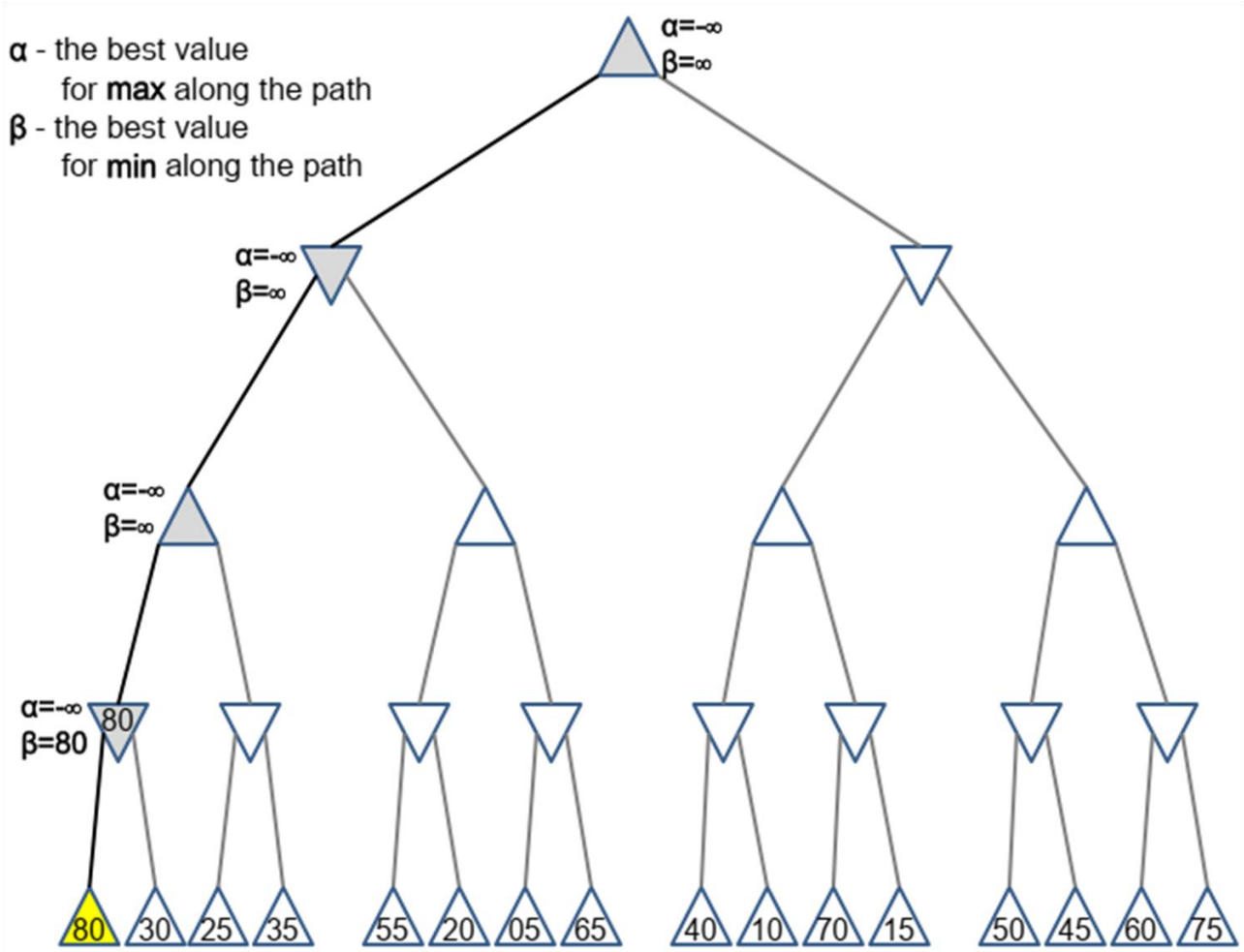
- Alpha: a lower bound on the value that a max node may ultimately be assigned.

$$v \geq \alpha$$

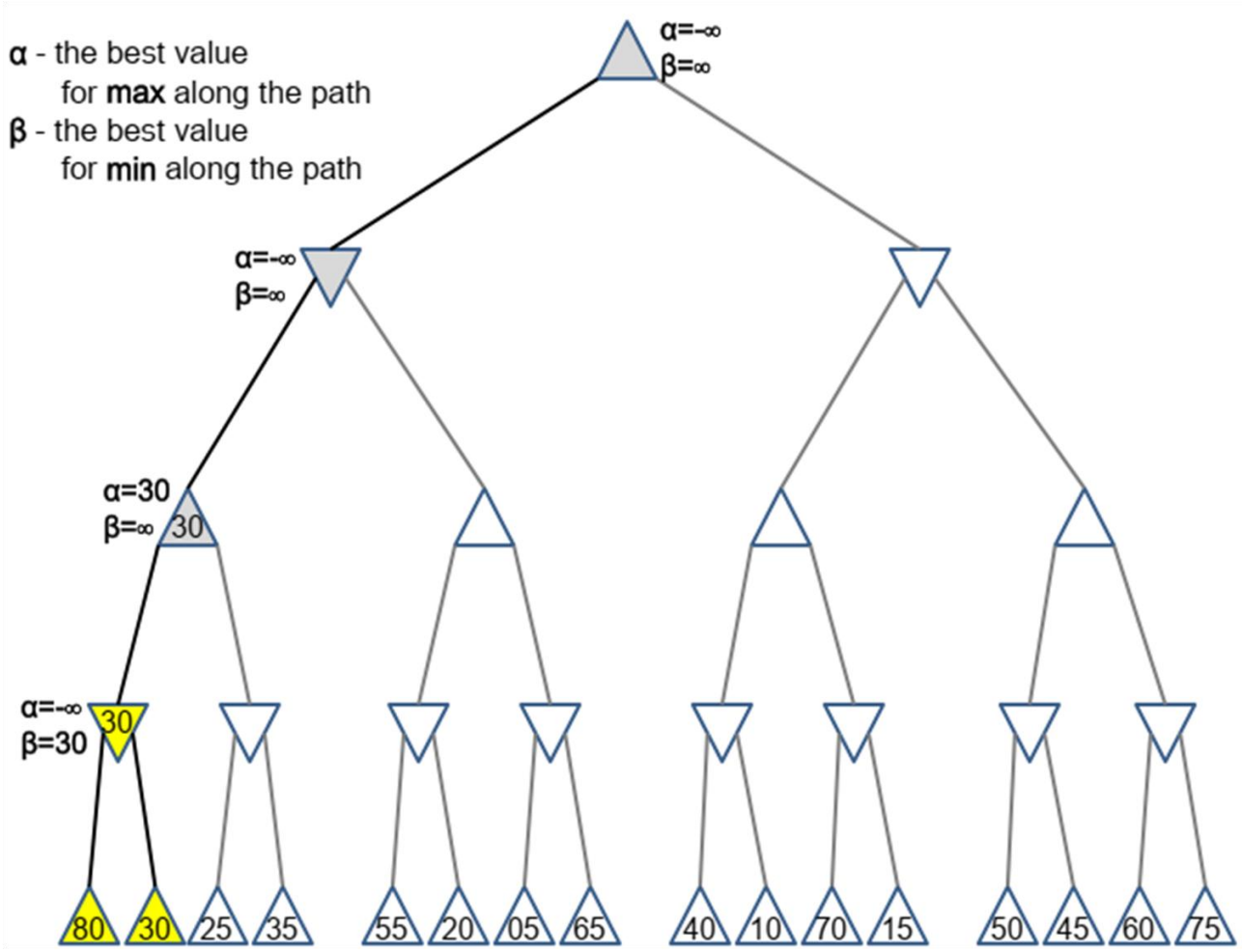
- Beta: an upper bound on the value that a minimizing node may ultimately be assigned.

$$v \leq \beta$$

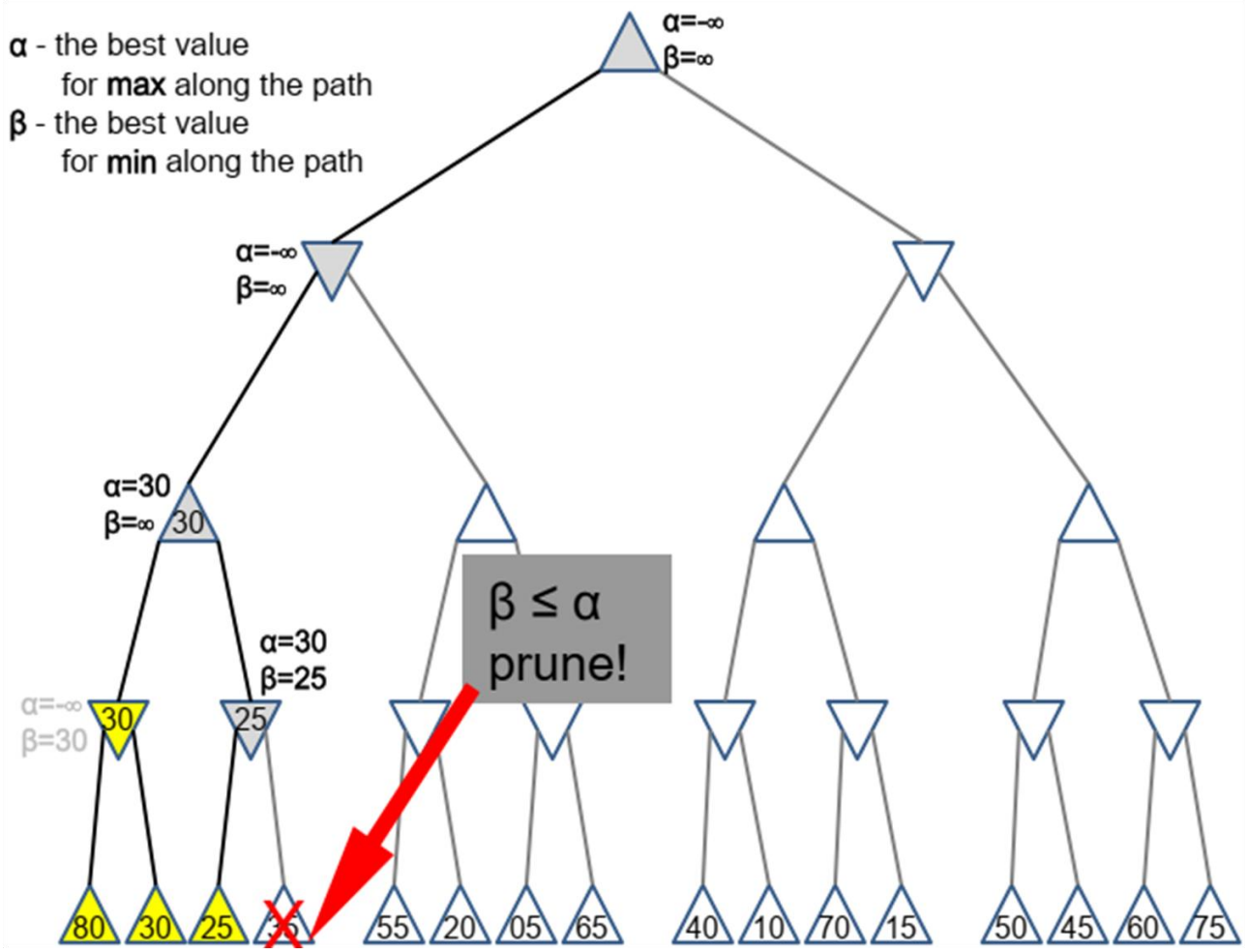
Mini-Max Search Tree



Mini-Max Search Tree

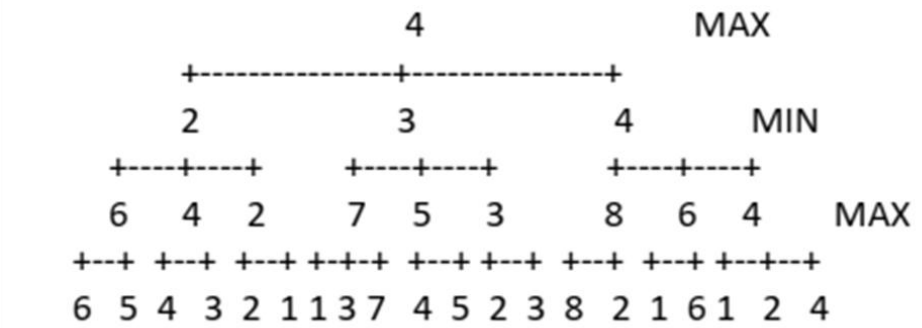


Mini-Max Search Tree

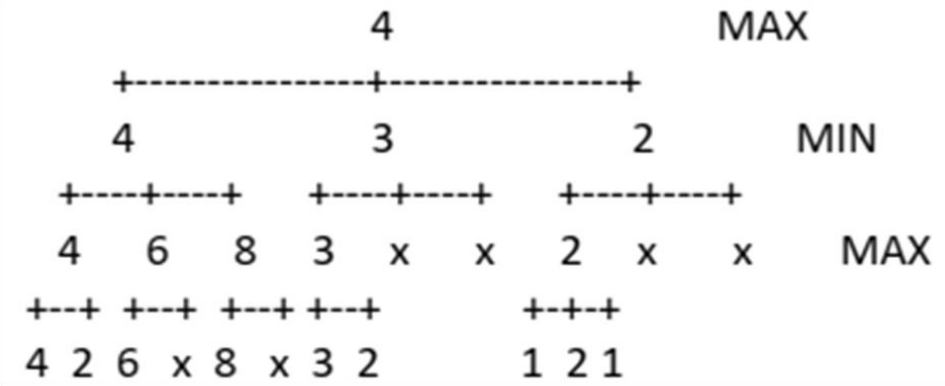


Bad and Good Cases for Alpha-Beta Pruning

- Bad: Worst moves encountered first



- Good: Good moves ordered first



If we can order moves, we can get more benefit from alpha-beta pruning

Properties of Alpha-Beta Pruning

- Pruning does not affect final result. This means that it gets the exact same result as does full minimax.
- Good move ordering improves effectiveness of pruning
  - With "perfect ordering," time complexity = $O(b^{\frac{m}{2}})$
    → doubles depth of search
- A simple example of reasoning about 'which computations are relevant' (a form of metareasoning)

Local Search and Optimization

- Local search techniques and optimization
  - Hill-climbing
  - Gradient methods
  - Simulated annealing
  - Genetic algorithms

Beyond Classical Search: "Local Search Algorithm"

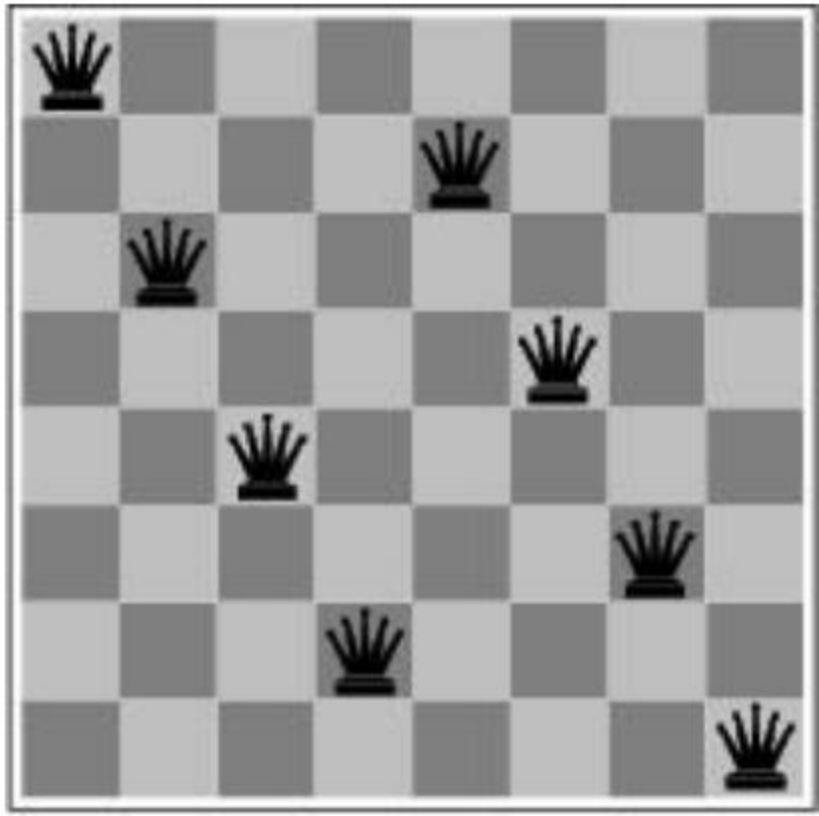
Local search algorithms operate by searching from a start state to neighboring states, without keeping track of the paths, nor the set of states that have been reached. That means they are not systematic—they might never explore a portion of the search space where a solution actually resides.

Advantages:

- ✓ Uses very little memory
- ✓ Can often find reasonable solutions in large or infinite state spaces for which systematic algorithms are unsuitable.

Path vs. State Optimization

- Classical Search: path to goal is solution to problem
  - ✓ systematic exploration of search space.



- Local Search: a state is solution to problem
  - ✓ for some problems path is irrelevant.
  - ✓ **E.g.**, 8-queens

Satisfaction vs. Optimization



Reach the goal node
Constraint satisfaction



Optimize(objective f_n)
Constraint Optimization

Local Search Optimization

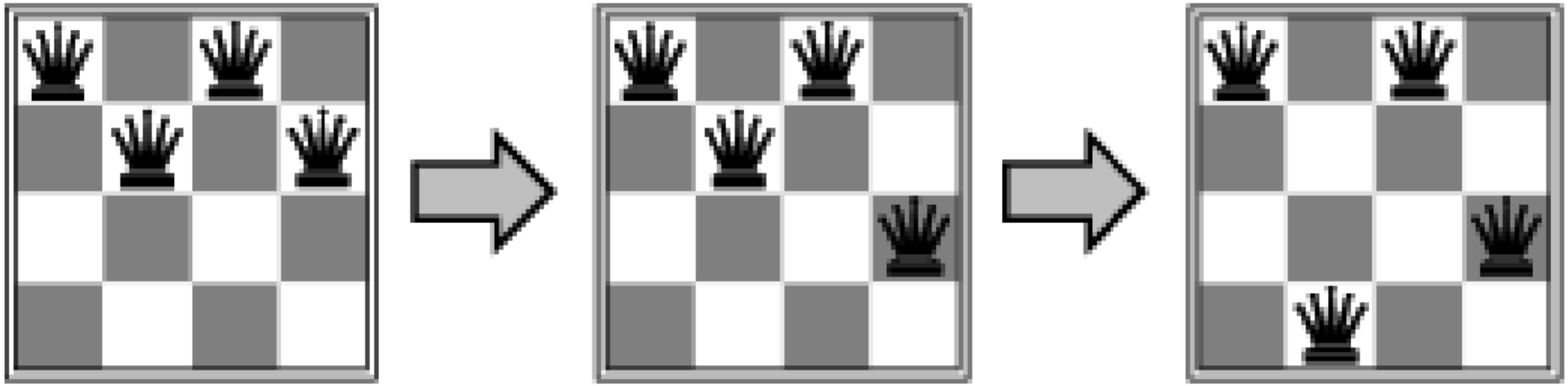
- Local search
  - ✓ Keep track of single current state
  - ✓ Move only to neighboring states
  - ✓ Ignore paths

- Advantages:
  - ✓ Use very little memory
  - ✓ Can often find reasonable solutions in large or infinite (continuous) state spaces.

- “Pure optimization” problems
  - ✓ All states have an objective function
  - ✓ Goal is to find state with max (or min) objective value
  - ✓ Does not quite fit into path-cost/goal-state formulation
  - ✓ Local search can do quite well on these problems.

Example: n-queens

- Put n queens on an $n \times n$ board with no two queens on the same row, column, or diagonal



- Formulating it in optimization problem!!!

Hill Climbing Algorithm in Artificial Intelligence:

- Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem.
- **It terminates when it reaches a peak value where no neighbor has a higher value**.
- Hill climbing algorithm is a technique which is used for optimizing the mathematical problems.
- One of the widely discussed examples of **Hill climbing algorithm** is Traveling-salesman Problem in which we need to minimize the distance traveled by the salesman.

Hill Climbing Algorithm in Artificial Intelligence:

- It is also called **greedy local search** as it only looks to its good immediate neighbor state and not beyond that.
- **A node of hill climbing algorithm has two components which are state and value.**
- Hill Climbing is mostly used when a good heuristic is available.
- In this algorithm, we don't need to maintain and handle the search tree or graph as **it only keeps a single current state.**

Features of Hill Climbing:

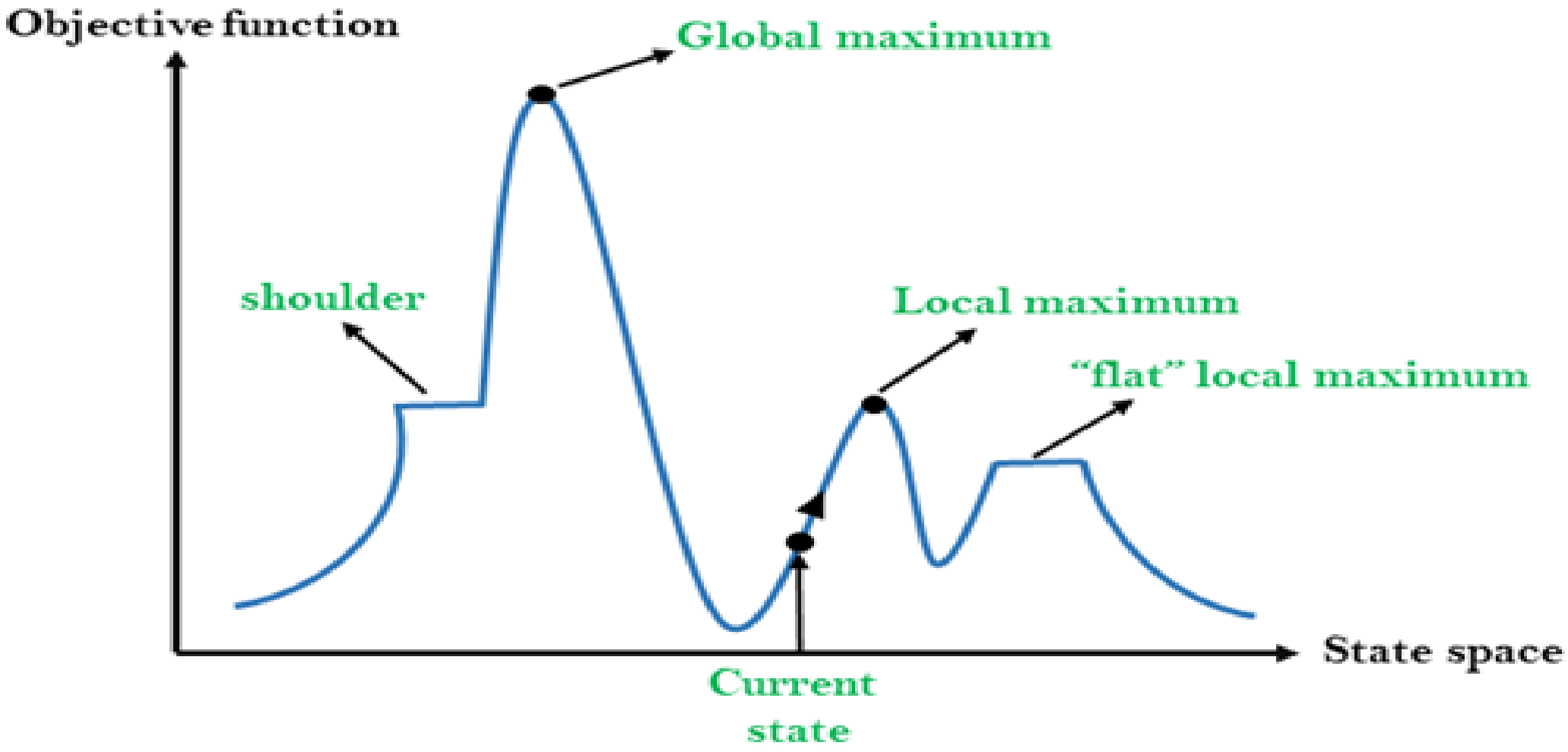
Following are some main features of Hill Climbing Algorithm:

- **Generate and Test variant:** Hill Climbing is the variant of Generate and Test method. The Generate and Test method produce feedback which helps to decide which direction to move in the search space.
- **Greedy approach:** Hill-climbing algorithm search moves in the direction which optimizes the cost.
- **No backtracking:** It does not backtrack the search space, as it does not remember the previous states.

State-space Diagram for Hill Climbing:

* The state-space landscape is a graphical representation of the hill-climbing algorithm which is showing a graph between various states of algorithm and Objective function/Cost.
* On Y-axis we have taken the function which can be an objective function or cost function, and state-space on the x-axis.
* If the function on Y-axis is cost then, the goal of search is to find the global minimum and local minimum.
* If the function of Y-axis is Objective function, then the goal of the search is to find the global maximum and local maximum.

State-space Diagram for Hill Climbing:



Different regions in the state space landscape:

* **Local Maximum:** Local maximum is a state which is better than its neighbor states, but there is also another state which is higher than it.
* **Global Maximum:** Global maximum is the best possible state of state space landscape. It has the highest value of objective function.
* **Current state:** It is a state in a landscape diagram where an agent is currently present.
* **Flat local maximum:** It is a flat space in the landscape where all the neighbor states of current states have the same value.
* **Shoulder:** It is a plateau region which has an uphill edge.

Hill-climbing search: 8-queens problem



- Need to convert to an optimization problem
- *h* = number of pairs of queens that are attacking each other
- *h* = *17* for the above state (Objective Function)

Hill-climbing search: 8-queens problem (Defining Search Space)

- **State**
  - All 8 queens on the board in some configuration
- **Successor function**
  - move a single queen to another square in the same column.
- **Example of a heuristic function h(n):**
  - the number of pairs of queens that are attacking each other
  - (so we want to minimize this)

Hill-climbing (Greedy Local Search) max version

```
function HILL-CLIMBING( problem) return a state that is a local maximum
        input: problem, a problem
        local variables: current, a node.
                         neighbor, a node.

current ← MAKE-NODE(INITIAL-STATE[problem])

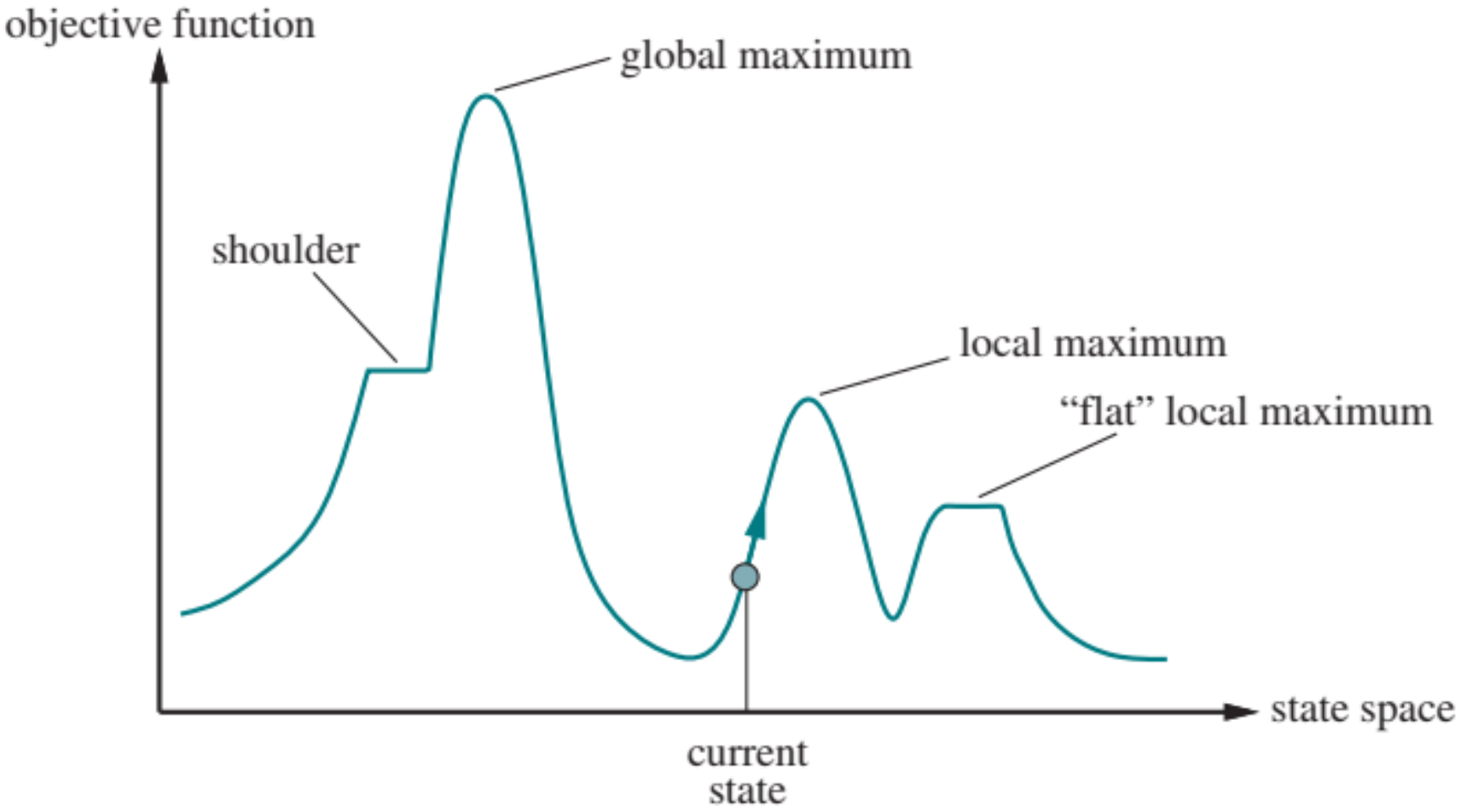
loop do
        neighbor ← a highest valued successor of current
        if VALUE [neighbor] ≤ VALUE[current] then return STATE[current]
                current ← neighbor
```

min version will reverse inequalities and look for lowest valued successor

Hill-climbing Search

- “a loop that continuously moves towards increasing value”
  - ✓ terminates when a peak is reached
  - ✓ Aka greedy local search
- Value can be either
  - ✓ Objective function value
  - ✓ Heuristic function value (minimized)
- Hill climbing does not look ahead of the immediate neighbors
- Can randomly choose among the set of best successors
  - ✓ if multiple have the best value

Hill-climbing Search

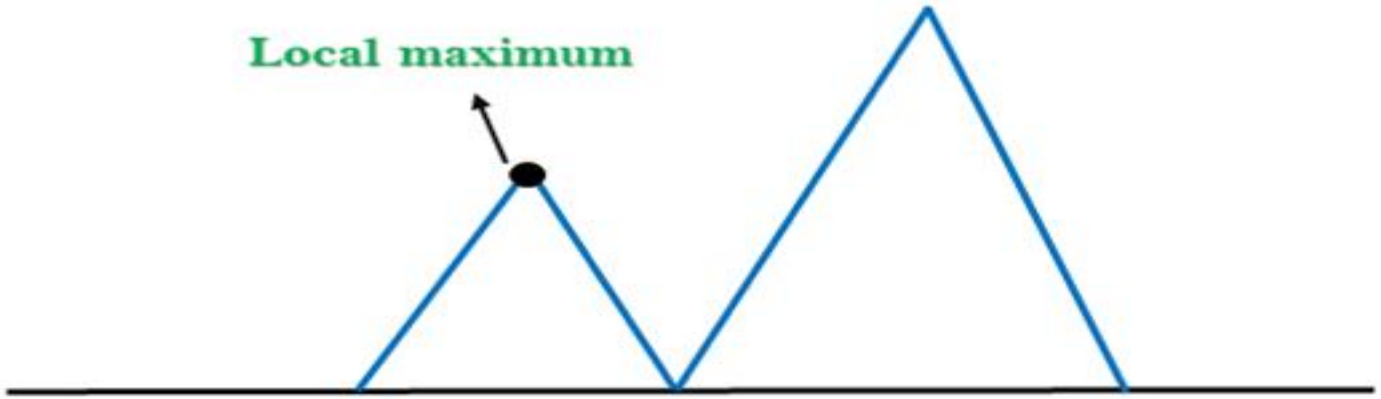


Hill Climbing gets stuck in local minima!!!

Example (8-Puzzle)

Problems in Hill Climbing Algorithm:

* **1. Local Maximum:** A local maximum is a peak state in the landscape which is better than each of its neighboring states, but there is another state also present which is higher than the local maximum.
* **Solution:** Backtracking technique can be a solution of the local maximum in state space landscape. Create a list of the promising path so that the algorithm can backtrack the search space and explore other paths as well.

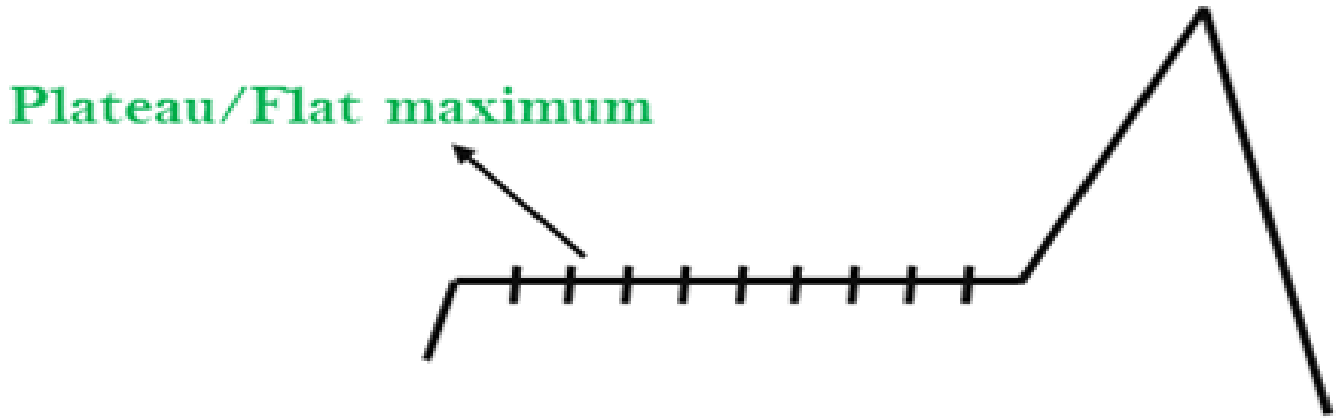


Problems in Hill Climbing Algorithm:

* **2. Plateau:** A plateau is the flat area of the search space in which all the neighbor states of the current state contains the same value, because of this algorithm does not find any best direction to move. A hill-climbing search might be lost in the plateau area.
* **Solution:** The solution for the plateau is to take big steps or very little steps while searching, to solve the problem. Randomly select a state which is far away from the current state so it is possible that the algorithm could find non-plateau region.

Problems in Hill Climbing Algorithm:

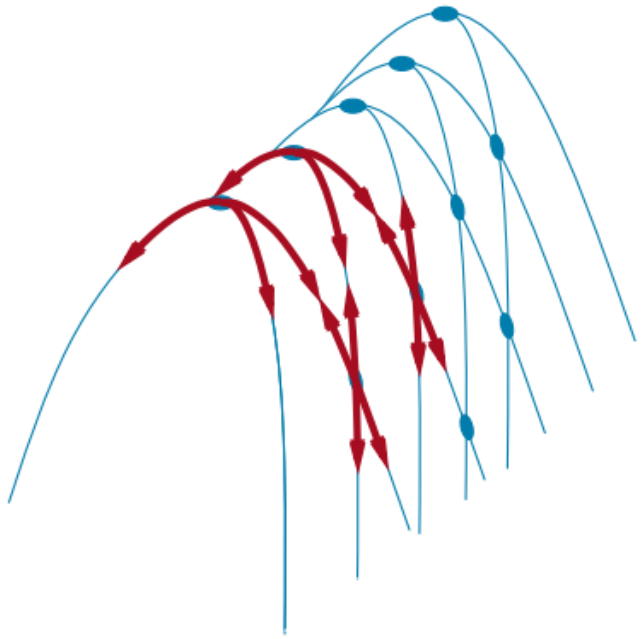
* **Solution:** The solution for the plateau is to take big steps or very little steps while searching, to solve the problem. Randomly select a state which is far away from the current state so it is possible that the algorithm could find non-plateau region.



Problems in Hill Climbing Algorithm:

* **Ridges:** A ridge is a special form of the local maximum. It has an area which is higher than its surrounding areas, but itself has a slope, and cannot be reached in a single move.
* **Solution:** With the use of bidirectional search, or by moving in different directions, we can improve this problem.





Thank You

CSA2001

FUNDAMENTALS IN AI AND ML

Module-2

Knowledge Representation

- **Logic** is the formal mathematical study of the methods, structure, and validity of mathematical deduction and proof.
- George Boole (born November 2, 1815, Lincoln, Lincolnshire, England—died December 8, 1864, Ballintemple, County Cork, Ireland) English mathematician who helped establish modern symbolic logic and whose algebra of logic, now called Boolean algebra, is basic to the design of digital computer circuits.
- In logic, a **predicate** is a symbol that represents a property or a relation. For instance, in the first-order formula , the symbol is a predicate that applies to the individual constant . Similarly, in the formula , the symbol is a predicate that applies to the individual constants and .

Semantics

Note: Truth value of a sentence is built from its parts "compositional semantics"

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
$false$	$false$	$true$	$false$	$false$	$true$	$true$
$false$	$true$	$true$	$false$	$true$	$true$	$false$
$true$	$false$	$false$	$false$	$true$	$false$	$false$
$true$	$true$	$false$	$true$	$true$	$true$	$true$

Mathematical Logic Formulas

- **Logical rules can be used to simplify logical formulas.**
- **Identity laws**

p∧T≡p

p∨F≡p

- **Domination laws**

p∧F≡F

p∨T≡T

- **Idempotent laws**

p∧p≡p

p∨p≡p

- **Absorption laws**
  p∧(p∨q)≡p
  p∨(p∧q)≡p

- **Negation laws**
  p∧¬p≡F
  p∨¬p≡T

Logical equivalences

$$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha) \quad \text{commutativity of } \wedge$$
$$(\alpha \vee \beta) \equiv (\beta \vee \alpha) \quad \text{commutativity of } \vee$$
$$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma)) \quad \text{associativity of } \wedge$$
$$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma)) \quad \text{associativity of } \vee$$
$$\neg(\neg\alpha) \equiv \alpha \quad \text{double-negation elimination}$$
$$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha) \quad \text{contraposition}$$
$$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta) \quad \text{implication elimination} \quad (*)$$
$$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) \quad \text{biconditional elimination}$$
$$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta) \quad \text{de Morgan}$$
$$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta) \quad \text{de Morgan}$$
$$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) \quad \text{distributivity of } \wedge \text{ over } \vee$$
$$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) \quad \text{distributivity of } \vee \text{ over } \wedge$$

First Order Predicate Logic

First Order Logic (FOL) can be simply put as a collection of objects, their attributes and relations among them to represent knowledge. It is also known as Predicate Logic.

First-order logic is another way of knowledge representation in artificial intelligence. It is an extension to propositional logic. FOL is sufficiently expressive to represent the natural language statements in a concise way. First-order logic is also known as Predicate logic or First-order predicate logic.

What is First Order Logic ?

- **FOL** is a mode of representation in AI. It is an extension of PL.
- FOL represents natural language statements in a concise way.
- FOL is also called *predicate logic*. It is a powerful language used to develop information about an object and express the relationship between objects.
- FOL not only assumes that does the world contains facts (like PL does), but it also assumes the following:
- —
- **Objects:** A, B, people, numbers, colors, wars, theories, squares, pit, etc.

First Order Predicate Logic

Relations: It is unary relation such as red, round, sister of, brother of, etc.

Function: father of, best friend, third inning of, end of, etc.

Representing Simple Statements in FOL:

It is important that you know the logical operators/connectives that are used in Propositional Logic.

Name of the connector	Symbol
AND	$\wedge$
OR	$\vee$
Negation	$\neg$
Exclusive OR	$\oplus$
Implication	$\rightarrow$
Double Implication	$\leftrightarrow$

Part of First Order Logic

FOL also has two parts:

1.Syntax

2.Semantics

—

Syntax: The **syntax** of FOL decides which collection of symbols is a logical expression. The basic syntactic elements of FOL are symbols. We use symbols to write statements in shorthand notation.

Basic Elements of FOL:

Name	Symbol
Constant	1, 6, A,W,New York, Elie, Dog...
Variables	a, b, c, x, y, z...
Predicates	<, >, brother, sister, father...
Equality	==
Function	Sqrt, LessThan, Sin(θ)...
Quantifier	∀, ∃
Connectives	∧, ∨, ¬, ⇒, ⇔

Atomic and Complex Sentences in FOL:

Atomic Sentence:

This is a basic sentence of FOL formed from a predicate symbol followed by a parenthesis with a sequence of terms.

We can represent atomic sentences as a predicate (value1, value2…., value n).

Example-

John and Michael are colleaguesà colleagues (John, Michael)

German Shepherd is a dogà Dog (German Shepherd)

Atomic and Complex Sentences in FOL:

Complex Sentence:

Complex sentences are made by combining atomic sentences using connectives.

FOL is further divided into two parts:

Subject: the main part of the statement.

Predicate: defined as a relation that binds two atoms together.

Example-

1.Colleague (Oliver, Benjamin) Colleague (Benjamin, Oliver)

2.“x is an integer”

Atomic and Complex Sentences in FOL:

It has two parts;

First, x is the subject.

Second, "is an integer" is called a predicate.



Quantifiers and their use in FOL:

Quantifiers generate quantification and specify the number of specimen in the universe.

Quantifiers allow us to determine or identify the range and scope of the variable in a logical expression.

There are two types of quantifiers:

1. **Universal quantifier**: for all, everyone, everything.
2. **Existential quantifier**: for some, at least one.

Universal Quantifiers:

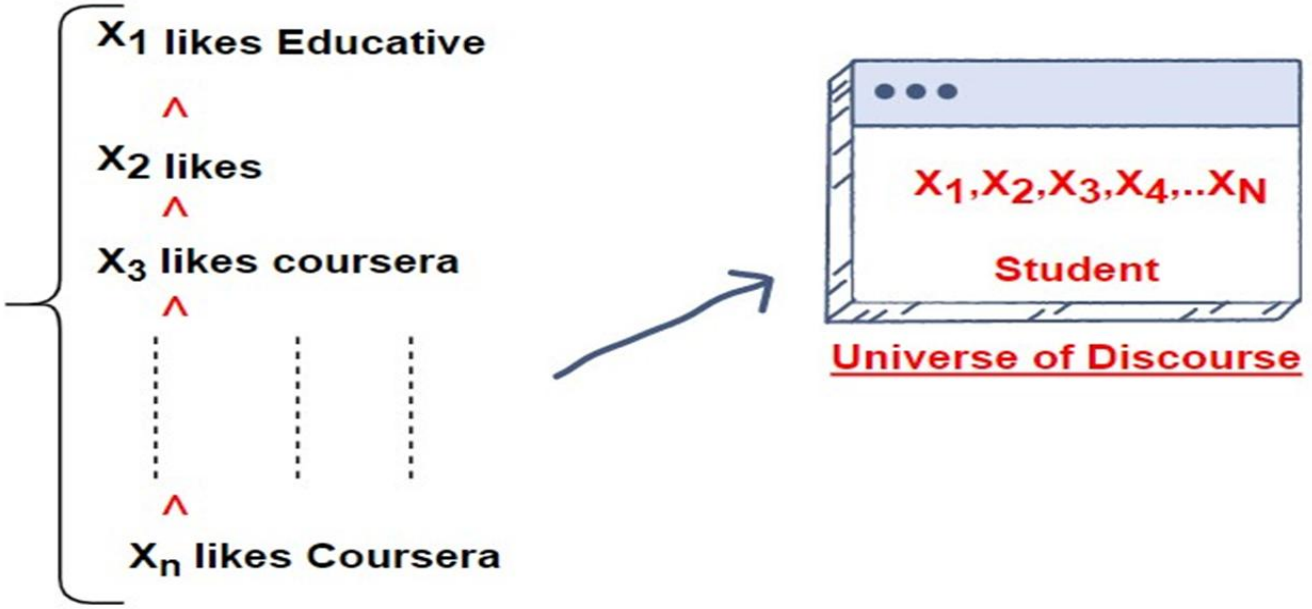
Universal quantifiers specify that the statement within the range is true for everything or every instance of a particular thing.

Universal quantifiers are denoted by a symbol that looks like an inverted A.

In a universal quantifier, we use

- If x is a variable then,
    1. For all x
    2. For every x
    3. For each x

Example- Every student likes educative.



Explanation:

So, in logical notation, it can be written as:

$$\forall x \; \text{student}(x) \rightarrow \text{likes}(x, \text{Educative})$$

This can be interpreted as: *There is every x where x is a student who likes Educative.*

Existential Quantifiers:

Existential quantifiers are used to express that the statement within their scope is true for at least one instance of something.

which looks like an inverted E, is used to represent them. We always use **AND** or conjunction symbols.

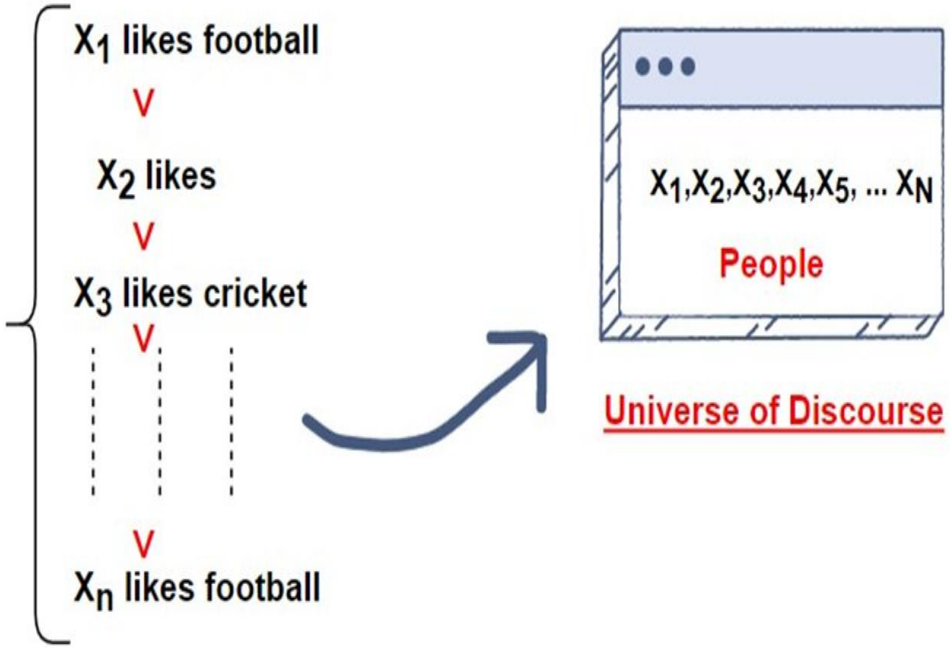
Existential Quantifiers:

If x is a variable, the existential quantifier will be

1. For some x
2. There exists an x
3. For at least one x

Example-

Some people like Football.



Existential Quantifiers:

Explanation:

So, in logical notation, it can be written as:

$$\exists x: \text{people}(x) \wedge \text{likes Football}(x)$$

It can be interpreted as: *There are some x where x is people who like football.*

Thank you!

Knowledge representation
First-order predicate logic

Dr. Rudra Kalyan Nayak
Senior Assistant Professor
School of Computing Science & Engineering



VIT®
BHOPAL

What is knowledge-engineering?

- The process of constructing a knowledge-base in first-order logic is called as knowledge-engineering.
- In **knowledge-engineering**, someone who investigates a particular domain, learns important concept of that domain, and generates a formal representation of the objects, is known as **knowledge engineer**.

What is knowledge acquisition?

- Often, the knowledge engineer is trained in representation but is not an expert in the domain at hand, be it circuit design, space station mission scheduling, or whatever.
- The knowledge engineer will usually interview the real experts to become educated about the domain and to elicit the required knowledge, in a process called knowledge acquisition.

The knowledge-engineering process:

- 1. Identify the task
- 2. Assemble the relevant knowledge
- 3. Decide on vocabulary
- 4. Encode general knowledge about the domain
- 5. Encode a description of the problem instance
- 6. Pose queries to the inference procedure and get answers
- 7 Debug the knowledge base

KNOWLEDGE ENGINEERING vs. PROGRAMMING

- A useful analogy can be made between knowledge engineering and programming. Both activities can be seen as consisting of four steps:

	Knowledge Engineering	*Programming*
(1)	Choosing a logic	Choosing a programming language
(2)	Building a knowledge base	Writing a program
(3)	Implementing the proof theory	Choosing or writing a compiler
(4)	Inferring new facts	Running a program

KNOWLEDGE ENGINEERING vs. PROGRAMMING

- In both activities, one writes down a description of a problem or state of affairs, and then uses the definition of the language to derive new consequences.
- In the case of a program, the output is derived from the input and the program; in the case of a knowledge base, answers are derived from descriptions of problems and the knowledge base.

KNOWLEDGE ENGINEERING vs. PROGRAMMING

- The main advantage of knowledge engineering is that it requires less commitment, and thus less work.
- A knowledge engineer only has to decide what objects and relations are worth representing, and which relations hold among which objects.
- A programmer has to do all that, and in addition must decide how to compute the relations between objects, given some initial input.

KNOWLEDGE ENGINEERING vs. PROGRAMMING

- The knowledge engineer specifies what is true, and the inference procedure figures out how to turn the facts into a solution to the problem.
- Furthermore, because a fact is true regardless of what task one is trying to solve, knowledge bases can, in principle, be reused for a variety of different tasks without modification.

KNOWLEDGE ENGINEERING vs. PROGRAMMING

- Finally, debugging a knowledge base is made easier by the fact that any given sentence is true or false by itself, whereas the correctness of a program statement depends very strongly on its context

Inference in First-Order Logic

- Inference in First-Order Logic is used to deduce new facts or sentences from existing sentences.
- Before understanding the FOL inference rule, let's understand some basic terminologies used in FOL.

Inference in First-Order Logic

- **Substitution:**
  - Substitution is a fundamental operation performed on terms and formulas. It occurs in all inference systems in first-order logic. The substitution is complex in the presence of quantifiers in FOL. If we write **F[a/x]**, so it refers to substitute a constant "**a**" in place of variable "**x**".
- Note: First-order logic is capable of expressing facts about some or all objects in the universe.

Inference in First-Order Logic

- **Equality:**
  - First-Order logic does not only use predicate and terms for making atomic sentences but also uses another way, which is equality in FOL. For this, we can use **equality symbols** which specify that the two terms refer to the same object.
  - **Example: Brother (John) = Smith.**
  - As in the above example, the object referred by the **Brother (John)** is similar to the object referred by **Smith**. The equality symbol can also be used with negation to represent that two terms are not the same objects.
  - **Example: ¬(x=y) which is equivalent to x ≠y.**

FOL inference rules for quantifier:

- As propositional logic we also have inference rules in first-order logic, so following are *some basic inference rules in FOL*:
- **Universal Generalization**
- **Universal Instantiation**
- **Existential Instantiation**
- **Existential introduction**

FOL inference rules for quantifier:

- **1. Universal Generalization:**
  - Universal generalization is a valid inference rule which states that if premise P(c) is true for any arbitrary element c in the universe of discourse, then we can have a conclusion as ∀ x P(x).
  - It can be represented as: . $\frac{P(c)}{\forall x\, P(x)}$
  - This rule can be used if we want to show that every element has a similar property.
  - In this rule, x must not appear as a free variable.

FOL inference rules for quantifier:

- **1. Universal Generalization:**
  - **Example:** Let's represent, P(c): "**A byte contains 8 bits**", so for **∀ x P(x)** "**All bytes contain 8 bits**.", it will also be true.

FOL inference rules for quantifier:

- **2 Universal Instantiation::**
  - Universal instantiation is also called as universal elimination or UI is a valid inference rule. It can be applied multiple times to add new sentences.
  - The new KB is logically equivalent to the previous KB.
  - As per UI, **we can infer any sentence obtained by substituting a ground term for the variable**.
  - The UI rule state that we can infer any sentence P(c) by substituting a ground term c (a constant within domain x) from **∀ x P(x) for any object in the universe of discourse**.

FOL inference rules for quantifier:

- **2 Universal Instantiation::**
  - It can be represented as:. $\therefore \frac{\forall x\, P(x)}{P(c)}$.
  - **Example:1.**
  - IF "Every person like ice-cream"=> ∀x P(x) so we can infer that
    "John likes ice-cream" => P(c)
  - **Example: 2.**
  - Let's take a famous example,
  - "All kings who are greedy are Evil." So let our knowledge base contains this detail as in the form of FOL:

FOL inference rules for quantifier:

- **2 Universal Instantiation:**
  - "All kings who are greedy are Evil." So let our knowledge base contains this detail as in the form of FOL:
  - **∀x king(x) ∧ greedy (x) → Evil (x),**
  - So from this information, we can infer any of the following statements using Universal Instantiation:
  - **King(John) ∧ Greedy (John) → Evil (John),**
  - **King(Richard) ∧ Greedy (Richard) → Evil (Richard),**
  - **King(Father(John)) ∧ Greedy (Father(John)) → Evil (Father(John)),**

FOL inference rules for quantifier:

- **3 Existential Instantiation:**
  - Existential instantiation is also called as Existential Elimination, which is a valid inference rule in first-order logic.
  - It can be applied only once to replace the existential sentence.
  - The new KB is not logically equivalent to old KB, but it will be satisfiable if old KB was satisfiable.

FOL inference rules for quantifier:

- **3 Existential Instantiation:**
  - This rule states that one can infer P(c) from the formula given in the form of ∃x P(x) for a new constant symbol c.
  - The restriction with this rule is that c used in the rule must be a new term for which P(c) is true.
  - It can be represented as: $\frac{\exists x\ P(x)}{P(c)}$

FOL inference rules for quantifier:

- **3 Existential Instantiation- Example:**
- From the given sentence: **∃x Crown(x) ∧ OnHead(x, John),**
- So we can infer: **Crown(K) ∧ OnHead(K, John),** as long as K does not appear in the knowledge base.
  - The above used K is a constant symbol, which is called **Skolem constant**.
  - The Existential instantiation is a special case of **Skolemization process**.

FOL inference rules for quantifier:

- **4 Existential introduction:**
- An existential introduction is also known as an existential generalization, which is a valid inference rule in first-order logic.
- This rule states that if there is some element c in the universe of discourse which has a property P, then we can infer that there exists something in the universe which has the property P.
- It can be represented as: $\frac{P(c)}{\exists x P(x)}$

FOL inference rules for quantifier:

- **4 Existential introduction: Example: Let's say that,**
  - "Priyanka got good marks in English."
    "Therefore, someone got good marks in English."

What is Unification?

- *Unification is a process of making two different logical atomic expressions identical by finding a substitution. Unification depends on the substitution process.*
- *It takes two literals as input and makes them identical using substitution.*
- Let Ψ_1 and Ψ_2 be two atomic sentences and σ be a unifier such that, $\boldsymbol{\Psi_1\sigma = \Psi_2\sigma}$, then it can be expressed as **UNIFY(Ψ_1, Ψ_2)**.

What is Unification?

- Example: Find the MGU (Most General Unifier) for Unify{King(x), King(John)}
- Let Ψ1 = King(x), Ψ2 = King(John),
- Substitution θ = {John/x} is a unifier for these atoms and applying this substitution, and both expressions will be identical.
- The UNIFY algorithm is used for unification, which takes two atomic sentences and returns a unifier for those sentences (If any exist).
- Unification is a key component of all first-order inference algorithms.
- It returns fail if the expressions do not match with each other.
- The substitution variables are called Most General Unifier or MGU.

What is Unification?

- **E.g.** Let's say there are two different expressions, **P(x, y), and P(a, f(z))**.
- In this example, we need to make both above statements identical to each other. For this, we will perform the substitution.

  P(x, y)..... (i)
  P(a, f(z))..... (ii)

- Substitute x with a, and y with f(z) in the first expression, and it will be represented as **a/x** and f(z)/y.
- With both the substitutions, the first expression will be identical to the second expression and the substitution set will be: **[a/x, f(z)/y]**.

Conditions for Unification:

- **Following are some basic conditions for unification:**
- Predicate symbol must be same, atoms or expression with different predicate symbol can never be unified.
- Number of Arguments in both expressions must be identical.
- Unification will fail if there are two similar variables present in the same expression.

Conditions for Unification:

Unification Algorithm:

Algorithm: Unify(Ψ_1, Ψ_2)

Step. 1: If Ψ_1 or Ψ_2 is a variable or constant, then:

- a) If Ψ_1 or Ψ_2 are identical, then return NIL.
- b) Else if $\Psi_1$is a variable,
  - a. then if Ψ_1 occurs in Ψ_2, then return FAILURE
  - b. Else return { (Ψ_2/ Ψ_1)}.
- c) Else if Ψ_2 is a variable,
  - a. If Ψ_2 occurs in Ψ_1 then return FAILURE,
  - b. Else return {(Ψ_1/ Ψ_2)}.
- d) Else return FAILURE.

Conditions for Unification:

Unification Algorithm:

Algorithm: Unify(Ψ_1, Ψ_2)

Step.2: If the initial Predicate symbol in Ψ_1 and Ψ_2 are not same, then return FAILURE.

Step. 3: IF Ψ_1 and Ψ_2 have a different number of arguments, then return FAILURE.

Step. 4: Set Substitution set(SUBST) to NIL.

Step. 5: For i=1 to the number of elements in Ψ_1.

a) Call Unify function with the ith element of Ψ_1 and ith element of Ψ_2, and put the result into S.

b) If S = failure then returns Failure

c) If S ≠ NIL then do,

a. Apply S to the remainder of both L1 and L2.
b. b. SUBST= APPEND(S, SUBST).

Step.6: Return SUBST.

Implementation of the Algorithm

- **Step.1:** Initialize the substitution set to be empty.
- **Step.2:** Recursively unify atomic sentences:

a) Check for Identical expression match.

b) If one expression is a variable v_i, and the other is a term t_i which does not contain variable v_i, then:
   a) Substitute t_i / v_i in the existing substitutions
   b) Add t_i /v_i to the substitution setlist.
   c) If both the expressions are functions, then function name must be similar, and the number of arguments must be the same in both the expression.

- **For each pair of the following atomic sentences find the most general unifier (If exist).**

Example

Find the MGU of {p(b, X, f(g(Z))) and p(Z, f(Y), f(Y))}

- Here, Ψ_1 = p(b, X, f(g(Z))) , and Ψ_2 = p(Z, f(Y), f(Y))
  S_0 => { p(b, X, f(g(Z))); p(Z, f(Y), f(Y))}
  SUBST θ={b/Z}
- S_1 => { p(b, X, f(g(b))); p(b, f(Y), f(Y))}
  SUBST θ={f(Y) /X}
- S_2 => { p(b, f(Y), f(g(b))); p(b, f(Y), f(Y))}
  SUBST θ= {g(b) /Y}
- S_2 => { p(b, f(g(b)), f(g(b)); p(b, f(g(b)), f(g(b))} **Unified Successfully.**

Example

Find the MGU of {p (X, X), and p (Z, f(Z))}

- Here, Ψ_1 = {p (X, X), and Ψ_2 = p (Z, f(Z))
  S_0 => {p (X, X), p (Z, f(Z))}
  SUBST θ= {X/Z}
  S1 => {p (Z, Z), p (Z, f(Z))}
  SUBST θ= {f(Z) / Z}, **Unification Failed**.

Example

Find the MGU of UNIFY(prime (11), prime(y))

- Here, Ψ_1 = {prime(11) , and Ψ_2 = prime(y)}
  S_0 => {prime(11) , prime(y)}
  SUBST θ= {11/y}
- S_1 => {prime(11) , prime(11)} , **Successfully unified.**

  Unifier: {11/y}.

Example

Find the MGU of Q(a, g(x, a), f(y)), Q(a, g(f(b), a), x)}

- Here, Ψ_1 = Q(a, g(x, a), f(y)), and Ψ_2 = Q(a, g(f(b), a), x)
  S_0 => {Q(a, g(x, a), f(y)); Q(a, g(f(b), a), x)}
  SUBST θ= {f(b)/x}
  S_1 => {Q(a, g(f(b), a), f(y)); Q(a, g(f(b), a), f(b))}
- SUBST θ= {b/y}
  S_1 => {Q(a, g(f(b), a), f(b)); Q(a, g(f(b), a), f(b))}, **Successfully Unified.**
- **Unifier: [a/a, f(b)/x, b/y].**

Resolution in FOL

- *Resolution is a theorem proving technique that proceeds by building refutation proofs, i.e., proofs by contradictions. It was invented by a Mathematician John Alan Robinson in the year 1965.*
- Resolution is used, if there are various statements are given, and we need to prove a conclusion of those statements. Unification is a key concept in proofs by resolutions. Resolution is a single inference rule which can efficiently operate on the **conjunctive normal form or clausal form**.
- **Clause**: Disjunction of literals (an atomic sentence) is called a **clause**. It is also known as a unit clause.
- **Conjunctive Normal Form**: A sentence represented as a conjunction of clauses is said to be **conjunctive normal form** or **CNF**.

Steps for Resolution:

- Conversion of facts into first-order logic.
- Convert FOL statements into CNF
- Negate the statement which needs to prove (proof by contradiction)
- Draw resolution graph (unification).

Example:

- John likes all kind of food.
- Apple and vegetable are food
- Anything anyone eats and not killed is food.
- Anil eats peanuts and still alive
- Harry eats everything that Anil eats.
  Prove by resolution that:
- John likes peanuts.

Example:

- **Step-1: Conversion of Facts into FOL**
- In the first step we will convert all the given statements into its first order logic.

a. $\forall x$: food(x) $\rightarrow$ likes(John, x)

b. food(Apple) $\wedge$ food(vegetables)

c. $\forall x \forall y$: eats(x, y) $\wedge \neg$ killed(x) $\rightarrow$ food(y)

d. eats (Anil, Peanuts) $\wedge$ alive(Anil).

e. $\forall x$: eats(Anil, x) $\rightarrow$ eats(Harry, x)

f. $\forall x$: $\neg$ killed(x) $\rightarrow$ alive(x)

g. $\forall x$: alive(x) $\rightarrow \neg$ killed(x)

} **added predicates.**

h. likes(John, Peanuts)

Example:

Step-2: Conversion of FOL into CNF

- In First order logic resolution, it is required to convert the FOL into CNF as CNF form makes easier for resolution proofs.
- **Eliminate all implication (→) and rewrite**
  a. ∀x ¬ food(x) V likes(John, x)
  b. food(Apple) Λ food(vegetables)
  c. ∀x ∀y ¬ [eats(x, y) Λ ¬ killed(x)] V food(y)
  d. eats (Anil, Peanuts) Λ alive(Anil)
  e. ∀x ¬ eats(Anil, x) V eats(Harry, x)
  f. ∀x¬ [¬ killed(x)] V alive(x)
  g. ∀x ¬ alive(x) V ¬ killed(x)
  h. likes(John, Peanuts).

Example:

- **Move negation (¬)inwards and rewrite**
  a. ∀x ¬ food(x) V likes(John, x)
  b. food(Apple) Λ food(vegetables)
  c. ∀x ∀y ¬ eats(x, y) V killed(x) V food(y)
  d. eats (Anil, Peanuts) Λ alive(Anil)
  e. ∀x ¬ eats(Anil, x) V eats(Harry, x)
  f. ∀x ¬killed(x)] V alive(x)
  g. ∀x ¬ alive(x) V ¬ killed(x)
  h. likes(John, Peanuts).

Example:

- **Rename variables or standardize variables**
  a. ∀x ¬ food(x) V likes(John, x)
  b. food(Apple) Λ food(vegetables)
  c. ∀y ∀z ¬ eats(y, z) V killed(y) V food(z)
  d. eats (Anil, Peanuts) Λ alive(Anil)
  e. ∀w¬ eats(Anil, w) V eats(Harry, w)
  f. ∀g ¬killed(g)] V alive(g)
  g. ∀k ¬ alive(k) V ¬ killed(k)
  h. likes(John, Peanuts).

Example:

Eliminate existential instantiation quantifier by elimination.

In this step, we will eliminate existential quantifier ∃, and this process is known as **Skolemization**. But in this example problem since there is no existential quantifier so all the statements will remain same in this step.

Skolemization –

Example 1

Every philosopher writes at least one book.

$\forall x[Philo(x) \rightarrow \exists y[Book(y) \wedge Write(x, y)]]$

Eliminate Implication:

$\forall x[\neg Philo(x) \vee \exists y[Book(y) \wedge Write(x, y)]]$

Eliminate Implication:

Skolemize:

substitute y by g(x) $\forall x[\neg Philo(x) \vee [Book(g(x)) \wedge Write(x, g(x))]]$

Example:

- **Drop Universal quantifiers.**
  In this step we will drop all universal quantifier since all the statements are not implicitly quantified so we don't need it.
  - ¬ food(x) V likes(John, x)
  - food(Apple)
  - food(vegetables)
  - ¬ eats(y, z) V killed(y) V food(z)
  - eats (Anil, Peanuts)
  - alive(Anil)
  - ¬ eats(Anil, w) V eats(Harry, w)
  - killed(g) V alive(g)
  - ¬ alive(k) V ¬ killed(k)
  - likes(John, Peanuts).

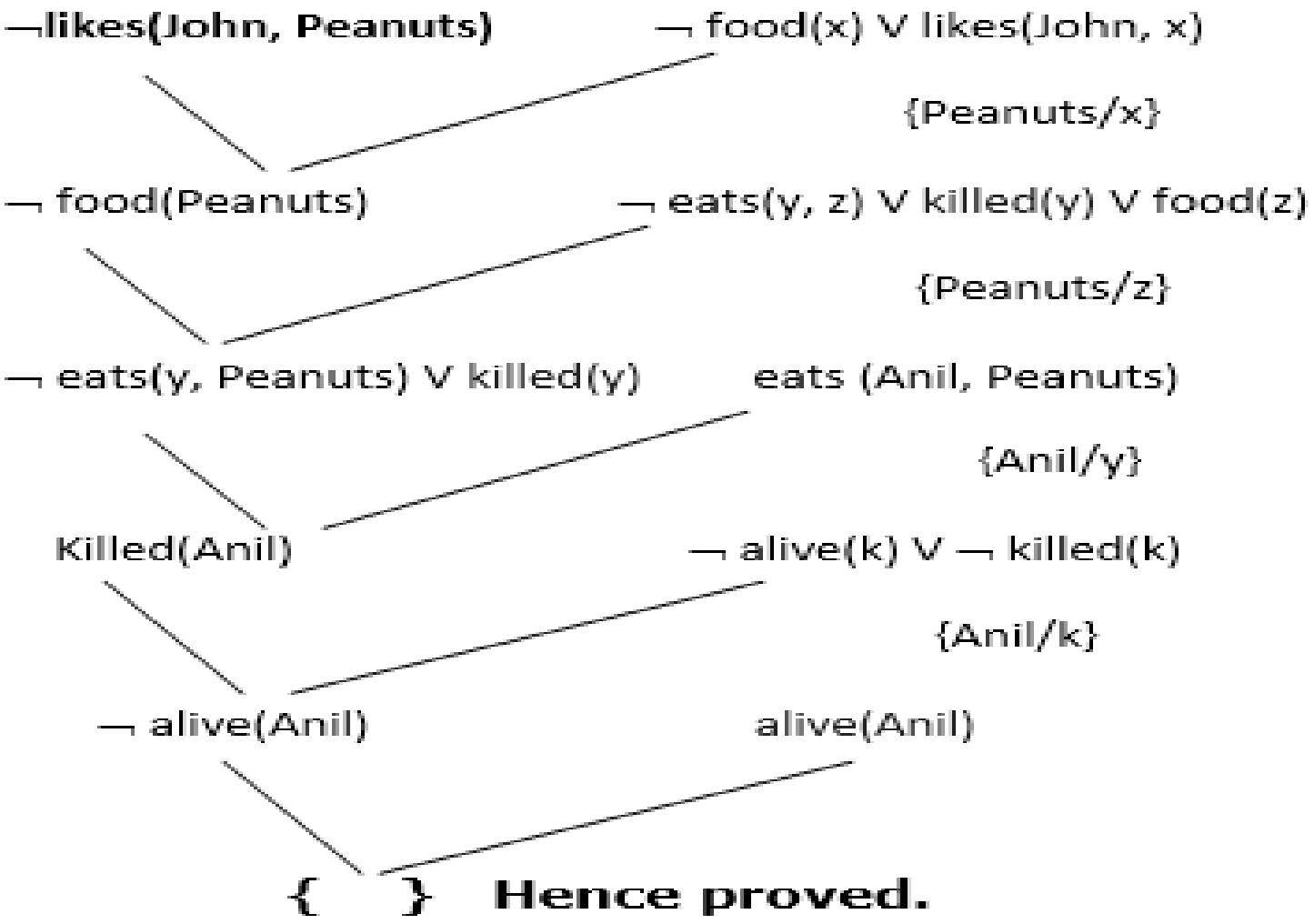
Example:

- **Distribute conjunction ∧ over disjunction ¬.**
  This step will not make any change in this problem.

- **Step-3: Negate the statement to be proved**
  - In this statement, we will apply negation to the conclusion statements, which will be written as ¬likes(John, Peanuts)

- **Step-4: Draw Resolution graph:** Now in this step, we will solve the problem by resolution tree using substitution. For the above problem, it will be given as follows:

Example:


¬likes(John, Peanuts)
¬ food(x) V likes(John, x)
{Peanuts/x}
¬ food(Peanuts)
¬ eats(y, z) V killed(y) V food(z)
{Peanuts/z}
¬ eats(y, Peanuts) V killed(y)
eats (Anil, Peanuts)
{Anil/y}
Killed(Anil)
¬ alive(k) V ¬ killed(k)
{Anil/k}
¬ alive(Anil)
alive(Anil)
{ } Hence proved.

Example:

- Explanation of Resolution graph:
- In the first step of resolution graph, **¬likes(John, Peanuts)** , and **likes(John, x)** get resolved(canceled) by substitution of **{Peanuts/x}**, and we are left with **¬ food(Peanuts)**
- In the second step of the resolution graph, **¬ food(Peanuts)** , and **food(z)** get resolved (canceled) by substitution of **{ Peanuts/z}**, and we are left with **¬ eats(y, Peanuts) V killed(y)** .
- In the third step of the resolution graph, **¬ eats(y, Peanuts)** and **eats (Anil, Peanuts)** get resolved by substitution **{Anil/y}**, and we are left with **Killed(Anil)** .
- In the fourth step of the resolution graph, **Killed(Anil)** and **¬ killed(k)** get resolve by substitution **{Anil/k}**, and we are left with **¬ alive(Anil)** .
- In the last step of the resolution graph **¬ alive(Anil)** and **alive(Anil)** get resolved.

Practice Questions on Resolution

Practice Question-1

- All hounds howl at night.
- Anyone who has any cats will not have any mice.
- Light sleepers do not have anything which howls at night.
- John has either a cat or a hound.

Prove by resolution that

- (Conclusion) If John is a light sleeper, then John does not have any mice.

Practice Question-2

- Everyone who feels warm either is drunk, or every costume they have is warm.
- Every costume that is warm is furry.
- Every AI student is a CS student.
- Every AI student has some robot costume.
- No robot costume is furry.

Prove by resolution that

- (Conclusion) If every CS student feels warm, then every AI student is drunk.

Forward Chaining and backward chaining in AI

- In artificial intelligence, forward and backward chaining is one of the important topics,
- lets first understand that from where these two terms came.

Inference engine:

- The inference engine is the component of the intelligent system in artificial intelligence, which applies logical rules to the knowledge base to infer new information from known facts. The first inference engine was part of the expert system. Inference engine commonly proceeds in two modes, which are:
- **Forward chaining**
- **Backward chaining**

Forward Chaining and backward chaining in AI

- **Horn Clause and Definite clause:**
- Horn clause and definite clause are the forms of sentences, which enables knowledge base to use a more restricted and efficient inference algorithm. Logical inference algorithms use forward and backward chaining approaches, which require KB in the form of the **first-order definite clause**.
- **Definite clause:** A clause which is a disjunction of literals with **exactly one positive literal** is known as a definite clause or strict horn clause.
- **Horn clause:** A clause which is a disjunction of literals with **at most one positive literal** is known as horn clause. Hence all the definite clauses are horn clauses.
- **Example: (¬ p V ¬ q V k)**. It has only one positive literal k.
- It is equivalent to $p \land q \rightarrow k$.

Forward Chaining and backward chaining in AI

A. Forward Chaining

- Forward chaining is also known as a forward deduction or forward reasoning method when using an inference engine.
- Forward chaining is a form of reasoning which start with atomic sentences in the knowledge base and applies inference rules (Modus Ponens) in the forward direction to extract more data until a goal is reached.
- The Forward-chaining algorithm starts from known facts, triggers all rules whose premises are satisfied, and add their conclusion to the known facts. This process repeats until the problem is solved.

Forward Chaining and backward chaining in AI

A. Forward Chaining

- **Properties of Forward-Chaining:**
- It is a down-up approach, as it moves from bottom to top.
- It is a process of making a conclusion based on known facts or data, by starting from the initial state and reaches the goal state.
- Forward-chaining approach is also called as data-driven as we reach to the goal using available data.
- Forward -chaining approach is commonly used in the expert system, such as CLIPS, business, and production rule systems.

Forward Chaining and backward chaining in AI

A. Forward Chaining

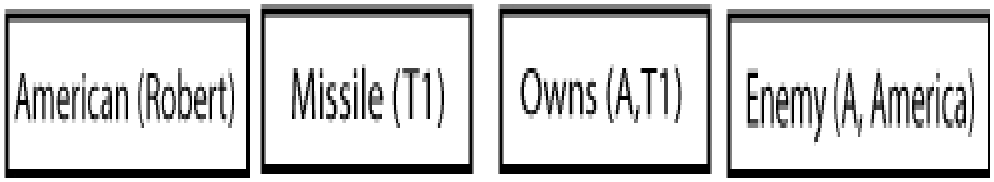
- **Example:**
- **"As per the law, it is a crime for an American to sell weapons to hostile nations. Country A, an enemy of America, has some missiles, and all the missiles were sold to it by Robert, who is an American citizen."**
- **Prove that "Robert is criminal."**
- To solve the above problem, first, we will convert all the above facts into first-order definite clauses, and then we will use a forward-chaining algorithm to reach the goal.

Facts Conversion into FOL:

- Facts Conversion into FOL:
- It is a crime for an American to sell weapons to hostile nations. (Let's say p, q, and r are variables)
  American (p) ∧ weapon(q) ∧ sells (p, q, r) ∧ hostile(r) → Criminal(p) ...(1)
- Country A has some missiles. **?p Owns(A, p) ∧ Missile(p)**. It can be written in two definite clauses by using Existential Instantiation, introducing new Constant T1.
  Owns(A, T1) .....(2)
  Missile(T1) .....(3)
- All of the missiles were sold to country A by Robert.
  U: p Missiles(p) ∧ Owns (A, p) → Sells (Robert, p, A) .....(4)
- Missiles are weapons.
  Missile(p) → Weapons (p) .....(5)
- Enemy of America is known as hostile.
  Enemy(p, America) →Hostile(p) .....(6)
- Country A is an enemy of America.
  Enemy (A, America) .....(7)
- Robert is American
  American(Robert). .....(8)

Forward chaining proof:

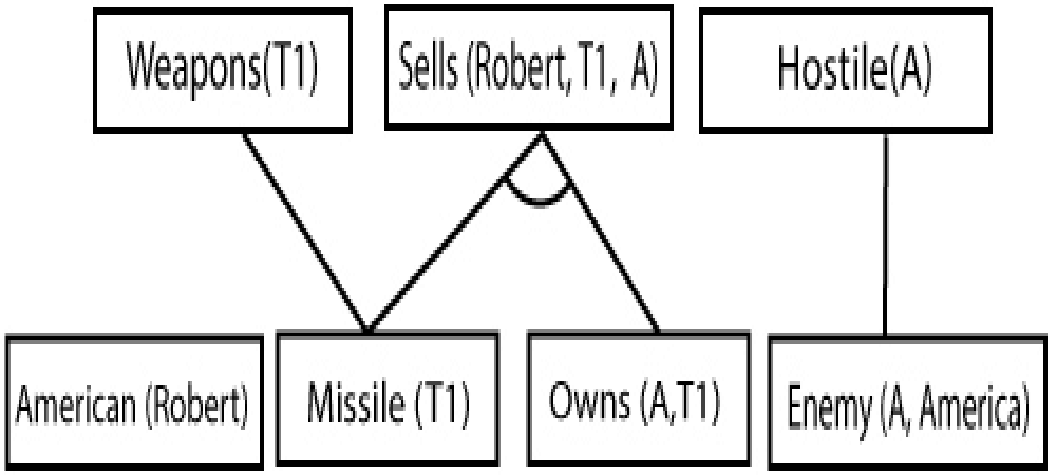
- **Step-1:**
- In the first step we will start with the known facts and will choose the sentences which do not have implications, such as: **American(Robert), Enemy(A, America), Owns(A, T1), and Missile(T1)**. All these facts will be represented as below.



- **Step-2**
- At the second step, we will see those facts which infer from available facts and with satisfied premises.
- Rule-(1) does not satisfy premises, so it will not be added in the first iteration.
- Rule-(2) and (3) are already added.

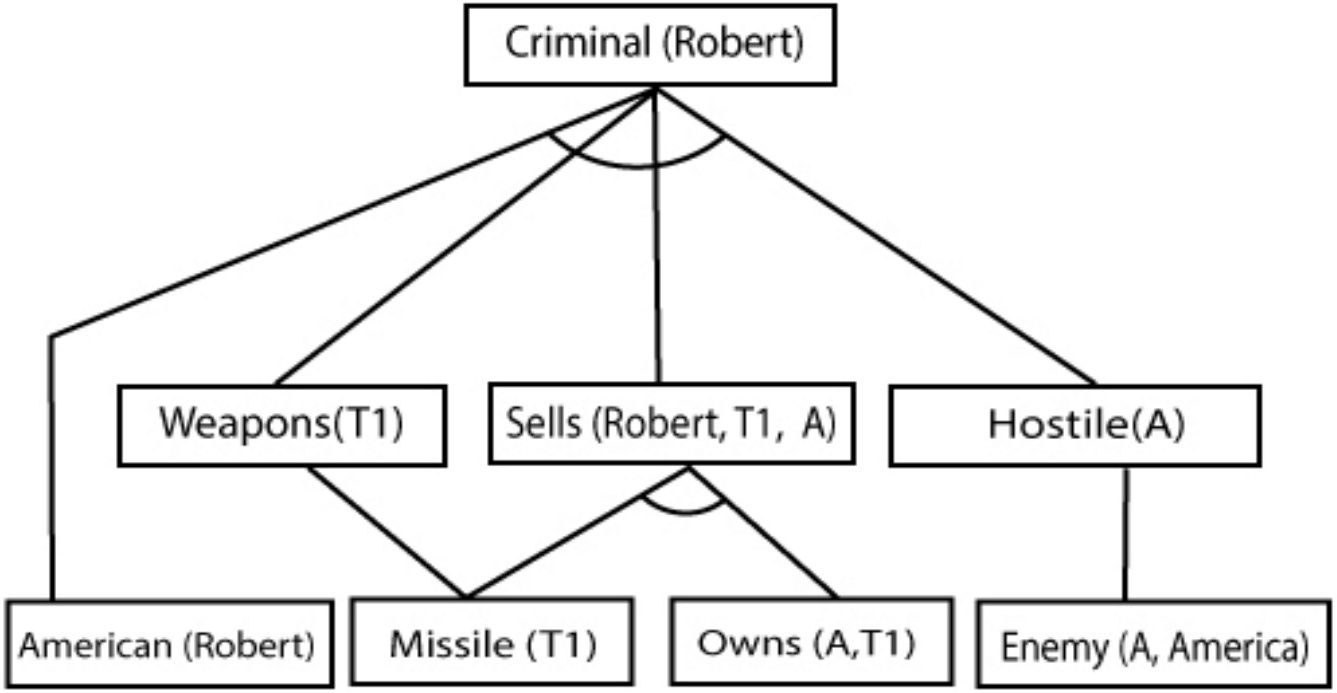
Forward chaining proof:

- **Step-1:**
- Rule-(4) satisfy with the substitution {p/T1}, **so Sells (Robert, T1, A)** is added, which infers from the conjunction of Rule (2) and (3).
- Rule-(6) is satisfied with the substitution(p/A), so Hostile(A) is added and which infers from Rule-(7).



Forward chaining proof:

- **Step-3:**
- At step-3, as we can check Rule-(1) is satisfied with the substitution **{p/Robert, q/T1, r/A}, so we can add Criminal(Robert)** which infers all the available facts. And hence we reached our goal statement.



- **Hence it is proved that Robert is Criminal using forward chaining approach.**

B. Backward Chaining:

- Backward-chaining is also known as a backward deduction or backward reasoning method when using an inference engine.
- A backward chaining algorithm is a form of reasoning, which starts with the goal and works backward, chaining through rules to find known facts that support the goal.
- **Properties of backward chaining:**
- It is known as a top-down approach.
- Backward-chaining is based on modus ponens inference rule.
- In backward chaining, the goal is broken into sub-goal or sub-goals to prove the facts true.

B. Backward Chaining:

- It is called a goal-driven approach, as a list of goals decides which rules are selected and used.
- Backward -chaining algorithm is used in game theory, automated theorem proving tools, inference engines, proof assistants, and various AI applications.
- The backward-chaining method mostly used a **depth-first search** strategy for proof.

B. Backward Chaining:

Example:

- In backward-chaining, we will use the same above example, and will rewrite all the rules.
- **American (p) ∧ weapon(q) ∧ sells (p, q, r) ∧ hostile(r) → Criminal(p) ...(1)**
- **Owns(A, T1) .....(2)**
- **Missile(T1)**
- **?p Missiles(p) ∧ Owns (A, p) → Sells (Robert, p, A) .....(4)**
- **Missile(p) → Weapons (p) .....(5)**
- **Enemy(p, America) →Hostile(p) .....(6)**
- **Enemy (A, America) .....(7)**
- **American(Robert). .....(8)**

B. Backward Chaining:

Backward-Chaining proof:

- In Backward chaining, we will start with our goal predicate, which is **Criminal(Robert)**, and then infer further rules.

Step-1:

- At the first step, we will take the goal fact. And from the goal fact, we will infer other facts, and at last, we will prove those facts true. So our goal fact is "Robert is Criminal," so following is the predicate of it.

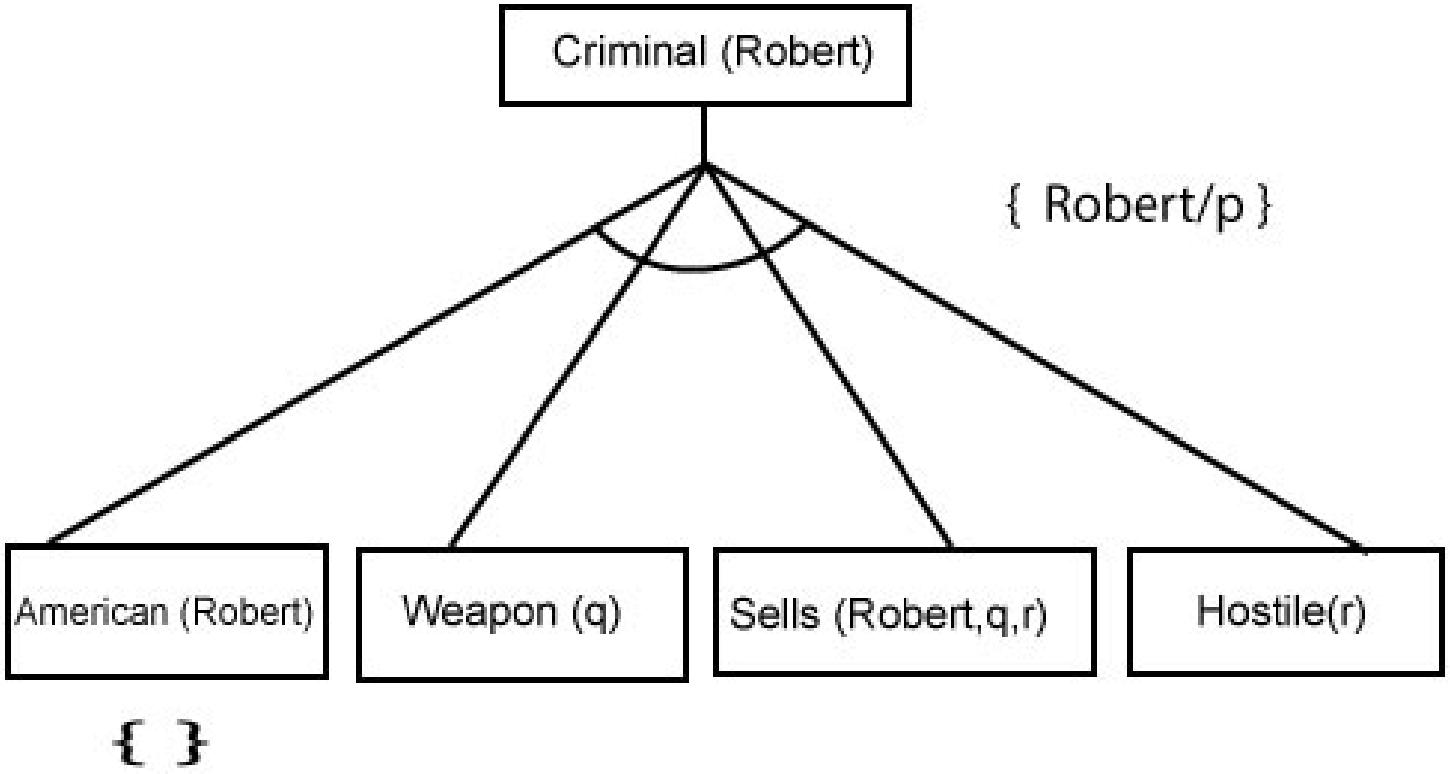


B. Backward Chaining:

Step-2:

- At the second step, we will infer other facts form goal fact which satisfies the rules. So as we can see in Rule-1, the goal predicate Criminal (Robert) is present with substitution {Robert/P}. So we will add all the conjunctive facts below the first level and will replace p with Robert.
- **Here we can see American (Robert) is a fact, so it is proved here.**

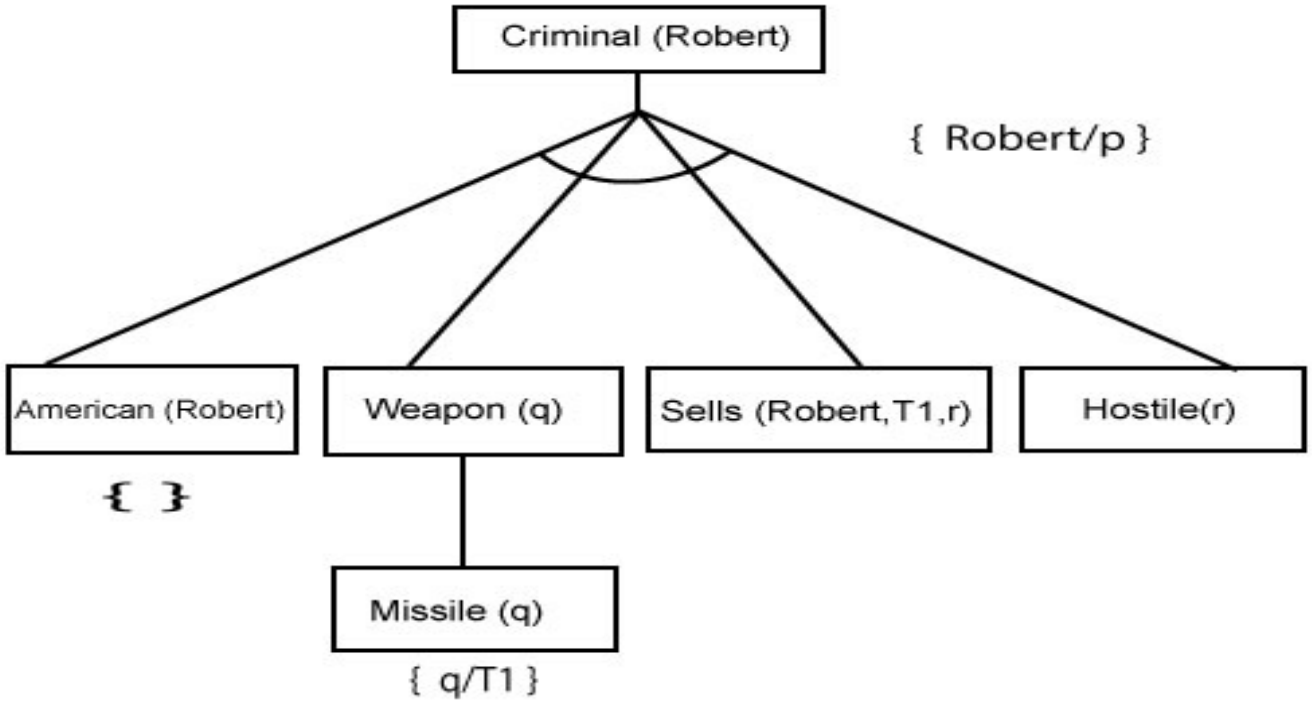
B. Backward Chaining:



B. Backward Chaining:

Step-3:

- At step-3, we will extract further fact Missile(q) which infer from Weapon(q), as it satisfies Rule-(5). Weapon (q) is also true with the substitution of a constant T1 at q.

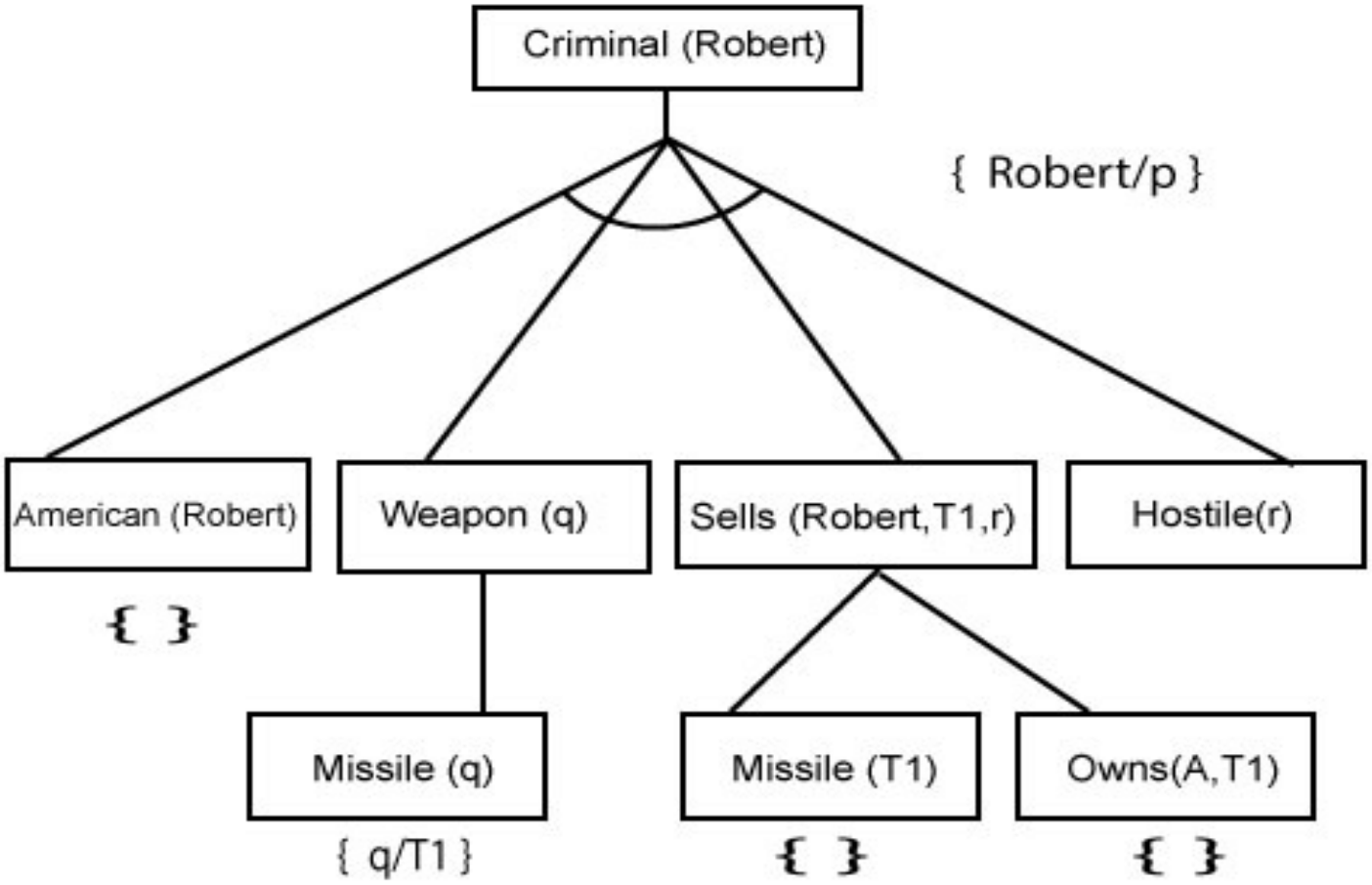


B. Backward Chaining:

Step-4:

- At step-4, we can infer facts Missile(T1) and Owns(A, T1) form Sells(Robert, T1, r) which satisfies the **Rule- 4**, with the substitution of A in place of r.
- So these two statements are proved here.

B. Backward Chaining:

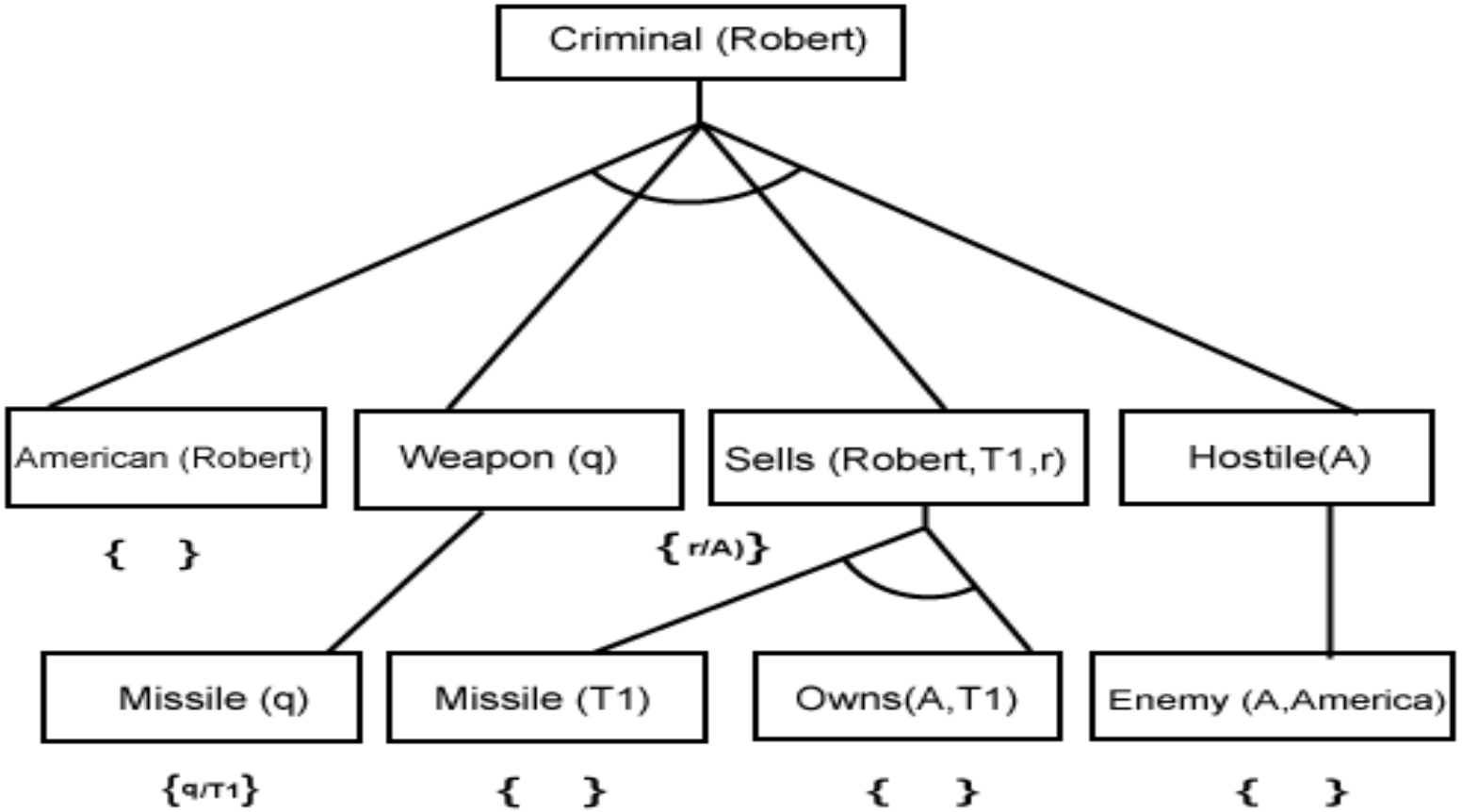


B. Backward Chaining:

Step-5:

- At step-5, we can infer the fact **Enemy(A, America)** from **Hostile(A)** which satisfies Rule- 6.
- And hence all the statements are proved true using backward chaining.

B. Backward Chaining:





(CSA2001)

Informed (Heuristic) Search

Dr. Rudra Kalyan Nayak
Senior Assistant Professor
SCAI

VIT BHOPAL

Informed (Heuristic) Search

- ✓ Need to add some kind of guidance to reach from start state to goal state. This guidance that we are going to add in search algorithms is called as Informed Search.

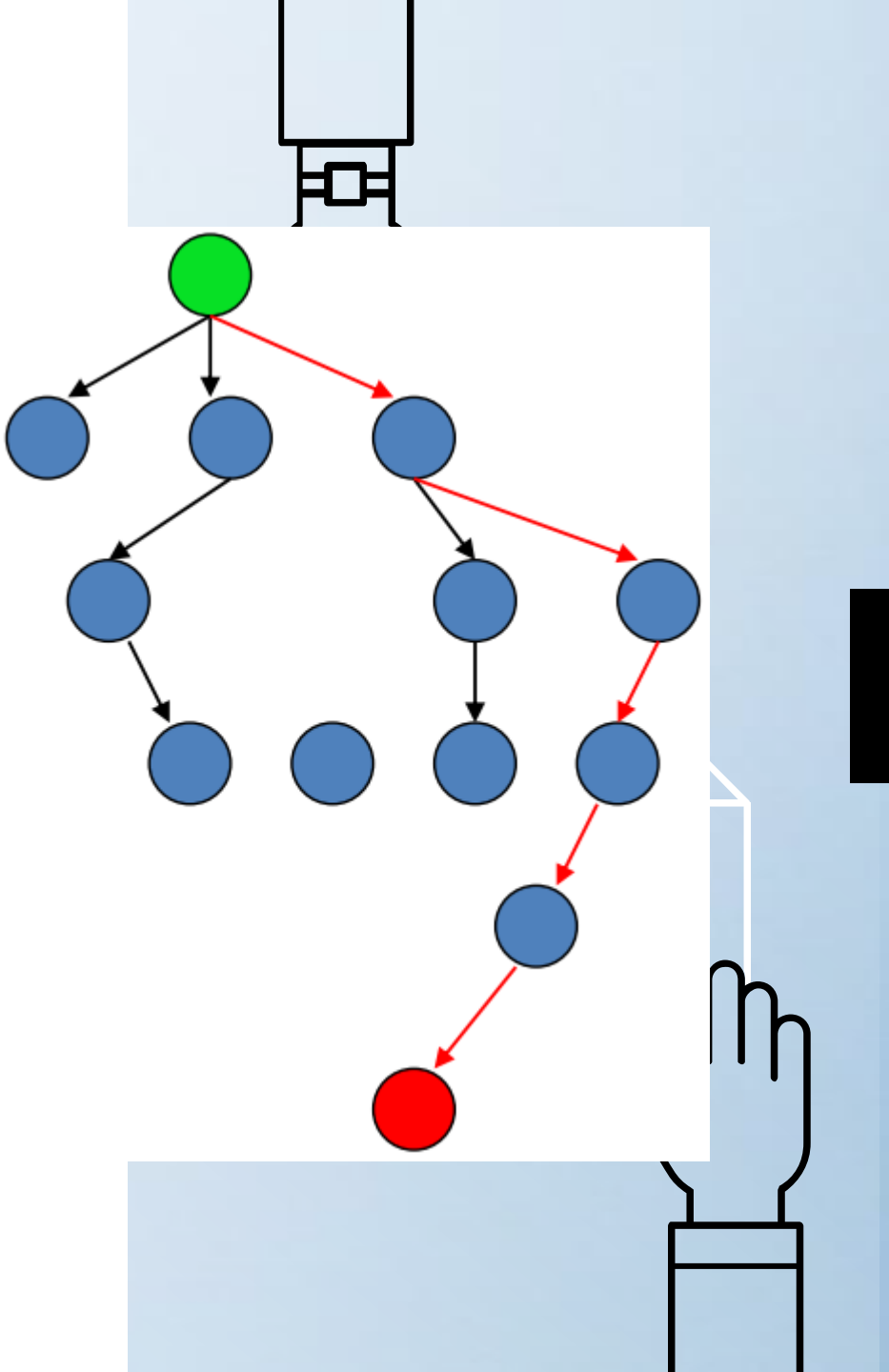
Some kind of intuition.

"Intuition, like the rays of the sun, acts only in an inflexibly straight line; it can guess right only on condition of never diverting its gaze; the freaks of chance disturb it."

-- Honore de Balzac

Informed (Heuristic) Search

Idea: be **smart**
about what paths
to try



- Evaluation function $f(n) = g(n) + h(n)$

✓ $g(n)$ = cost so far to reach n
✓ $h(n)$ = estimated cost from n to goal
✓ $f(n)$ = estimated total cost of path through n to goal

Heuristic ?

Uninformed Search Complexity: NP hard Problem

- A node is selected for expansion based on an evaluation function that estimates cost to goal.

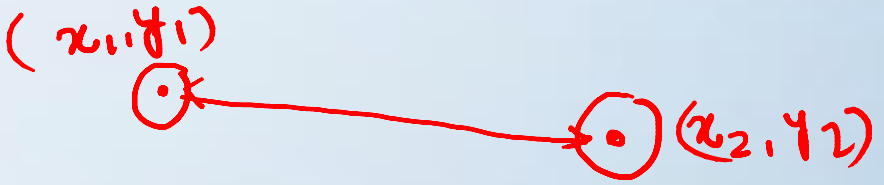
- Rule of thumb
- The function is called as heuristic function.
- This will help to guide the search algorithm towards goal state without exploring all the nodes (uninformed search).
- Ex: For 8 puzzle problem uninformed search: $O(b^d)$ Time Complexity = 3^{20}
  - For 15 puzzle: Time complexity = $\mathbf{10^{13}}$
  - For 24 puzzle: Time Complexity = $\mathbf{10^{24}}$
- For Chess Problem: $b = 35$ and $d = 80\ to\ 90$.
- To convert the problem NP hard to P.

- **What Could be heuristic function????**

Heuristic Functions

"f - function" =

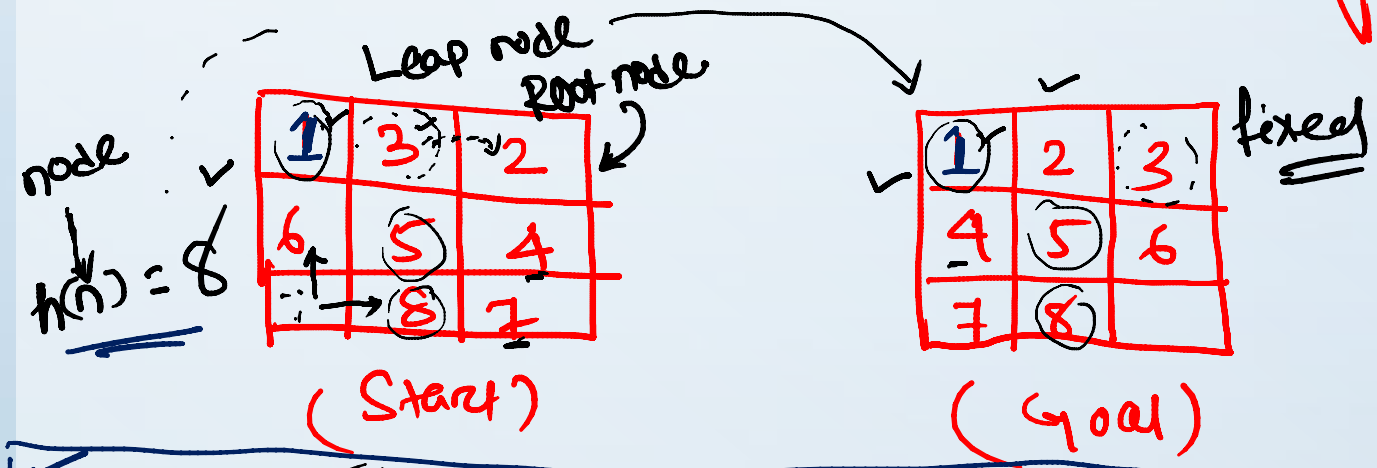
- Euclidean Distance (Straight line distance)
- Manhattan Distance
- Any function with better guess!!!!



"Heuristic function"

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

8 Puzzle problem with heuristic...!



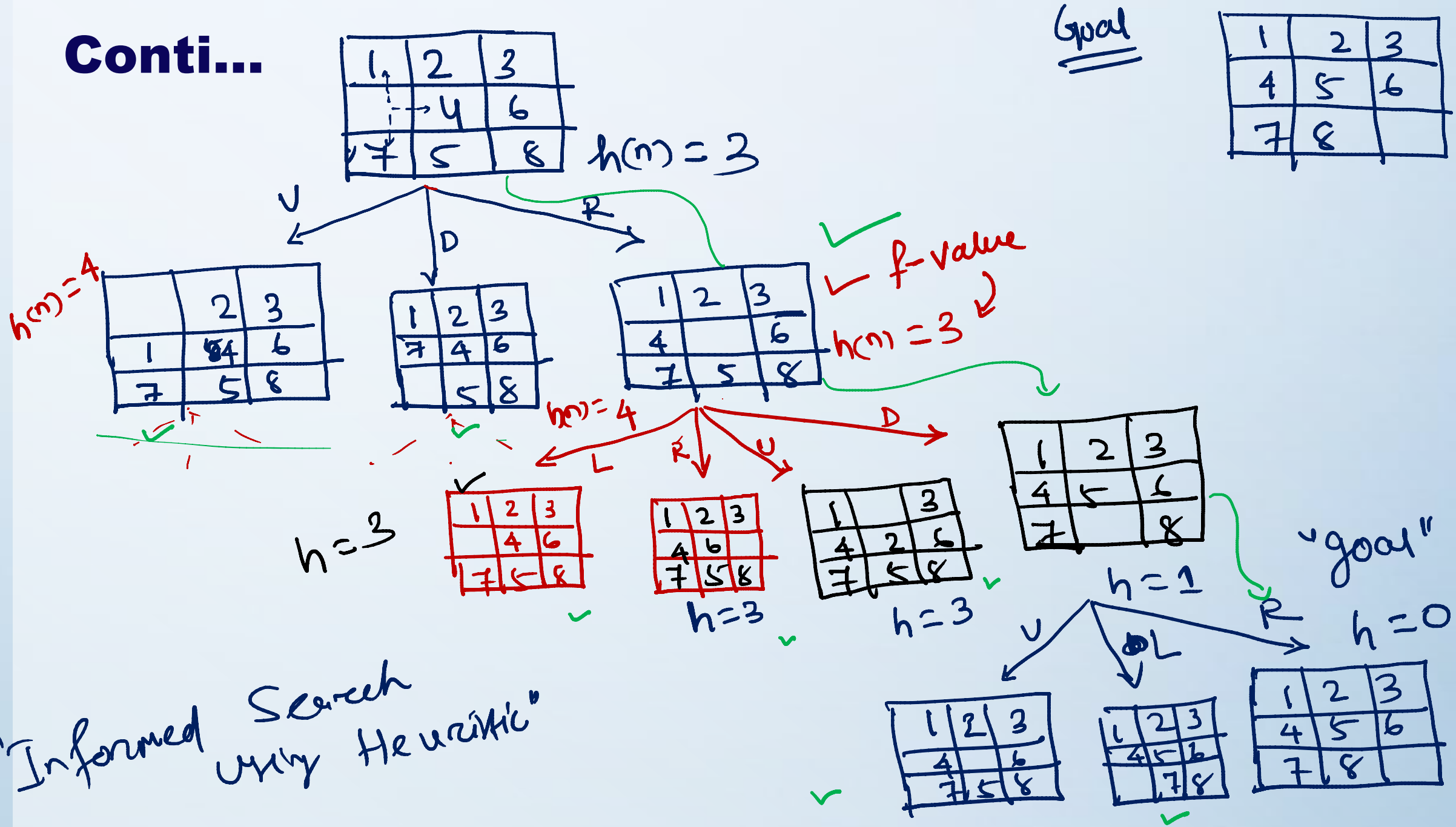
Manhattan Distance:

1 :	0	moves
2 :	1	move
3 :	1	move
4 :	2	move
5 :	0	move
6 :	2	moves
7 :	2	moves
8 :	0	move

Manhattan dist $h(n) = 8$

$h(n)$ = No. of misplaced tiles: = 0+1+1+1+0+1+1+0 = 5

Conti...


Goal
h(n) = 3
U
D
R
h(n) = 4
f-value
h(n) = 3
h(n) = 4
L
R
U
D
h = 3
h = 3
h = 3
h = 1
"goal"
h = 0
U
L
R
"Informed Search
using Heuristic"

Best First (Greedy) Search

- Use an evaluation function f(n) for node n.
- Always choose the node from fringe that has the lowest f value.
  - Define f value in terms of heuristic!
- Idea: use an evaluation function f(n) for each node
  – estimate of "desirability“
  Expand most desirable unexpanded node
- Evaluation function *f(n)* = *h(n)* (heuristic function) = estimate of cost from *n* to *goal*
- e.g., $h_{SLD}(n)$ = straight-line (Euclidean) distance from *n* to Bucharest. (Romania City)
- Greedy best-first search expands the node that appears to be closest to goal

Best First Search

Let *fringe* be a *priority queue* containing the initial state

Loop

if *fringe* is empty return *failure*

Node ← remove-first (*fringe*)

if Node is a *goal*

then return the path from initial state to Node

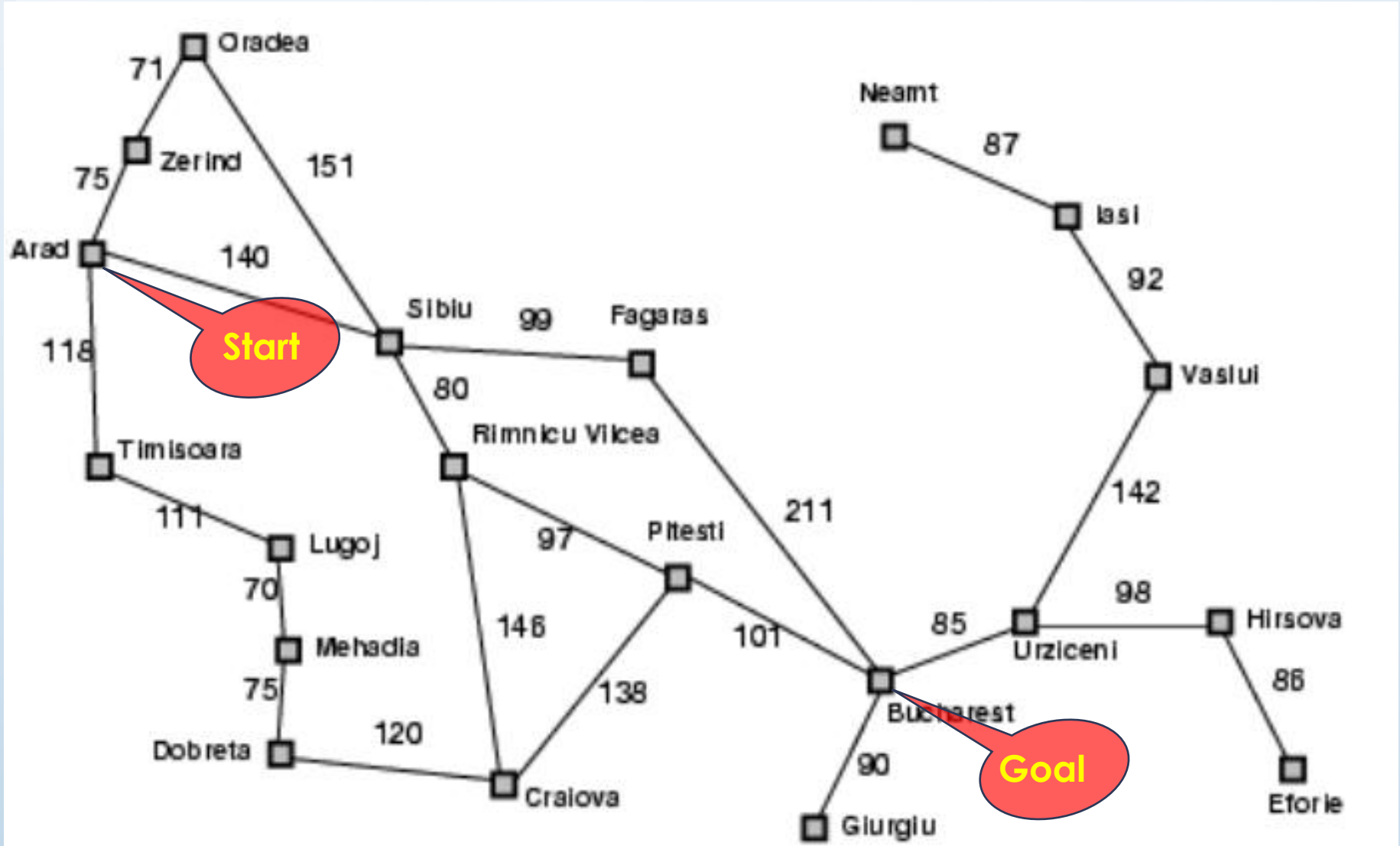
else generate all successors of Node, and

put the newly generated nodes into *fringe*

according to their f values

End Loop

Romania with step costs in km



Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Values of h_{SLD}—straight-line distances to Bucharest. b

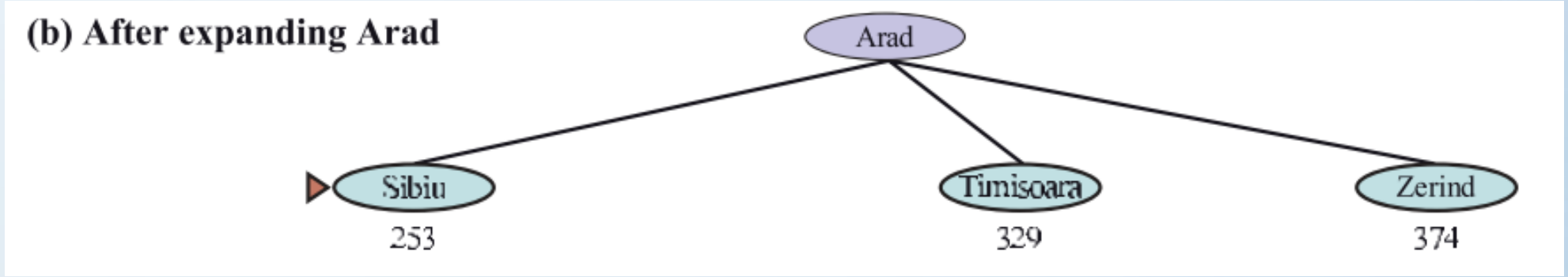
Best First (Greedy) Search Algorithm

```
function tree-search(root-node)
fringe ← successors(root-node)

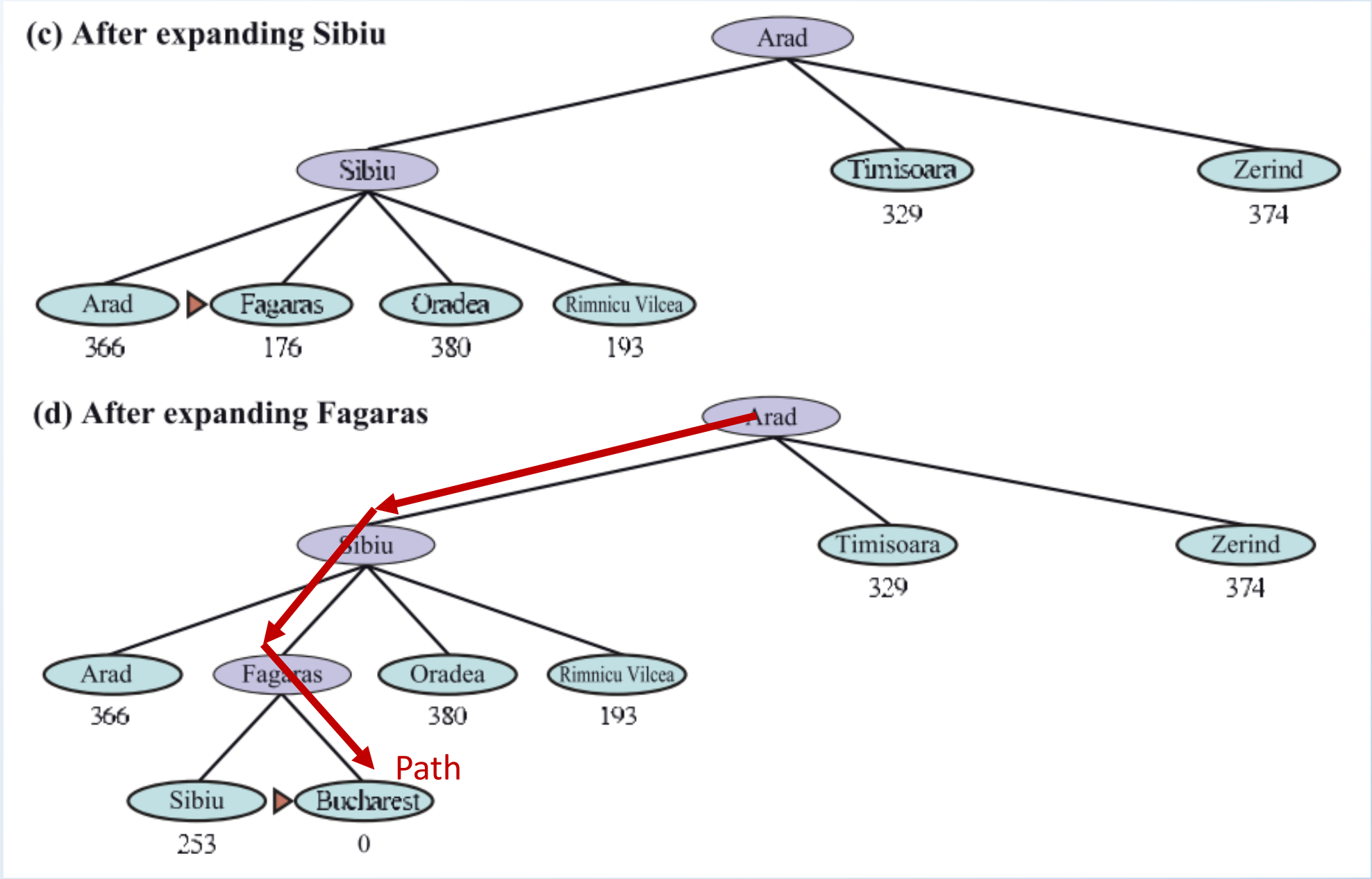
while ( notempty(fringe) )
    {node ← remove-first(fringe) //lowest f value
     state ← state(node)
     if goal-test(state) return solution(node)
     fringe ← insert-all(successors(node),fringe) }
return failure

end tree-search
```





Best First (Greedy) Search Algorithm



Properties of greedy best-first search

- Complete?
  - ✓ No – can get stuck in loops, e.g., Iasi → Neamt → Iasi → Neamt →
- Time?
  - ✓ $O(b^m)$, but a good heuristic can give dramatic improvement
- Space?
  - ✓ $O(b^m)$ -- keeps all nodes in memory
- Optimal?
  - ✓ No

A* search

- Idea: avoid expanding paths that are already expensive

- Evaluation function $f(n) = g(n) + h(n)$

✓ $g(n)$ = cost so far to reach n
✓ $h(n)$ = estimated cost from n to goal
✓ $f(n)$ = estimated total cost of path through n to goal

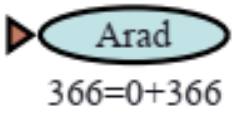
A* search: Algorithm

- **Step1**: Place the starting node in the *OPEN* list.
- **Step 2**: Check if the $OPEN$ list is empty or not, if the list is empty then return failure and stops.
- **Step 3**: Select the node from the $OPEN$ list which has the smallest value of evaluation function $(g + h)$, if $n$ode n is goal node then return success and stop, otherwise
- **Step 4**: Expand node n and generate all of its successors, and put n into the closed list. For each successor n', check whether n' is already in the $OPEN$ or $CLOSED$ list, if not then compute evaluation function for n' and place into Open list.
- **Step 5**: Else if node n' is already in $OPEN$ and $CLOSED$, then it should be attached to the back pointer which reflects the lowest $g(n')$ value.
- **Step 6**: Return to **Step 2**.

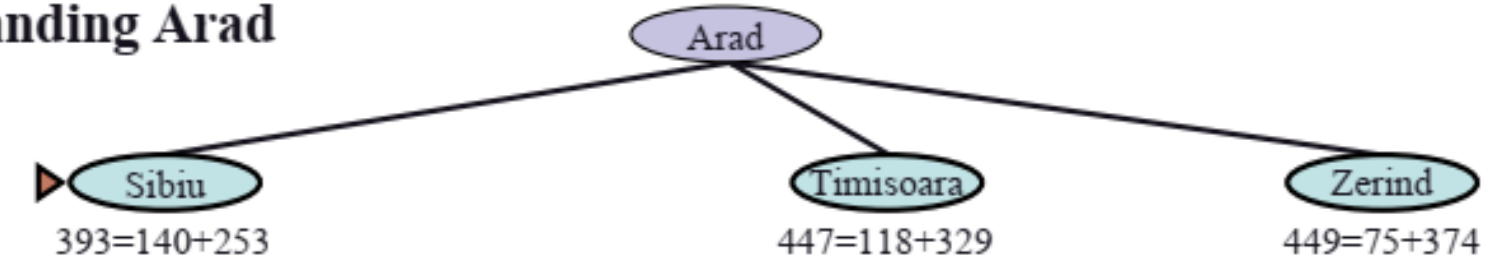
Stages in an A* search for Bucharest

Nodes are labeled with $f = g + h$. The h values are the straight-line distances to Bucharest

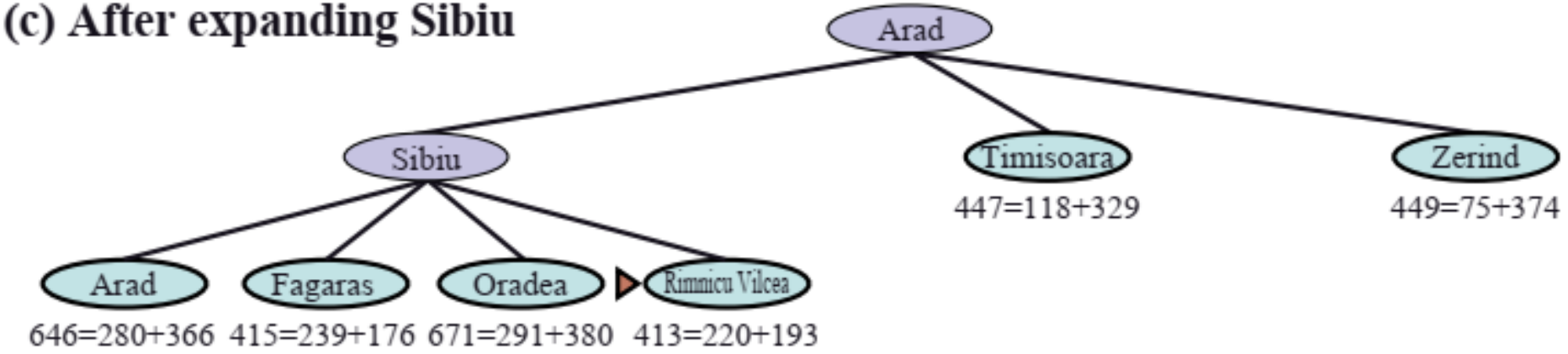
(a) The initial state



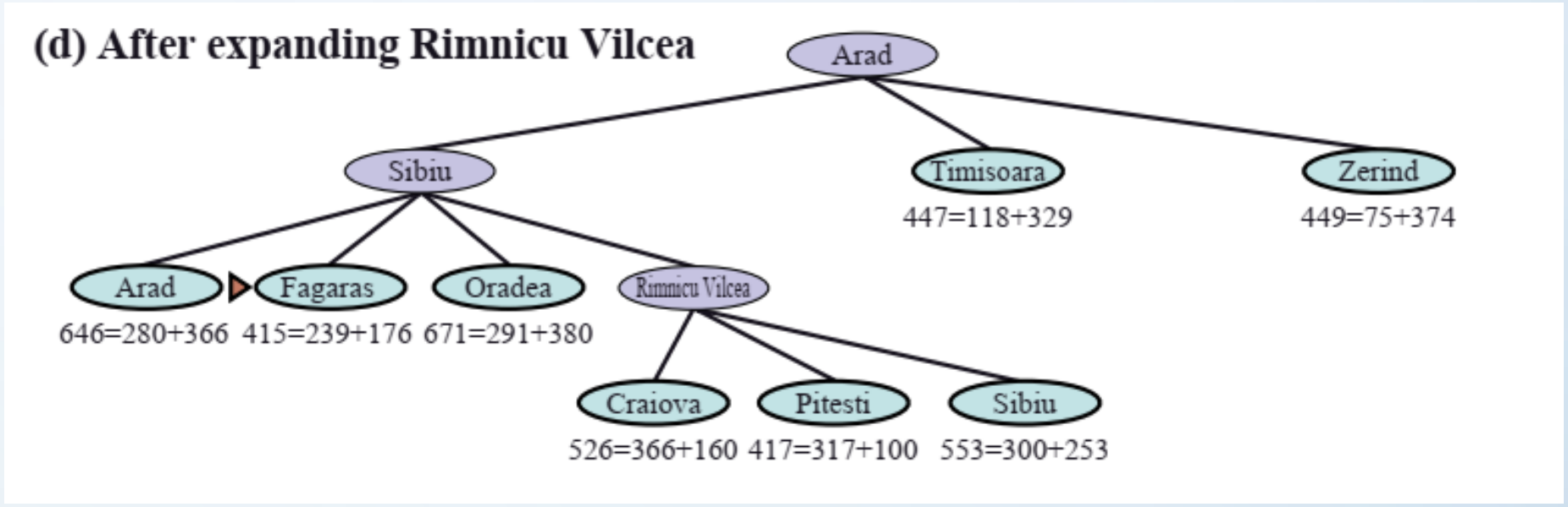
(b) After expanding Arad

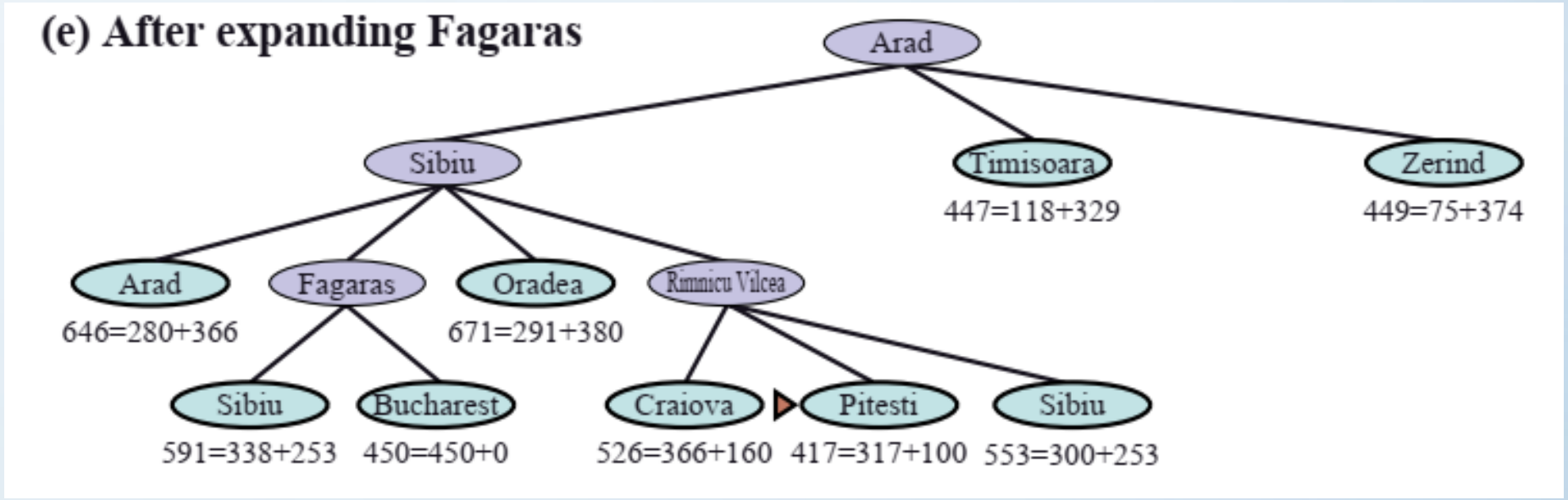


(c) After expanding Sibiu

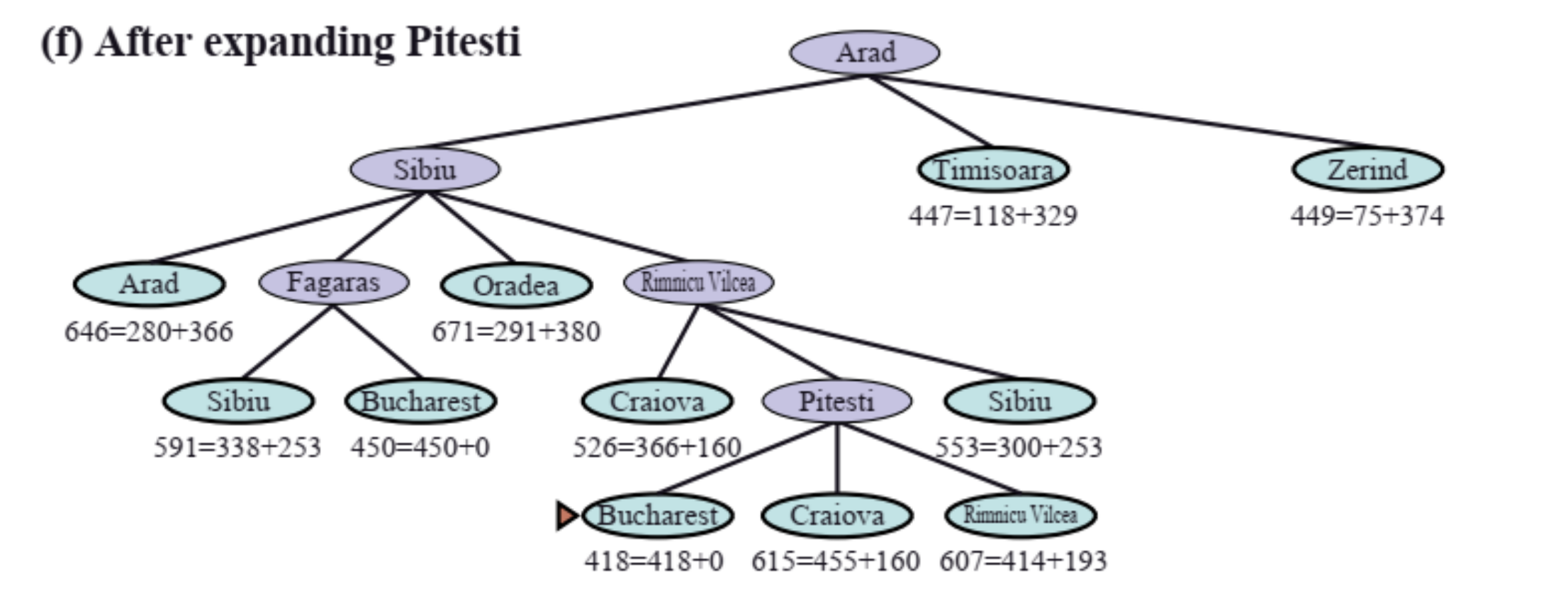


Stages in an A* search for Bucharest





Stages in an A* search for Bucharest



Properties of A*

- Complete?
  Yes (unless there are infinitely many nodes with $f \leq f(G)$)
- Time? Exponential (worst case all nodes are added)
- Space? Keeps all nodes in memory
- Optimal?
  Yes (depending upon search algorithm and heuristic property)